



```
def deploy_pipeline():
    project = "AI"
    env = "production"

    > DeployResult:
        agent = CodingAgent(model="gpt-4o")
        await agent.analyze_project()
        tests = await agent.run_tests()
        if tests.passed:
            return await agent.deploy(env)

def pipeline = nx.AIPipeline(
    model="Claude-sonnet",
    tools=["read", "write", "browser"],
    max_iterations=50
)

await pipeline.execute()
# Resultado: Sistema completo
# de gestão de pedidos com
# testes unitários e deploy
```

— A PERICIA FORENSE DE CONFIGURACOES —

TOKENS SOB PERICIA

Guia verificado de economia de tokens em IDEs agenticas

- ◆ PARA INICIANTES
- 16 capítulos
- projetos práticos

Heverton Eduardo Peres

Tokens Sob Perícia

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

São Paulo
2026

Dados Internacionais de Catalogação na Publicação (CIP)

P542t PERES, Heverton Eduardo

Tokens Sob Perícia / Heverton Eduardo Peres. – São Paulo : Fábrica
Agêntica de Livros,
2026.

109 p. ; 21 cm.

ISBN 978-65-00000-20-7

1. Perícia. 2. Manual. 3. Comandos. 4. Real. I. Título.

CDD 006.3

Sumário

Capítulo 1: O Manual Suspeito: Por Que Comandos Bonitos Enganam	7
1. Introdução	7
2. Explica	7
3. Ilustra	9
4. Técnica	11
Diagnosticando os dois casos de abertura	11
Formalizando a árvore de decisão	12
Registrando a classe da fonte no laudo	14
O padrão se repete em todo o manual	14
5. Aplica	15
Exercício	16
6. Conclusão	17
7. Referências Bibliográficas	17
Capítulo 2: Prompt Caching de Verdade: Como o Provedor Cobra Menos por Repetição ...	20
1. Introdução	20
2. Explica	20
3. Ilustra	21
4. Técnica	22
Onde a configuração de verdade mora, por ferramenta	23
Verificando a estabilidade do prefixo com sha256sum	25
As outras seis frentes: Hermes e o restante do catálogo	26
Medindo cache hit de verdade	26
5. Aplica	27
Exercício	29
6. Conclusão	29
7. Referências Bibliográficas	30
Capítulo 3: Modelos Gratuitos Sem Lenda: o Catálogo Real Por Trás do Gateway	32
1. Introdução	32
2. Explica	32
3. Ilustra	33
4. Técnica	35
A configuração real: opencode.json	35
O erro que você não vai cometer: config fabricada vs. config real	35
O protocolo de verificação de catálogo (aplicável a qualquer CLI nova) ..	36

Detectando quando o catálogo mudou	38
Bônus: o script de benchmark que já estava certo	38
5. Aplica	39
6. Conclusão	41
7. Referências Bibliográficas	41
Capítulo 4: Compressão e Cache Semântico Sob a Lupa: Bibliotecas, Não Comandos	44
1. Introdução	44
2. Explica	44
3. Ilustra	45
4. Técnica	48
A Classe Certa: PromptCompressor em Vez de LlmLingua	48
Por Que pipx Não Cria um Comando Que Não Existe	49
O Hook de Cache Semântico Completo (semantic_cache.py)	50
A Segunda Forma Real de Configurar o Limiar: Objeto Config	51
O Modo Servidor: gptcache_server, Para Quando Você Não Quer Embutir o Cache no Script	52
Onde Isso Se Encaixa na Cadeia Real de Hooks	52
Blindando o Wrapper Contra Falha em Cascata	53
Observando o Resultado em Produção	53
5. Aplica	53
Exercício	54
6. Conclusão	55
7. Referências Bibliográficas	55
Capítulo 5: Paralelismo e Resiliência Que Sempre Funcionaram	57
1. Introdução	57
2. Explica	57
3. Ilustra	59
4. Técnica	60
Paralelismo Controlado em Python: semáforo Antes de Disparar Tudo ...	60
O Disjuntor Que Já Existia: Circuit Breaker Testável	62
Backoff Exponencial Com Jitter: Espaçando as Novas Tentativas	63
Combinando os Três Padrões em Uma Única Chamada	64
Onde Isso Se Encaixa em Cada IDE Que Você está Auditando	65
5. Aplica	66
6. Conclusão	67
7. Referências Bibliográficas	67
Capítulo 6 — Orçamento e Fallback: Medir Antes de Limitar	70
1. Introdução	70
2. Explica	70
3. Ilustra	72
4. Técnica	74

Onde o Orçamento Vive em Cada IDE (Além do Claude Code/Codex) ...	77
Alternativa ao Cron: Acoplando o Guard a um Hook do Claude Code	77
5. Aplica	78
6. Conclusão	79
7. Referências Bibliográficas	80
Capítulo 7: O Agente Avançado e as Outras Sete Ferramentas: Produto Real, Sintaxe	
Fabricada	83
1. Introdução	83
2. Explica	84
3. Ilustra	86
4. Técnica	87
Checklist de Terminal: <code>hermes_pericia.sh</code>	87
Tabela de Campo: As Outras Sete Ferramentas	88
Regra Geral: <code>verificar_cli</code> Antes de Automatizar	89
5. Aplica	92
Exercício	93
6. Conclusão	93
7. Referências Bibliográficas	94
Capítulo 8: O Perito Permanente: Seu Protocolo Para Qualquer Manual Novo	96
1. Introdução	96
2. Explica	96
3. Ilustra	99
4. Técnica	101
O Protocolo de perícia em Formato Portátil	101
Restaurando o Cofre Confirmado com <code>chezmoi</code> (Sintaxe Real)	101
O Script de Alerta Rápido	103
Blindagem Extra: Grep de Segredo Antes do Commit	104
5. Aplica	105
Exercício	106
6. Conclusão	106
7. Referências Bibliográficas	107

Capítulo 1: O Manual Suspeito: Por Que Comandos Bonitos Enganam

1. Introdução

Um manual de 15 laboratórios chega até você prometendo cortar o gasto de tokens das suas IDEs agênticas — Claude Code, OpenCode, Aider, Codex e mais uma dezena de ferramentas. Ele tem tom técnico, comandos formatados, caminhos de arquivo exatos, nomes de modelo específicos. Tudo nele parece ter passado por um revisor experiente. E é exatamente essa aparência de rigor que faz o operador baixar a guarda e colar o primeiro bloco de código no terminal sem checar nada.

Este capítulo abre o processo pericial que sustenta o livro inteiro. Você vai ver, lado a lado, dois casos reais em que uma ferramenta genuína foi descrita com uma sintaxe que nunca existiu — e vai sair daqui com o método de verificação que vai aplicar, capítulo após capítulo, a cada comando, config e nome de modelo que cruzar seu caminho. Ao final, você não vai mais ler manuais técnicos como consumidor: vai lê-los como perito.

2. Explica

O erro central deste manual não é o que a maioria dos leitores esperaria. Não é uma lista de ferramentas inventadas do zero — boa parte das ferramentas citadas existe de verdade. O erro sistemático é outro, mais sutil e mais perigoso: **ferramenta real, sintaxe fabricada**. O nome do produto é autêntico; o comando exato para operá-lo foi escrito de memória, por aproximação, sem checagem final contra a fonte que o próprio fornecedor pública.

Pegue o primeiro caso, logo no laboratório de setup do manual: o comando `pipx install ccusage`. O `ccusage` existe — é uma ferramenta real e amplamente usada para analisar o consumo de tokens do Claude Code a partir dos arquivos JSONL que a própria ferramenta já grava localmente [1]. O problema é que `ccusage` é um pacote **Node/npm**, não um pacote Python — e `pipx` só instala pacotes Python. O comando do manual simplesmente não

tem como funcionar: o pacote nunca vai aparecer no índice que o `pipx` consulta. A forma real e documentada de rodar a ferramenta é `npv ccusage@latest <subcomando>`, sem instalação prévia necessária — o mesmo caminho confirmado por fontes independentes que documentam o uso diário da ferramenta, sem qualquer menção a `pipx` [2]. Repare no padrão: a ferramenta é 100% real, a confiança que o nome desperta é justificada — só a sintaxe de instalação está errada.

O segundo caso aparece pouco depois, e vale entender por que ele dói tanto: o cache de prompt real da Anthropic pode cortar até 90% do custo de leitura do prefixo repetido — tecnicamente, a leitura de cache custa 0,1x o preço padrão de input [3]. É um mecanismo caro demais para ficar inoperante por causa de um caminho de arquivo errado. Ainda assim, é exatamente isso que o manual descreve: configurar esse cache em um arquivo `~/.config/claude-code/settings.json`, com campos `systemPrompt` e `cacheControl`. Nenhuma parte disso existe. O caminho real, documentado pela própria Anthropic, é `~/.claude/settings.json` (configuração de usuário) ou `.claude/settings.json / .claude/settings.local.json` (configuração de projeto) [4]. Os campos aceitos nesse arquivo incluem `model`, `permissions`, `env`, `hooks` e outras dezenas de chaves — mas não existe `cacheControl` nem `systemPrompt` expostos ao usuário nesse schema. O cache de prompt do Claude Code é automático, gerenciado internamente pela ferramenta junto da API; não há toggle manual para ele. Se o operador copiar esse trecho e colocá-lo no arquivo certo, o Claude Code simplesmente ignora as chaves desconhecidas — falha silenciosa, sem mensagem de erro, sem cache extra nenhum sendo ativado.

Esse é o padrão que se repete, com pequenas variações, ao longo de todo o manual. Nesta perícia — a auditoria própria que sustenta esta obra —, mapeamos 61 itens tecnicamente verificáveis do documento inteiro contra a documentação oficial de cada fornecedor: 26 bateram exatamente com a fonte primária (CONFIRMADO), 13 descreviam uma ferramenta real com nome de comando ou caminho de arquivo errado (PARCIALMENTE CORRETO), 18 eram fabricação pura (FABRICADO) e 4 não puderam ser confirmados nem refutados por falta de fonte pública (NÃO VERIFICÁVEL). O núcleo técnico do manual é genuíno e vale a pena aprender — é por isso que este livro existe, em vez de simplesmente descartar o material inteiro. O que falta é o filtro.

Vale demorar-se num detalhe que a contagem acima esconde: a hierarquia de fontes deste livro tem três degraus, não dois. O primeiro — fonte classe A — é a documentação oficial publicada pelo próprio fornecedor do produto, como o guia de `settings.json` da Anthropic. O segundo — classe B — é o repositório ou pacote oficial do projeto, quando não existe um manual de referência formal, mas o código-fonte e o `README` do próprio mantenedor já bastam para confirmar a sintaxe, como o repositório do `ccusage` no GitHub. O terceiro degrau — classe C — é o material de terceiros: post de blog, tutorial independente, guia de comunidade. Uma fonte classe C nunca decide sozinha um veredicto de CONFIRMADO; ela serve como corroboração adicional quando já existe A ou B, e como pista provisória quando não existe nenhuma das duas. No caso do `ccusage`, o guia independente do ClaudeLog [2] é exatamente

isso: uma fonte classe C que reforça o que o repositório oficial (classe B) já mostrava, nunca a autoridade que decidiu o veredicto sozinha. Confundir uma fonte classe C bem escrita — com capturas de tela, tom técnico seguro, formatação impecável — com uma fonte classe A é o mesmo erro de fundo que abre este capítulo: aparência de rigor substituindo verificação de fato.

3. Ilustra

Pense em como um perito documental trabalha. Um documento chega à mesa dele com **letra timbrada** de uma instituição conhecida — logotipo, papel timbrado, assinatura no rodapé. A reação automática de quem não é perito é aceitar: “reconheço essa instituição, então o documento deve ser válido”. O perito faz o oposto: reconhecer a instituição é só o primeiro passo. O que ele confirma de fato é se **aquela assinatura específica** bate com a assinatura de referência arquivada, se o número do documento existe no cadastro oficial, se o timbre não foi escaneado e colado por cima de outro texto. Como **Perito de Configuração Agêntica**, seu trabalho com um manual técnico é idêntico: reconhecer o nome da ferramenta (`ccusage` , `Claude Code` , `Hermes`) é só a letra timbrada. A perícia de verdade é conferir se o comando específico bate com a fonte primária.

Essa é a primeira camada da analogia — a mecânica geral de desconfiar da aparência. Mas o ponto mais difícil deste capítulo pede uma segunda camada, porque ele contraria a intuição: um comando classificado como **NÃO VERIFICÁVEL** não é o mesmo que um comando **aprovado**. Pense num inquérito policial em que não se encontra prova suficiente para indiciar ninguém: o inquérito não vira, por causa disso, um atestado de inocência — ele simplesmente permanece aberto, pendente de mais evidência. Da mesma forma, quando você não encontra nenhuma fonte primária pública que confirme uma variável de ambiente ou uma flag de CLI, a conclusão correta não é “deve estar certo, ninguém provou o contrário” — é “fica pendente até eu conseguir testar com `--help` na minha própria máquina”. Tratar ausência de prova como prova de inocência é exatamente o erro que deixa comando fabricado passar despercebido.

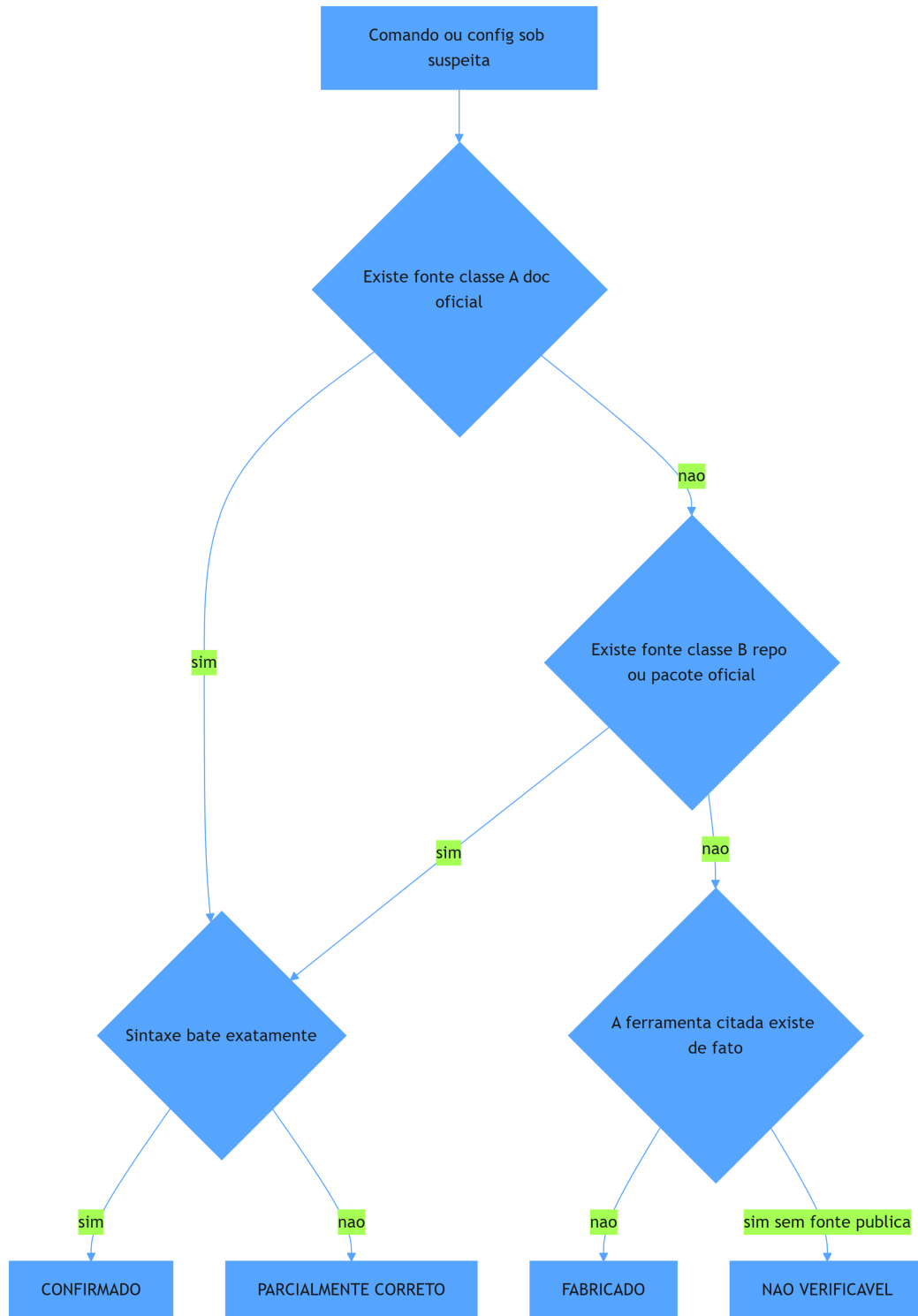


Figura 1: Protocolo de pericia — hierarquia de fontes A/B/C e os 4 veredictos possíveis

O diagrama acima é o laudo-padrão que você vai preencher, mentalmente ou por escrito, para cada comando que aparecer no resto deste livro — e no resto da sua carreira lidando com manuais de terceiros.

4. Técnica

A perícia não é um exercício de opinião — ela vira código quando você a transforma em checklist reproduzível. Abaixo estão os dois artefatos que sustentam o método deste capítulo: um script de diagnóstico para os dois casos apresentados na Introdução, e a função de classificação que formaliza a árvore de decisão da seção anterior.

Diagnosticando os dois casos de abertura

O primeiro artefato não instala nada e não depende de rede — ele apenas expõe, de forma segura, por que o comando do manual falha e qual é o comando real:

```
#!/usr/bin/env bash
# Diagnostico dos dois casos de abertura do Capitulo 1.
# Nao depende de rede: usa apenas checagens locais (command -v, teste de
path).

echo "=== CASO 1: instalar o ccusage ==="
echo "Comando do manual (fabricado): pipx install ccusage"
if command -v pipx >/dev/null 2>&1; then
    echo "pipx esta presente, mas ccusage e' um pacote Node/npm -- pipx so
instala pacotes Python."
    echo "O comando do manual falharia com 'No matching distribution found'."
else
    echo "pipx nao esta instalado neste ambiente; de qualquer forma o comando
falharia (pacote errado)."
fi

echo "Comando real (documentado pelo projeto): npx ccusage@latest daily"
if command -v npx >/dev/null 2>&1; then
    echo "npx disponivel -- este e' o caminho real, sem instalacao previa
necessaria."
else
    echo "npx nao esta instalado neste ambiente, mas e' o comando correto
segundo a documentacao do projeto."
fi

echo ""
echo "=== CASO 2: onde mora a config de cache do Claude Code ==="
CAMINHO_FABRICADO="$HOME/.config/claude-code/settings.json"
CAMINHO_REAL="$HOME/.claude/settings.json"

echo "Caminho citado pelo manual (fabricado): ${CAMINHO_FABRICADO}"
echo "Caminho real documentado pela Anthropic:  ${CAMINHO_REAL}"
```

```

if [ -f "${CAMINHO_REAL}" ]; then
    echo "Arquivo real encontrado. Campos validos: model, permissions, env,
hooks -- nao cacheControl."
else
    echo "Arquivo real ainda nao existe neste ambiente, mas o caminho e' o
unico documentado oficialmente."
fi

```

Rodar esse tipo de script antes de automatizar qualquer coisa é barato e determinístico: ele não depende de você “confiar” em nada, só de checar o que já existe no seu próprio ambiente contra o que a documentação afirma.

Formalizando a árvore de decisão

O segundo artefato transforma o fluxograma da seção Ilustra em uma função que você pode reaproveitar — literalmente — em qualquer manual novo que chegar até você daqui em diante:

```

"""
Perito de Configuracao Agentica -- classificador de evidencia (Capitulo 1).

Checklist antes de aceitar qualquer comando/config de um manual de
terceiros:
    1. Existe documentacao oficial do proprio fornecedor (fonte classe A)?
    2. Existe repositorio ou pacote oficial do projeto -- GitHub/PyPI/npm
(fonte classe B)?
    3. A sintaxe do manual bate exatamente com o que a fonte A ou B
documenta?
    4. Antes de automatizar, rode "<comando> --help" (ou equivalente) e
compare
        com o que o manual afirma -- nunca depois de automatizar.
"""

from dataclasses import dataclass

@dataclass
class Evidencia:
    existe_doc_oficial: bool      # fonte classe A
    existe_repo_pacote: bool      # fonte classe B
    ferramenta_existe: bool      # a ferramenta/produto citado existe de
fato
    bate_com_manual: bool         # a sintaxe do manual confere com a fonte
encontrada

```

```

def classificar_fonte(evidencia: Evidencia) -> str:
    """Aplica a hierarquia de fontes A/B/C e devolve um dos 4 veredictos.

    Regra central deste capítulo: NAO_VERIFICAVEL nunca e' sinonimo de
    aprovado. Sem fonte primaria (classe A ou B), o comando fica pendente
    ate confirmacao manual -- nunca entra em producao so porque "parece
    plausivel".
    """

    tem_fonte_primaria = evidencia.existe_doc_oficial or
evidencia.existe_repo_pacote

    if not tem_fonte_primaria:
        if evidencia.ferramenta_existe:
            return "NAO_VERIFICAVEL"
        return "FABRICADO"

    return "CONFIRMADO" if evidencia.bate_com_manual else
"PARCIALMENTE_CORRETO"

if __name__ == "__main__":
    casos = {
        "pipx install ccusage": Evidencia(
            existe_doc_oficial=False,
            existe_repo_pacote=True,    # ccusage existe, mas via npm, nao
pipx
            ferramenta_existe=True,
            bate_com_manual=False,
        ),
        "npx ccusage@latest daily": Evidencia(
            existe_doc_oficial=False,
            existe_repo_pacote=True,
            ferramenta_existe=True,
            bate_com_manual=True,
        ),
        "~/.config/claude-code/settings.json com cacheControl": Evidencia(
            existe_doc_oficial=True,    # existe doc oficial de settings do
Claude Code
            existe_repo_pacote=False,
            ferramenta_existe=True,
            bate_com_manual=False,    # caminho e campo nao batem com a
doc real
        ),
    }

    for comando, evidencia in casos.items():

```

```
veredicto = classificar_fonte(evidencia)
classe = classe_predominante(evidencia)
print(f"{comando}: {veredicto} (fonte classe {classe})")
```

Registrando a classe da fonte no laudo

O classificador `classificar_fonte` decide entre os quatro veredictos, mas ele esconde uma informação que vale registrar explicitamente em qualquer laudo real: qual classe de fonte sustentou a decisão. A função abaixo formaliza o terceiro degrau da hierarquia — a fonte classe C, apresentada na seção Explica — como um rótulo auditável, sem alterar o veredicto final calculado por `classificar_fonte`:

```
def classe_predominante(evidencia: Evidencia) -> str:
    """Retorna o rotulo de classe (A/B/C) que sustenta o veredicto.

    Uma fonte classe C (blog, guia de comunidade) nunca aparece aqui como
    suficiente sozinha -- ela so recebe o rotulo "C" quando nem A nem B
    existem, para deixar explicito no laudo que aquela evidencia e'
    corroborativa, nunca primaria.
    """
    if evidencia.existe_doc_oficial:
        return "A"
    if evidencia.existe_repo_pacote:
        return "B"
    return "C" if evidencia.ferramenta_existe else "sem_fonte"
```

Aplicado aos três casos deste capítulo, o rótulo de classe expõe uma assimetria que o veredicto sozinho esconde: tanto `pipx install ccusage` quanto `npm ccusage@latest daily` apoiam-se em classe B — o próprio repositório do projeto, sem manual de referência formal — e a diferença entre PARCIALMENTE_CORRETO e CONFIRMADO não vem da classe da fonte, vem de o comando bater ou não com o que essa fonte documenta. Já o terceiro caso, o `settings.json` do Claude Code, apoia-se em classe A — documentação oficial da Anthropic — e ainda assim erra por caminho e campo, prova de que nem toda fonte classe A impede erro humano na hora de aplicar o que ela diz. Guardar essa distinção no laudo evita um vício comum: tratar “a fonte é boa” e “o comando bate com a fonte” como se fossem a mesma pergunta.

O padrão se repete em todo o manual

O mesmo par “produto real, sintaxe imprecisa” reaparece em praticamente todo laboratório do manual. Isso reforça por que este não é um problema isolado de dois comandos, mas um padrão sistemático que os próximos sete capítulos vão dissecar um a um, tema por tema. O gateway de modelos gratuitos do OpenCode, chamado Zen, existe de fato [5]. O que não existe é a

tabela de sete modelos com nomes como `opencode-zen-free-medium`: o catálogo real segue o padrão `opencode/<model-id>` e muda de composição com frequência, então não vale a pena memorizar nomes fixos [6]. As bibliotecas de compressão de prompt e cache semântico seguem o mesmo roteiro. LLMingua é um pacote Python genuíno, mantido pela Microsoft Research [7], e também está publicado oficialmente no índice do Python [8]. GPTCache também é real e bem documentado — mas nenhuma das duas ferramentas tem uma CLI de terminal com os subcomandos que o manual descreve; ambas são operadas via API Python [9]. `agenttrace`, hospedado no PyPI, é real [10]. `agentlytics`, hospedado no npm, também é real, mas é um projeto totalmente diferente, mantido por outra equipe [11]. O manual trata os dois como se fossem o mesmo pacote instalado de duas formas.

Nem tudo no manual está errado, e vale destacar isso com a mesma precisão que se aplica ao erro. `asyncio.gather` com semáforo é usado exatamente como a biblioteca padrão do Python documenta [12]. O mesmo vale para o GNU parallel, aplicado com a sintaxe real de paralelismo de shell [13]. O circuit breaker também é descrito sem erro, seguindo o padrão de arquitetura de resiliência amplamente documentado [14]. O backoff exponencial com jitter para retry em rate limit segue a mesma régua de precisão [15]. O fallback para modelos locais via Ollama, incluindo a lista de modelos testados, também confere com a biblioteca oficial [16]. A única ressalva de nomenclatura nesse bloco é que o binário do `llama.cpp` foi renomeado para `llama-cli` [17], e que o vLLM usa `vllm serve` em vez do módulo `python -m vllm.server` citado pelo manual [18]. Até o Hermes Agent, que a primeira leitura deste dossiê suspeitou ser uma IDE inventada, revelou-se um produto real da NousResearch, com CLI, memória e cron scheduler documentados [19]. Só a sintaxe exata dos comandos de skill está incorreta [20], assim como a dos comandos de memória e sessão [21]. Fecham este levantamento dois casos quase perfeitos: o `chezmoi`, ferramenta de dotfiles usada no capítulo final deste livro, está quase inteiramente correto, com uma única ressalva no instalador legado [22]; e o 1Password CLI, citado no mesmo laboratório, bate exatamente com a sintaxe documentada pelo fornecedor, sem nenhuma correção necessária [23].

5. Aplica

Você acabou de configurar um notebook novo. O manual está aberto ao lado do terminal, e você segue o roteiro do Laboratório 4: criar um hook de pré-processamento em `~/.config/claude-code/hooks.json` para acionar o cache semântico do GPTCache antes de cada chamada. Você cria o diretório, escreve o JSON com o campo `preProcess` exatamente como o manual mostra, salva, reinicia o Claude Code e... nada muda. Sem erro. Sem aviso. O cache simplesmente nunca é chamado, e você gasta a tarde seguinte tentando descobrir por que sua “otimização” não fez diferença nenhuma na fatura.

O diagnóstico, à luz da seção Explica deste capítulo, é direto: hooks do Claude Code não vivem em um arquivo separado `hooks.json` — eles vivem dentro do próprio `~/.claude/settings.json`, na chave `hooks`, amarrados a eventos nomeados como `PreToolUse` e `PostToolUse` [24]. O arquivo que você criou é, para a ferramenta, um arquivo qualquer sem significado nenhum — por isso o silêncio, sem erro. A correção tem duas partes: mover a configuração de hook para dentro do `settings.json` real, usando o evento correto, e — separadamente — implementar o cache semântico como o que ele de fato é. O GPTCache é uma biblioteca Python chamada programaticamente (`cache.init(...)`), nunca um serviço acionado por um hook de terceiro que não existe com esse nome [9].

Esse tipo de falha silenciosa é o risco central de todo o método deste livro, e por isso a perícia tem um limite conhecido que você precisa carregar consigo: catálogos de modelo mudam de uma semana para a outra (o catálogo Zen do OpenCode é o exemplo mais volátil), produtos lançados depois de qualquer corte de conhecimento exigem reconferência pontual, e mesmo uma fonte classe A pode estar temporariamente desatualizada em relação à versão que você instalou localmente. A perícia reduz o risco de forma drástica — não o elimina. “Fonte classe A” é o melhor sinal disponível no momento da checagem, nunca uma garantia perpétua.

Armadilhas recorrentes que valem registrar antes de avançar:

- Confiar no nome da ferramenta como se fosse prova da sintaxe exata do comando.
- Tratar “não encontrei fonte que desminta” como “está confirmado”.
- Copiar caminho de arquivo de configuração sem abrir a documentação oficial da versão instalada.
- Automatizar (cron, script, hook) antes de rodar `--help` uma única vez que seja.

Erro comum vs. prática correta, aplicado à fonte classe C: encontrar um post de blog bem escrito, com prints de tela e comandos formatados, e tratá-lo como equivalente à documentação oficial é o erro mais comum entre operadores que já ouviram falar em “verificar a fonte”, mas nunca aplicaram a hierarquia até o fim — o erro comum é parar na pergunta “essa fonte parece confiável?”. A prática correta troca essa pergunta por outra, mais estreita: essa fonte é o próprio fornecedor (classe A), o repositório ou pacote oficial do projeto (classe B), ou alguém de fora comentando sobre o produto (classe C)? Um post de blog pode estar certo — o ClaudeLog está, por exemplo [2] — mas ele só vira evidência suficiente quando corrobora uma fonte A ou B já encontrada, nunca quando é a única fonte disponível para confirmar uma flag de CLI ou um caminho de arquivo de configuração.

Exercício

- ☐ Escolha 1 comando de qualquer manual, tutorial ou post de blog que você usou nas últimas semanas sem checar a fonte oficial
- ☐ Rode `<comando> --help` (ou o equivalente da ferramenta) e compare linha a linha com o que você havia copiado

- ☐ Classifique o comando usando os 4 veredictos deste capítulo: CONFIRMADO, PARCIALMENTE CORRETO, FABRICADO ou NÃO VERIFICÁVEL
- ☐ Se for PARCIALMENTE CORRETO ou FABRICADO, escreva a versão corrigida ao lado da original, com a fonte primária anotada

6. Conclusão

Três pontos sustentam tudo que vem a seguir neste livro: primeiro, o erro mais perigoso de um manual técnico não é a fabricação total, mas a ferramenta real vestida de sintaxe inventada — porque o nome familiar desarma exatamente a checagem que evitaria o problema. Segundo, existe um protocolo reproduzível para separar as duas coisas: hierarquia de fontes classe A/B/C, os quatro veredictos (CONFIRMADO, PARCIALMENTE CORRETO, FABRICADO, NÃO VERIFICÁVEL) e a regra de nunca tratar ausência de prova como aprovação. Terceiro, esse protocolo já foi aplicado às 61 afirmações verificáveis do manual auditado — o resultado é o mapa que guia cada um dos sete capítulos seguintes.

No Capítulo 2, você vai aplicar esse mesmo laudo pericial ao mecanismo mais lucrativo do manual — o cache de prompt — e vai descobrir exatamente onde o desconto real de 90% se configura [3], e onde o manual inventou um caminho de arquivo que nunca existiu.

7. Referências Bibliográficas

[1] RYOPPIPI. *ccusage: A CLI tool for analyzing Claude Code/Codex CLI usage from local JSONL files*. Disponível em: <https://github.com/ryoppiippi/ccusage>. Acesso em: 20 ago. 2026.

[2] CLAUDELOG. *What is ccusage tool for Claude Code*. Disponível em: <https://claudelog.com/faqs/what-is-ccusage-tool/>. Acesso em: 20 ago. 2026.

[3] ANTHROPIC. *Prompt caching*. Disponível em: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>. Acesso em: 20 ago. 2026.

[4] ANTHROPIC. *Claude Code settings*. Disponível em: <https://code.claude.com/docs/en/settings>. Acesso em: 20 ago. 2026.

[5] OPENCODE. *Zen*. Disponível em: <https://opencode.ai/docs/zen/>. Acesso em: 20 ago. 2026.

[6] OPENCODE. *Config*. Disponível em: <https://opencode.ai/v2/docs/config>. Acesso em: 20 ago. 2026.

[7] MICROSOFT. *LLMLingua* (repositório oficial). Disponível em: <https://github.com/microsoft/LLMLingua>. Acesso em: 20 ago. 2026.

- [8] PYTHON PACKAGE INDEX. *llmlingua*. Disponível em: <https://pypi.org/project/llmlingua/>. Acesso em: 20 ago. 2026.
- [9] ZILLIZTECH. *GPTCache: A Library for Creating Semantic Cache for LLM Queries* (docs/usage.md). Disponível em: <https://github.com/zilliztech/GPTCache/blob/main/docs/usage.md>. Acesso em: 20 ago. 2026.
- [10] TENSORSTAX. *agenttrace: lightweight observability library to trace and evaluate agentic systems*. Disponível em: <https://github.com/tensorstax/agenttrace>. Acesso em: 20 ago. 2026.
- [11] F. *agentlytics: Comprehensive analytics dashboard for AI coding agents*. Disponível em: <https://github.com/f/agentlytics>. Acesso em: 20 ago. 2026.
- [12] PYTHON SOFTWARE FOUNDATION. *asyncio — Asynchronous I/O*. Disponível em: <https://docs.python.org/3/library/asyncio.html>. Acesso em: 20 ago. 2026.
- [13] GNU. *GNU Parallel*. Disponível em: <https://www.gnu.org/software/parallel/>. Acesso em: 20 ago. 2026.
- [14] MICROSOFT. *Circuit Breaker pattern — Azure Architecture Center*. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker>. Acesso em: 20 ago. 2026.
- [15] AMAZON WEB SERVICES. *Timeouts, retries and backoff with jitter — Builders' Library*. Disponível em: <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>. Acesso em: 20 ago. 2026.
- [16] OLLAMA. *ollama/ollama*. Disponível em: <https://github.com/ollama/ollama>. Acesso em: 20 ago. 2026.
- [17] GGML-ORG. *llama.cpp*. Disponível em: <https://github.com/ggml-org/llama.cpp>. Acesso em: 20 ago. 2026.
- [18] VLLM PROJECT. *CLI Reference — serve*. Disponível em: <https://docs.vllm.ai/en/stable/cli/serve/>. Acesso em: 20 ago. 2026.
- [19] NOUSRESEARCH. *hermes-agent*. Disponível em: <https://github.com/NousResearch/hermes-agent>. Acesso em: 20 ago. 2026.
- [20] NOUSRESEARCH. *CLI Interface — Hermes Agent*. Disponível em: <https://hermes-agent.nousresearch.com/docs/user-guide/cli>. Acesso em: 20 ago. 2026.
- [21] NOUSRESEARCH. *Configuration — Hermes Agent*. Disponível em: <https://hermes-agent.nousresearch.com/docs/user-guide/configuration>. Acesso em: 20 ago. 2026.
- [22] CHEZMOI. *Install e Daily operations*. Disponível em: <https://www.chezmoi.io/install/>. Acesso em: 20 ago. 2026.
- [23] 1PASSWORD. *op signin — CLI command reference*. Disponível em: <https://developer.1password.com/docs/cli/reference/commands/signin>. Acesso em: 20 ago. 2026.

[24] ANTHROPIC. *Hooks reference*. Disponível em: <https://code.claude.com/docs/en/hooks>. Acesso em: 20 ago. 2026.

Capítulo 2: Prompt Caching de Verdade: Como o Provedor Cobra Menos por Repetição

1. Introdução

No Capítulo 1, você aprendeu o método de perícia que vai carregar pelo resto deste livro: nunca aceitar um comando só porque o nome da ferramenta é familiar, sempre exigir a hierarquia de fontes A/B/C e rodar `--help` ou abrir a documentação oficial antes de confiar em qualquer sintaxe. Você também viu o gatilho psicológico que o manual explora — o nome real destrava a confiança, e é exatamente aí que a sintaxe fabricada se esconde. Este capítulo aplica esse método ao primeiro item da mesa de perícia: o LAB 1 do manual, que promete cortar seu gasto com tokens “em até 90%” ativando um cache de prompt através de um arquivo de configuração — o percentual tem lastro na documentação oficial do próprio fornecedor [1], mas o arquivo de configuração citado, como você vai ver, não.

Como Perito de Configuração Agêntica, seu trabalho agora é separar o que é mecanismo real de cobrança — porque o desconto de cache existe e é documentado pelo próprio fornecedor — do que é caminho de arquivo forjado. Ao dominar isso, você deixa de copiar blocos de configuração de manuais de terceiros e passa a exigir o documento com fonte primária antes de tocar em qualquer `settings.json`, `.yaml` ou `.toml` do seu ambiente de trabalho.

2. Explica

O cache de prompt da Anthropic é real, documentado e funciona por um princípio simples: se o início de uma chamada de API for **byte-a-byte idêntico** a uma chamada anterior, o provedor reaproveita o processamento já feito daquele trecho em vez de refazê-lo do zero [1]. Esse mecanismo entra no payload da chamada — não em um arquivo de configuração de IDE — através do campo `cache_control` com o valor `{"type": "ephemeral"}`, que pode ser colocado no bloco `system`, em mensagens específicas, ou no último item de uma lista de `tools` [1].

Existem duas operações distintas, e confundi-las é o primeiro erro que a perícia precisa evitar. Uma chamada que cria um trecho de cache pela primeira vez é uma operação de **escrita** (write); uma chamada posterior que encontra o mesmo prefixo já cacheado é uma operação de **leitura** (read) [1]. A cobrança dessas duas operações é oposta ao que a intuição sugere: escrever no cache custa **mais caro** que uma chamada normal (1,25 vezes o preço de input padrão para um cache de 5 minutos de validade, ou 2 vezes para um cache de 1 hora), enquanto ler de um cache já existente custa **90% mais barato** — apenas 0,1 vez o preço de input padrão [1]. Esse é o número real por trás do “desconto de até 90%” citado pelo manual: ele é verdadeiro, mas só se aplica à leitura, e a documentação oficial não pública um percentual fixo de ganho de latência equivalente — apenas menciona melhora de tempo até o primeiro token em documentos longos, sem uma cifra fechada [1]. Análises externas que tentam quantificar esse ganho de latência tratam o número como estimativa de mercado, não como métrica publicada pelo fornecedor [2].

Dois regras adicionais fecham o mecanismo. Primeiro, existe um piso: blocos com menos de 1024 tokens não são cacheados, mesmo que o campo `cache_control` esteja presente [1]. Segundo, o cache é frágil por natureza — qualquer alteração de um único caractere no prefixo (um timestamp dinâmico, uma ordem de bloco trocada) invalida o cache a partir daquele ponto, forçando uma nova escrita [1]. É esse comportamento que explica por que ferramentas como o Aider organizam deliberadamente o histórico de chat (prompt de sistema, arquivos somente leitura, mapa do repositório, arquivos editáveis) em uma ordem fixa: manter o prefixo estável é a única forma de manter o cache “quente” [3].

Vale entender também onde exatamente o marcador `cache_control` pode ser colocado dentro do payload, porque essa é outra camada que o manual auditado simplesmente ignora. Existem duas formas de posicionamento documentadas pela própria Anthropic: (a) no nível raiz do request, caso em que o provedor aplica cache automático ao bloco elegível mais recente sem exigir marcação explícita em cada trecho; e (b) em blocos de conteúdo individuais — o array `system`, mensagens específicas dentro de `messages`, ou o último item de uma lista de `tools` — como um breakpoint explícito que você escolhe [1]. Um agente que injeta o mapa do repositório e o histórico de arquivos editáveis como blocos separados de `messages`, por exemplo, pode marcar cada um com seu próprio breakpoint, em vez de depender de um único cache automático no topo do payload. Essa granularidade é o que permite a ferramentas como o Aider manter partes do prompt “frias” (o pedido do usuário, que muda a cada turno) e partes “quentes” (o mapa do repositório, que muda raramente) na mesma chamada, sem que a parte volátil invalide o cache da parte estável [1].

3. Ilustra

Pense no cache de prompt como um laudo pericial que precisa ser reautenticado toda vez que um único traço da assinatura muda. A primeira vez que um documento — no nosso caso, o

prefixo da sua conversa com o modelo — passa pela mesa do perito, ele exige análise completa: comparação de traços, verificação de tinta, autenticação formal. Esse trabalho completo é a operação de **escrita** no cache, e por isso custa mais caro (1,25x a 2x) — você está pagando pelo trabalho pericial extra de deixar aquele documento pronto para reconhecimento futuro [1].

Mas se o mesmo documento, byte-a-byte idêntico, voltar à mesa depois, o perito não repete a análise inteira: ele reconhece a assinatura já catalogada e emite o laudo em segundos. Essa é a operação de **leitura**, e por isso ela custa apenas 10% do preço normal — 90% de desconto [1]. Agora, se um único traço mudar — uma vírgula a mais, uma data diferente no cabeçalho —, o documento deixa de ser “o mesmo” para efeitos de cadeia de custódia, e o perito precisa recomençar a autenticação do zero. É exatamente assim que uma linha de timestamp dinâmico dentro do seu prompt de sistema destrói silenciosamente todo o benefício do cache [1].

A segunda analogia cobre o ponto mais contraintuitivo do mecanismo: por que a Anthropic cobra **mais** para criar o cache. Pense em uma perícia que precisa deixar uma cópia autenticada arquivada para consultas futuras — abrir uma pasta nova, catalogar, indexar. Esse trabalho extra de preparo é o que você paga no cache write; o retorno vem depois, em todas as consultas de leitura que reaproveitam aquela pasta já pronta [1]. Quem faz uma única chamada e nunca repete o prefixo paga o preço da abertura da pasta sem nunca colher o desconto — por isso o cache só compensa quando o mesmo prefixo é reaproveitado diversas vezes na mesma janela de validade.

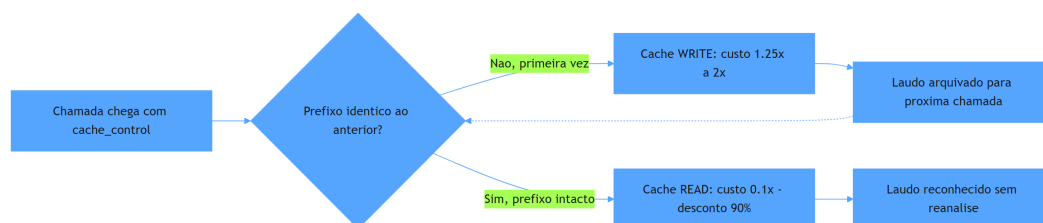


Figura 2: Fluxo de decisão do perito diante de uma chamada repetida a API com cache de prompt

Como veterano nessa mesa de perícia, você passa a olhar para qualquer alegação de “economia automática de tokens” com a mesma pergunta: o prefixo está realmente estável entre chamadas, ou algo está sutilmente mudando a assinatura do documento a cada requisição?

4. Técnica

O primeiro artefato de evidência é o próprio payload da Messages API. Ele mostra onde o campo `cache_control` realmente vive — dentro da chamada, não em um arquivo de configuração de IDE [1].

```
{
  "model": "claude-sonnet-4-5",
  "system": [
    {
      "type": "text",
      "text": "Voce e um assistente de codigo especializado em Python.
Regras do projeto: use type hints, docstrings no padrao Google, e nunca
escreva codigo sem tratamento de erro.",
      "cache_control": { "type": "ephemeral" }
    }
  ],
  "messages": [
    { "role": "user", "content": "Refatore a funcao calcular_total
abaixo." }
  ]
}
```

Nessa primeira chamada, o bloco `system` inteiro é marcado com `cache_control` — isso dispara uma operação de **escrita** (custo 1,25x, TTL padrão de 5 minutos) [1]. Se a próxima chamada, dentro da janela de 5 minutos, reenviar esse mesmo bloco `system` **byte-a-byte idêntico**, a Anthropic reconhece o prefixo e cobra apenas 0,1x por aquele trecho — a operação de **leitura** [1]. Qualquer edição no texto do `system` (mesmo um espaço a mais) recomeça o ciclo em uma nova escrita.

Onde a configuração de verdade mora, por ferramenta

Este é o ponto em que o manual auditado comete a fabricação mais perigosa do capítulo: ele descreve um arquivo `~/.config/claude-code/settings.json` com campos `systemPrompt` e `cacheControl` como se fosse o botão de ativação do cache no Claude Code. Esse caminho e esses campos **não existem** [4]. O arquivo real de configuração do Claude Code fica em `~/.claude/settings.json` (escopo de usuário) ou `.claude/settings.json` / `.claude/settings.local.json` (escopo de projeto), e os campos documentados são outros — `model`, `permissions`, `env`, `hooks`, `apiKeyHelper`, `cleanupPeriodDays`, entre dezenas de outros [4]. Não existe campo `cacheControl` nem `systemPrompt` exposto ao usuário: o cache de prompt do Claude Code é **automático**, gerenciado internamente pela ferramenta e pela API, sem toggle manual [4].

```
{
  "model": "claude-sonnet-4-5",
  "env": {
    "ANTHROPIC_MODEL": "claude-sonnet-4-5"
  },
  "permissions": {
```

```
"allow": ["Bash(git log:*)", "Read(**)"]
},
"cleanupPeriodDays": 30
}
```

Esse é o formato real de `~/.claude/settings.json` — repare que não há nenhum campo relacionado a cache. O que existe, e que o manual acerta, é o arquivo `CLAUDE.md` na raiz do projeto: ele é lido e incluído automaticamente no contexto de cada sessão, funcionando como o prefixo estável ideal para maximizar cache hit, desde que seu conteúdo não mude a cada execução [4].

O Aider segue outro caminho, também real: o arquivo `~/.aider.conf.yml` aceita o campo `cache-prompts`, que por padrão vem `false` e precisa ser ligado explicitamente [5].

```
# ~/.aider.conf.yml
cache-prompts: true
cache-keepalive-pings: 6
```

`cache-prompts: true` ativa o cache automático do provedor (a mesma mecânica de escrita/leitura da Anthropic descrita acima) para as chamadas feitas pelo Aider [5]. Já `cache-keepalive-pings` envia pings periódicos para manter o cache “quente” além da janela padrão de 5 minutos, evitando reescritas desnecessárias em sessões longas de edição [3].

O terceiro documento sob perícia é o Codex CLI, da OpenAI. O manual auditado erra de um jeito mais sutil aqui: acerta o **caminho** do arquivo (`~/.codex/config.toml`), mas erra a **estrutura interna**, colocando os campos de modelo dentro de uma seção `[codex]` que o repositório oficial do projeto não documenta em lugar nenhum [6].

```
# ~/.codex/config.toml
model = "gpt-5-codex"
model_provider = "openai"
model_reasoning_effort = "high"
wire_api = "responses"

[model_providers.meu_provedor_customizado]
name = "Meu Provedor"
base_url = "https://api.exemplo.com/v1"
```

Os campos reais (`model`, `model_provider`, `model_reasoning_effort`, `model_context_window`, `wire_api`) ficam no **nível raiz** do arquivo; provedores customizados vão em uma seção `[model_providers.<id>]`, não em `[codex]` [6]. A referência de configuração publicada pela própria OpenAI confirma esse formato plano e não documenta nenhum campo `cache_prompts` — quando aplicável, o cache da OpenAI também é automático [7].

Vale registrar, para efeito de cadeia de custódia, que mesmo dentro do próprio Claude Code existe outro documento fácil de confundir: hooks reais não vivem em um

arquivo separado `hooks.json` com campo `preProcess`, mas dentro da própria chave `hooks` de `~/.claude/settings.json`, disparados em eventos nomeados como `PreToolUse` / `PostToolUse` / `SessionStart` / `Stop` [8]. Guarde esse detalhe: ele volta a importar quando você for automatizar qualquer verificação em torno do cache no seu próprio fluxo de trabalho.

Verificando a estabilidade do prefixo com sha256sum

Se o cache depende de o prefixo permanecer byte-a-byte idêntico [1], então o passo de perícia mais barato antes de investigar qualquer queda de cache hit é confirmar que o seu próprio `CLAUDE.md` não está mudando de uma execução para outra sem você perceber — um espaço a mais inserido por um editor, uma linha de metadado gerada automaticamente, um timestamp de “última atualização” injetado por outra automação. Essa checagem não depende de nenhuma ferramenta fabricada: um hash SHA-256 do arquivo, comparado entre execuções, é o suficiente [4].

```
# Grava o hash atual do CLAUDE.md e compara com o hash da execucao
anterior.
# Se os hashes diferem, o prefixo mudou e o cache sera reescrito na proxima
chamada.
ARQUIVO="CLAUDE.md"
HASH_ANTERIOR_ARQ=".claude_md.sha256"

HASH_ATUAL=$(sha256sum "$ARQUIVO" | cut -d ' ' -f1)

if [ -f "$HASH_ANTERIOR_ARQ" ]; then
    HASH_ANTERIOR=$(cat "$HASH_ANTERIOR_ARQ")
    if [ "$HASH_ATUAL" = "$HASH_ANTERIOR" ]; then
        echo "CLAUDE.md estavel -- prefixo intacto, cache elegivel para READ."
    else
        echo "CLAUDE.md mudou desde a ultima execucao -- proxima chamada sera
WRITE."
    fi
else
    echo "Primeira execucao registrada -- sem baseline para comparar ainda."
fi

echo "$HASH_ATUAL" > "$HASH_ANTERIOR_ARQ"
```

Rodar esse script antes de cada sessão longa custa nada e responde, de forma determinística, a pergunta que normalmente vira suposição: “meu cache caiu porque o provedor mudou algo, ou porque eu mesmo alterei o prefixo sem perceber?” Na esmagadora maioria dos casos reais, a segunda hipótese é a correta — e o hash prova isso em uma linha, sem depender de nenhum campo de telemetria exposto pelo fornecedor.

As outras seis frentes: Hermes e o restante do catálogo

O manual não para em quatro ferramentas — ele estende a mesma tabela de cache para Hermes e mais seis IDEs agênticas. Aqui a perícia encontra o padrão mais traiçoeiro do capítulo: **o produto é genuíno em todos os sete casos**, mas o caminho de configuração citado raramente resiste à checagem contra fonte primária.

O Hermes Agent, da NousResearch, existe de fato como agente pessoal com CLI, memória persistente e delegação para subagentes — e o caminho `~/.hermes/config.yaml` citado pelo manual está correto [9]. O problema é o conteúdo: a documentação oficial lista campos reais como `model`, `memory`, `skills`, `prompt_caching`, `agent`, `terminal`, `runtime` e `worktree`, mas **não** existe `system_prompt` nem `cache_control: {type: ephemeral}` dentro desse arquivo — são nomes emprestados da API da Anthropic e colados em um documento que não os define [9]. O verbo real para gerenciar extensões do agente também diverge do manual: o guia oficial de CLI documenta `hermes skills install / browse / list`, sempre no plural [10]. O próprio repositório do projeto, aberto para inspeção pública, não registra nenhuma variante no singular equivalente à forma que o manual apresenta [11].

Das outras seis ferramentas citadas, quatro merecem registro individual antes da conclusão desta seção. MiMo Code, da Xiaomi, é um CLI real de codificação por agente, com repositório próprio aberto ao público [12]. Google Antigravity também é real: foi anunciada em preview público em novembro de 2025 como plataforma de desenvolvimento agent-first construída sobre um fork do VS Code [13]. Oh My Pi (o binário `omp`) é outro produto confirmado, um fork do framework Pi voltado a agentes de terminal [14]. Orca ADE, por fim, é um ambiente open-source real para rodar múltiplos agentes de codificação em worktrees paralelos [15]. Em todos os quatro casos, contudo, o caminho de arquivo de configuração citado pela tabela do manual não tem confirmação em documentação primária até o fechamento desta perícia. A única exceção verificável entre as sete é a Gemini CLI, do Google: o produto correto se chama Gemini CLI (não “Google CLI”, como o manual escreve), e sua configuração real fica documentada em `~/.gemini/settings.json` [16]. A própria documentação de referência do Google confirma esse caminho e a lista de campos aceitos [17]. Diante de um caminho “não confirmado”, o protocolo de perícia é simples: trate como evidência pendente, nunca como fato — não incorpore o caminho em automação alguma até checar a documentação oficial daquele produto específico no momento em que for usá-lo.

Medindo cache hit de verdade

Depois de configurar corretamente, o passo seguinte da perícia é medir se o cache está mesmo funcionando — e aqui o manual comete outra fabricação: ele instrui `pipx install ccusage` e o subcomando `ccusage log --date`. O `ccusage` é real e muito útil, mas é uma ferramenta **Node/npm**, não um pacote Python instalável via `pipx` [18]. Ele lê diretamente os arquivos JSONL que o Claude Code (e, em versões recentes, o Codex CLI) já gravam localmente, sem precisar de chamadas de API externas [18].

```
# instalacao/uso real: sem instalar nada, via npx
npx ccusage@latest session --json
```

A saída real desse comando traz os campos que você deve usar como evidência de cache hit — e não os nomes inventados pelo manual (`cache_hit_ratio` , `cache_hit`):

```
{
  "sessionId": "abc123",
  "inputTokens": 1520,
  "outputTokens": 340,
  "cacheCreationTokens": 4800,
  "cacheReadTokens": 38200,
  "cacheHitRate": 0.89,
  "costUSD": 0.412
}
```

O campo real é `cacheHitRate` — não `cache_hit_ratio` nem `cache_hit` — e ele vem acompanhado de `cacheCreationTokens` (tokens gastos em operações de escrita) e `cacheReadTokens` (tokens que se beneficiaram do desconto de leitura) [18]. Os subcomandos reais do `ccusage` são `daily` , `weekly` , `monthly` , `session` e `blocks` (janelas de 5 horas) — o comando `log --date` citado pelo manual não existe no repositório oficial da ferramenta [18]. Um guia independente de uso cotidiano da ferramenta confirma a mesma lista de subcomandos reais, sem qualquer menção a `log --date` [19].

```
# extraindo so a taxa de acerto de cache com jq
npx ccusage@latest session --json | jq '.[ ] | {sessionId, cacheHitRate}'
```

Uma nota final de cadeia de custódia: o manual também confunde dois produtos ao tentar verificar a instalação do `agenttrace` . `agenttrace` existe de verdade no PyPI, mantido pela Tensorstax, como biblioteca de observabilidade para agentes [20]. `agentlytics` também existe de verdade, mas é um projeto **diferente**, distribuído via npm, que lê o histórico local de várias IDEs agênticas para montar um dashboard de custo [21]. Tratar os dois como o mesmo produto — instalar um e verificar com o comando do outro — é o mesmo padrão de erro que você vai encontrar de novo neste livro: nome parecido, produtos distintos, sintaxe cruzada por engano.

5. Aplica

Imagine a seguinte cena. Você termina de ler o manual de terceiros na sexta à noite, animado com a promessa de cortar 90% do seu gasto em tokens — o número que a documentação oficial confirma para cache read [1]. Ainda no mesmo terminal, você cria a pasta `~/.config/claude-code/` , escreve um `settings.json` com `"systemPrompt"` e

`"cacheControl": {"type": "ephemeral"}`, salva o arquivo e abre o Claude Code na segunda-feira esperando ver a fatura despencar. Nada muda. Você olha o extrato de uso no fim da semana e o custo está idêntico ao de sempre — nenhum sinal de cache read em lugar nenhum.

O diagnóstico, à luz do que você acabou de examinar na seção Técnica, é direto: o Claude Code nunca leu aquele arquivo, porque ele não olha para `~/.config/claude-code/settings.json` — esse caminho simplesmente não existe no software real [4]. Você não cometeu um erro de sintaxe dentro do arquivo; você entregou o laudo ao endereço errado. A correção é mover a configuração para `~/.claude/settings.json` (ou `.claude/settings.json` na raiz do projeto) — o documento com fonte primária confirmada [4] — e, mais importante, entender que não existe campo `cacheControl` para ativar: o cache já está ligado, automaticamente, sempre que o prefixo da chamada permanecer estável byte-a-byte entre requisições, exatamente como a documentação oficial de cache descreve [1]. O ganho real não vem de um toggle: vem de manter seu `CLAUDE.md` e seu prompt de sistema estáveis, sem timestamps dinâmicos ou blocos que mudam de ordem a cada execução [1].

Esse é o tipo de armadilha que separa quem apenas copia configuração de quem audita antes de aplicar. Como síntese, guarde três sinais de alerta recorrentes: (1) caminho de arquivo com estrutura “bonita demais” (`~/.config/<nome-da-ferramenta>/settings.json` é um padrão genérico, não uma confirmação); (2) campo de configuração que promete controlar um mecanismo que a documentação descreve como automático; (3) métrica de sucesso com nome que “faz sentido” (`cache_hit_ratio`) mas que você nunca viu literalmente na saída real da ferramenta.

Em termos de escala, o cache de prompt da Anthropic compensa quando o mesmo prefixo é reaproveitado várias vezes dentro da janela de validade (5 minutos no padrão, 1 hora na variante premium) [1] — em um agente que processa uma tarefa isolada por chamada, sem reaproveitar contexto, o custo de escrita (1,25x a 2x) pode superar o benefício, e o cache deixa de compensar. Da mesma forma, blocos abaixo de 1024 tokens nunca são cacheados, então textos curtos de sistema não geram economia nenhuma, por mais estáveis que sejam [1]. Conhecer esse contorno evita a armadilha oposta: prometer para o seu time um ganho de cache que a arquitetura do seu agente não tem estrutura para capturar.

Erro comum vs. prática correta, aplicado à variante de 1 hora: o erro comum é ignorá-la por completo, assumindo que o TTL padrão de 5 minutos é a única opção e que qualquer sessão mais longa está condenada a pagar escrita repetida. Isso é especialmente caro em fluxos de revisão de código ou depuração, onde o operador lê um trecho de log, pensa por alguns minutos, faz uma pergunta de acompanhamento, e só então volta ao agente — um intervalo comum de 6 a 10 minutos que estoura a janela padrão e força uma nova escrita a cada rodada [1]. A prática correta é fazer a conta antes de decidir: se o mesmo prefixo — o `CLAUDE.md`, o mapa do repositório, as instruções de sistema — vai ser reaproveitado várias vezes ao longo de uma sessão que naturalmente ultrapassa 5 minutos entre chamadas, pagar o premium mais alto de escrita (2x em vez de 1,25x) para destravar 1 hora de validade tende a compensar, porque cada leitura subsequente dentro dessa janela mais longa continua custando os mesmos 0,1x do preço

padrão [1]. A régua não muda: o benefício só existe se o prefixo permanecer byte-a-byte estável durante toda a janela escolhida — os 5 minutos do padrão ou a 1 hora da variante premium.

Exercício

- ☐ Localize o arquivo real de configuração do Claude Code na sua máquina (`~/.claude/settings.json`) e liste os campos existentes — confirme que não há `cacheControl` nem `systemPrompt`
- ☐ Rode `npx ccusage@latest session --json` em um projeto com Claude Code já usado e identifique os valores de `cacheCreationTokens`, `cacheReadTokens` e `cacheHitRate`
- ☐ Se você usa Aider, adicione `cache-prompts: true` ao `~/.aider.conf.yml` e rode duas chamadas seguidas sobre o mesmo arquivo para observar a diferença de custo
- ☐ Escreva, para o seu próprio `CLAUDE.md`, uma lista dos elementos que poderiam variar de uma execução para outra (data, hora, IDs aleatórios) e remova qualquer um deles do topo do arquivo

6. Conclusão

Você fechou este capítulo com três evidências autenticadas: o mecanismo real de cache de prompt cobra 90% menos na leitura e um premium de 1,25x a 2x na escrita, sempre condicionado a um prefixo idêntico byte-a-byte [1]; a configuração de verdade mora em caminhos e campos específicos e documentados, nunca no arquivo fantasma `~/.config/claude-code/settings.json` com campos `systemPrompt` / `cacheControl` do manual auditado — Claude Code usa `~/.claude/settings.json` [4], Aider usa `cache-prompts: true` dentro de `~/.aider.conf.yml` [5], e Codex usa campos soltos no nível raiz de `~/.codex/config.toml` [6]; e a medição de cache hit de verdade passa pelo `ccusage` via `npx`, lendo os campos reais `cacheCreationTokens`, `cacheReadTokens` e `cacheHitRate` — nunca os nomes inventados pelo manual [18].

O padrão que você aprendeu a reconhecer aqui — ferramenta real, caminho de arquivo fabricado — vai se repetir, em outras variações, em praticamente todo capítulo restante deste livro. No Capítulo 3, você vai aplicar a mesma régua pericial a um problema ainda mais traiçoeiro: um catálogo inteiro de “modelos gratuitos” com nomes plausíveis, tabelas de especificação bem formatadas e nenhum deles existindo de verdade no gateway real.

7. Referências Bibliográficas

- [1] ANTHROPIC. *Prompt caching*. Disponível em: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>. Acesso em: 20 ago. 2026.
- [2] AGENTBRISK. *Prompt Caching Deep Dive: How to Cut Anthropic API Costs by 90%*. Disponível em: <https://agentbrisk.com/blog/prompt-caching-deep-dive-2026/>. Acesso em: 20 ago. 2026.
- [3] AIDER. *Prompt caching*. Disponível em: <https://aider.chat/docs/usage/caching.html>. Acesso em: 20 ago. 2026.
- [4] ANTHROPIC. *Claude Code settings*. Disponível em: <https://code.claude.com/docs/en/settings>. Acesso em: 20 ago. 2026.
- [5] AIDER. *YAML config file*. Disponível em: https://aider.chat/docs/config/aider_conf.html. Acesso em: 20 ago. 2026.
- [6] OPENAI. *codex/docs/config.md*. Disponível em: <https://github.com/openai/codex/blob/main/docs/config.md>. Acesso em: 20 ago. 2026.
- [7] OPENAI. *Configuration Reference — Codex CLI*. Disponível em: <https://developers.openai.com/codex/config-basic>. Acesso em: 20 ago. 2026.
- [8] ANTHROPIC. *Hooks reference*. Disponível em: <https://code.claude.com/docs/en/hooks>. Acesso em: 20 ago. 2026.
- [9] NOUSRESEARCH. *Configuration — Hermes Agent*. Disponível em: <https://hermes-agent.nousresearch.com/docs/user-guide/configuration>. Acesso em: 20 ago. 2026.
- [10] NOUSRESEARCH. *CLI Interface — Hermes Agent*. Disponível em: <https://hermes-agent.nousresearch.com/docs/user-guide/cli>. Acesso em: 20 ago. 2026.
- [11] NOUSRESEARCH. *hermes-agent*. Disponível em: <https://github.com/NousResearch/hermes-agent>. Acesso em: 20 ago. 2026.
- [12] XIAOMIMIMO. *MiMo-Code*. Disponível em: <https://github.com/XiaomiMiMo/MiMo-Code>. Acesso em: 20 ago. 2026.
- [13] GOOGLE. *Build with Google Antigravity — our new agentic development platform*. Disponível em: <https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/>. Acesso em: 20 ago. 2026.
- [14] OH MY PI. *omp.sh*. Disponível em: <https://omp.sh/>. Acesso em: 20 ago. 2026.
- [15] ORCA. *Orca ADE*. Disponível em: <https://www.onorca.dev/>. Acesso em: 20 ago. 2026.
- [16] GOOGLE-GEMINI. *gemini-cli*. Disponível em: <https://github.com/google-gemini/gemini-cli>. Acesso em: 20 ago. 2026.

[17] GOOGLE. *Gemini CLI — Configuration*. Disponível em: <https://geminicli.com/docs/reference/configuration/>. Acesso em: 20 ago. 2026.

[18] RYOPPIPIPI. *ccusage: A CLI tool for analyzing Claude Code/Codex CLI usage from local JSONL files*. Disponível em: <https://github.com/ryoppippi/ccusage>. Acesso em: 20 ago. 2026.

[19] CLAUDELOG. *What is ccusage tool for Claude Code*. Disponível em: <https://claudeblog.com/faqs/what-is-ccusage-tool/>. Acesso em: 20 ago. 2026.

[20] TENSORSTAX. *agenttrace: lightweight observability library to trace and evaluate agentic systems*. Disponível em: <https://github.com/tensorstax/agenttrace>. Acesso em: 20 ago. 2026.

[21] F. *agentlytics: Comprehensive analytics dashboard for AI coding agents*. Disponível em: <https://github.com/f/agentlytics>. Acesso em: 20 ago. 2026.

Capítulo 3: Modelos Gratuitos Sem Lenda: o Catálogo Real Por Trás do Gateway

1. Introdução

No Capítulo 2, você autenticou o caminho de configuração fantasma do cache de prompt: o manual auditado apontava para `~/.config/claude-code/settings.json` com um campo `cacheControl` que simplesmente não existe nesse arquivo. Você aprendeu que o real é `~/.claude/settings.json`, sem esse campo, e que o cache de leitura é automático — não um toggle de usuário. Neste capítulo, o mesmo instinto de perito vai examinar um documento mais traiçoeiro: uma lista de modelos gratuitos com nomes que soam profissionais demais para serem verdade.

O caso de hoje envolve o OpenCode Zen, um gateway real de modelos de IA. Isso não é uma fabricação total — é o padrão mais perigoso deste livro: produto real, sintaxe inventada. Como Perito de Configuração Agêntica, você vai aprender a diferença entre confiar no nome de uma empresa (a letra timbrada) e confiar em cada linha da lista de nomes que ela supostamente assina (a assinatura). Ao final, você terá um protocolo repetível para checar qualquer catálogo de modelo antes de colar o nome dele num script de automação.

2. Explica

OpenCode é uma IDE Agêntica real, com repositório e documentação públicos, e “Zen” é o nome verdadeiro do gateway de modelos que o próprio time do OpenCode mantém: uma camada curada, com opção gratuita sem exigir cartão de crédito [1]. Até aqui, o manual auditado não errou nada — a ferramenta existe, o nome do gateway existe, e a proposta (acesso gratuito a modelos de terceiros através de um único ponto de entrada) também é real.

O problema aparece na hora de nomear os modelos. O catálogo de modelos gratuitos de um gateway como esse é **rotativo por natureza**: provedores oferecem acesso promocional a modelos novos para ganhar adoção, e essa oferta muda de mês a mês. Levantamentos de ter-

ceiros feitos ao vivo confirmam isso na prática — comparações publicadas em datas diferentes de 2026 já mostram catálogos de modelos gratuitos distintos entre si [2], e outro levantamento independente reforça o mesmo padrão de rotatividade [3]. Isso significa uma coisa incômoda para quem escreve documentação estática: qualquer lista de nomes de modelo impressa num manual (ou num livro) tem prazo de validade.

Vale abrir o catálogo por completo para entender o tamanho real da fabricação, porque o gateway não é uma lista plana — ele mistura dois andares na mesma estante. O andar pago reúne modelos de ponta mantidos por grandes laboratórios, com nomes de família reconhecíveis: GPT-5.x, a linha Claude (Opus, Sonnet, Haiku), Gemini 3.x, Grok, Qwen3.x, DeepSeek-v4 e Kimi-k. O andar gratuito rotativo é outro andar: acesso promocional de prazo curto, oferecido pelo laboratório de origem para ganhar adoção antes de cobrar por uso pleno. É nesse segundo andar que moram os nomes reais que deveriam ter substituído a tabela fabricada do manual: “Big Pickle” — descrito por ambos os levantamentos independentes como um modelo stealth voltado a agentes de código —, “MiMo-V2 Flash Free” (modelo aberto publicado pela Xiaomi) e a dupla “Nemotron 3 Super Free”/“Nemotron 3 Ultra Free” (linhagem NVIDIA) [2][3]. Repare no detalhe que denuncia a fabricação por si só: nenhum desses nomes carrega um sufixo de porte (tiny/small/medium/large/xlarge). Cada um carrega a marca do laboratório que o publicou, porque é assim que o catálogo real se organiza — por origem e por janela promocional, nunca por uma escala de tamanho padronizada entre fornecedores diferentes.

É exatamente aqui que mora a diferença entre o real e o fabricado. A configuração de modelo do OpenCode não vive num campo `defaultModel` dentro de um arquivo `config.json` genérico — ela vive no arquivo `opencode.json` (ou `opencode.jsonc`), num campo chamado simplesmente `model`, no formato `provider/model-id` [4]. Não existe `cacheControl` nem `maxTokens` no nível raiz desse schema. Note como esse é o mesmo tipo de erro do Capítulo 2: um caminho de arquivo plausível, com campos que parecem razoáveis, mas que não correspondem a nada que o fornecedor realmente documenta.

O achado central deste capítulo é que a Anthropic documenta o mecanismo real de configuração de modelo do Claude Code de forma parecida — por meio de um arquivo de settings versionável, não de variáveis soltas [5] — e a documentação de cache de prompt da própria Anthropic reforça que a seleção de modelo e o comportamento de cache são coisas tratadas separadamente, cada uma com seu próprio mecanismo documentado [6]. Perceber esse paralelo é o que separa quem só decorou um comando de quem entende a lógica por trás dele.

3. Ilustra

Pense no OpenCode Zen como uma escrivania de cartório: o balcão existe, o carimbo é real, mas cada nome digitado na lista de “modelos reconhecidos” precisa ser conferido contra o livro de registro oficial antes de virar evidência aceita em qualquer automação. É essa cadeia

de custódia — da requisição do seu agente até o `provider/model-id` autenticado — que o diagrama abaixo representa.

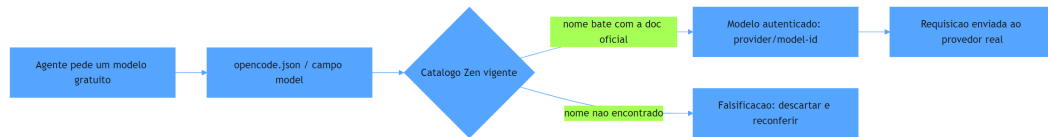


Figura 3: Cadeia de custódia de uma escolha de modelo, da requisição do agente até o `provider/model-id` autenticado no catálogo vigente do OpenCode Zen

Agora, a parte mais densa do caso: por que a tabela de 7 modelos do manual auditado (`opcode-zen-free-tiny/small/medium/large/xlarge/prompt/coder`) não pode ser simplesmente “corrigida” — ela precisa ser jogada fora inteira. Duas analogias ajudam a entender isso em camadas diferentes.

Primeiro, pense em uma carta com **letra timbrada de um cartório real, mas assinatura falsificada**. O timbre é autêntico — o papel, o cartório, o carimbo, tudo bate. Mas a assinatura do funcionário no rodapé foi desenhada por alguém que nunca trabalhou lá. Você não “conserta” essa assinatura comparando com outras cartas do mesmo cartório — você a rejeita inteira, porque não existe versão autêntica dela para recuperar. É o que acontece com `opcode-zen-free-medium`: o cartório (OpenCode Zen) existe, mas nenhum funcionário (modelo) assinou com esse nome.

Segundo, pense num **cardápio de restaurante real com um prato que nunca esteve no cardápio**. O restaurante existe, a mesa existe, o garçom é gentil e anota seu pedido — mas quando a cozinha procura a receita do “file Especial da Casa”, ela não existe em lugar nenhum, porque ninguém nunca a criou. Pedir de novo, com um nome ligeiramente diferente, não resolve: o prato inventado não tem receita porque nunca existiu, e a tabela de preço/calorias que alguém imprimiu ao lado dele também nunca podia ser real. É o mesmo raciocínio da tabela de benchmarks (latência, contexto, qualidade) que o manual auditado associou aos 7 nomes fabricados: não há como ajustar os números, porque não há objeto real por trás deles.

O padrão de nomenclatura fabricado (`tiny/small/medium/large/xlarge`) imita convenções reais de porte que você já viu em outros lugares — e é exatamente essa familiaridade que baixa sua guarda. Os nomes reais do catálogo Zen não seguem esse padrão limpo: um levantamento independente publicado em 2026 lista apelidos como “Big Pickle”, “Grok Code Fast 1” e “MiMo-V2 Flash Free” entre os modelos gratuitos vigentes [2], e outro guia de comunidade, feito na mesma época, confirma o mesmo catálogo instável incluindo variantes como “Nemotron 3 Free” [3]. Paradoxalmente, a falta de um padrão elegante é um sinal de autenticidade — catálogos reais raramente são tão arrumadinhos quanto uma fabricação.

4. Técnica

Como Perito de Configuração Agêntica, sua entrega técnica neste capítulo tem duas partes: a configuração correta do OpenCode apontando para um modelo real, e um protocolo reutilizável de verificação de catálogo — porque o problema que você acabou de investigar não é exclusivo do OpenCode Zen.

A configuração real: `opencode.json`

O arquivo `opencode.json` pode viver em `~/.config/opencode/opencode.json` (global) ou na raiz do projeto (`./opencode.json`) [4]. O schema documentado inclui campos como `$schema`, `model`, `agent`, `permission` e `mcp` — não existe `defaultModel`, `cacheControl` nem `maxTokens` no nível raiz [4].

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "opencode/big-pickle",
  "agent": {
    "build": {
      "model": "opencode/grok-code-fast-1"
    }
  }
}
```

Note o formato `provider/model-id`: o provedor vem antes da barra, o identificador do modelo vem depois. O nome `big-pickle` usado aqui reflete o catálogo gratuito descrito no levantamento independente já citado [2], e o segundo exemplo (`grok-code-fast-1`) aparece de forma equivalente no guia de comunidade que confirma a mesma safra de modelos rotativos [3] — e é exatamente por isso que o próximo bloco desta seção existe: nomes de modelo gratuito mudam, então essa configuração precisa ser reconferida, não copiada e esquecida.

O erro que você não vai cometer: config fabricada vs. config real

```
# ERRADO – nome de modelo fabricado, imitando o padrao de "porte" do manual
# auditado.
# opencode.json nao tem campo defaultModel nem cacheControl no nivel raiz.
cat > opencode-errado.json <<'EOF'
{
  "defaultModel": "opencode-zen-free-medium",
  "cacheControl": { "type": "ephemeral" },
  "maxTokens": 4096
```

```

}
EOF

# CORRETO — campo real "model", formato provider/model-id confirmado na doc
oficial.
cat > opencode-correto.json <<'EOF'
{
  "$schema": "https://opencode.ai/config.json",
  "model": "opencode/big-pickle"
}
EOF

echo "Config fabricada gravada em opencode-errado.json (referencia de erro,
nao use)."
echo "Config real gravada em opencode-correto.json."

```

Rodar o arquivo errado contra o OpenCode real não trava seu terminal com uma mensagem clara e didática — ele simplesmente falha ao resolver um modelo que não existe no catálogo, ou é ignorado silenciosamente pelo parser de schema. É esse silêncio que torna a fabricação perigosa: nada grita “isso está errado” até você depender daquele modelo em produção.

O protocolo de verificação de catálogo (aplicável a qualquer CLI nova)

A causa-raiz do erro deste capítulo não é exclusiva do OpenCode: é nomear um modelo, comando ou campo de configuração **de memória**, em vez de copiar da documentação oficial vigente no momento em que você escreve o script. O checklist abaixo é o que você aplica antes de automatizar qualquer escolha de modelo:

1. Abra a documentação oficial vigente do catálogo — nunca confie em uma tabela impressa num manual antigo ou num post de blog sem data.
2. Copie o `model-id` exato dali, caractere por caractere — não “arredonde” o nome para algo que pareça mais limpo.
3. Registre a data da checagem junto da configuração (comentário no arquivo, ou commit com data no dotfile) — assim, na próxima auditoria, você sabe se aquele nome ainda é válido.
4. Trate qualquer tabela de benchmark “bonita demais” sem link rastreável para a medição original como não verificada até prova em contrário.

Esse protocolo generaliza para qualquer ferramenta Agêntica nova que você adotar, porque cada uma documenta seu próprio mecanismo de configuração de modelo — nenhuma delas usa uma convenção universal:

Ferramenta	Onde checar o catálogo/config real	Fonte oficial
Aider	arquivo <code>~/.aider.conf.yml</code> , campo de modelo documentado na doc de config	YAML config file [7]
Codex CLI	<code>~/.codex/config.toml</code> , campos no nível raiz do documento	codex/docs/config.md [8]; Configuration Reference — Codex CLI [9]
ccusage	subcomandos reais de medição de uso, não flags inventadas	ccusage [10]
agenttrace	pacote real de observabilidade, API própria	agenttrace [11]
agentlytics	produto distinto de agenttrace, comandos próprios	agentlytics [12]
Ollama	biblioteca de modelos com tags versionadas	ollama/ollama [13]
llama.cpp	binário e flags do próprio repositório (ex.: <code>llama-cli</code>)	llama.cpp [14]
vLLM	subcomando <code>serve</code> documentado, flags reais	CLI Reference — serve [15]
Hermes Agent	comandos de skill, cron e memória documentados pelo próprio projeto	hermes-agent [16]; CLI Interface — Hermes Agent [17]; Configuration — Hermes Agent [18]
Gemini CLI	arquivo de configuração e schema documentados pela Google	gemini-cli [19]; Gemini CLI — Configuration [20]
Grok Build	repositório e anúncio oficial do produto	grok-build [21]; Introducing Grok Build [22]
chezmoi	subcomandos reais de instalação e operação diária	Install e Daily operations [23]
Orca ADE	documentação própria do produto	Orca ADE [24]

Cada linha dessa tabela representa uma ferramenta que este livro trata em outros capítulos — e em todas elas, o manual auditado errou pelo mesmo motivo: documentou de memória em vez de conferir na fonte no momento da escrita.

Detectando quando o catálogo mudou

A mesma técnica de hash usada no capítulo anterior para detectar mudança no `CLAUDE.md` serve para sinalizar quando uma página de catálogo mudou e precisa ser reconferida:

```
#!/usr/bin/env bash
# verificar-catalogo.sh – sinaliza quando a pagina de doc do catalogo mudou
# desde a ultima checagem, para voce reconferir o nome do modelo antes de
# usar.
set -euo pipefail

URL="https://opencode.ai/docs/zen/"
CACHE_DIR="$HOME/.cache/pericia-catalogo"
HASH_FILE="$CACHE_DIR/opencode-zen.sha256"

mkdir -p "$CACHE_DIR"

hash_atual=$(curl -fsSL "$URL" | sha256sum | awk '{print $1}')

if [[ -f "$HASH_FILE" ]]; then
    hash_anterior=$(cat "$HASH_FILE")
    if [[ "$hash_atual" != "$hash_anterior" ]]; then
        echo "ALERTA: o catalogo do OpenCode Zen mudou desde a ultima
checagem."
        echo "Reconfira os model-id antes de publicar ou automatizar."
    else
        echo "Catalogo sem mudanca detectada desde a ultima checagem."
    fi
else
    echo "Primeira checagem registrada."
fi

echo "$hash_atual" > "$HASH_FILE"
```

Esse script não substitui a leitura humana da documentação — ele só evita que você confie numa cópia mental desatualizada. O sinal de “mudou” é o gatilho para voltar ao passo 1 do checklist, nunca para presumir que o nome antigo ainda funciona.

Bônus: o script de benchmark que já estava certo

Nem tudo no laudo do manual auditado é fabricação. A tabela de latência/contexto/qualidade por modelo é irrecuperável — já vimos por quê —, mas a técnica de cronometragem usada para medir essa latência é engenharia genérica válida, e vale reaproveitá-la depois de trocar os nomes de modelo pelos reais. O princípio é simples: capturar um timestamp em nanossegundos antes e depois da chamada, e calcular a diferença em milissegundos.

```
#!/usr/bin/env bash
# benchmark-model.sh -- mede a latencia real de uma chamada a um model-id
do
# catalogo Zen. A tecnica de cronometragem (date +%s%N, nanossegundos) e
# generica e valida -- o erro do manual auditado nunca foi o cronometro,
# foi aplicar essa tecnica a nomes de modelo que nao existem [2][3].
set -euo pipefail

MODELO="${1:?informe um model-id conferido em opencode.ai/docs/zen/}" #
ex.: opencode/big-pickle [1]
PROMPT="${2:-Explique em uma frase o que e cache semantico.}"

chamada_ao_modelo() {
  # Substitua esta funcao pela chamada real ao SDK ou ao endpoint HTTP
  # autenticado do seu gateway -- este script nao inventa uma sintaxe de
  CLI
  # que o OpenCode nao documenta.
  echo "[integre aqui a chamada real ao modelo $MODELO]"
}

inicio_ns=$(date +%s%N)
resposta=$(chamada_ao_modelo)
fim_ns=$(date +%s%N)

latencia_ms=$(( (fim_ns - inicio_ns) / 1000000 ))
echo "Modelo testado: $MODELO"
echo "Latencia medida: ${latencia_ms}ms"
echo "Resposta: $resposta"
```

A lição para o seu protocolo de perícia é dupla: descartar a tabela fabricada não significa descartar a metodologia de medição por trás dela. Uma tabela de benchmark só vira evidência aceita quando (a) o modelo citado existe no catálogo vigente, checado pelo protocolo acima, e (b) o número foi de fato produzido por um script como este, rodado contra a chamada real — nunca estimado de memória e formatado para parecer uma medição.

5. Aplica

Imagine a cena: é sexta-feira à tarde, você está automatizando um script de fallback que troca o modelo do seu agente quando o principal está sobrecarregado, e lembra de ter visto, num manual salvo há meses, uma lista de nomes de modelo gratuitos “tiny/small/medium/large”. Você cola `opencode-zen-free-medium` direto no `opencode.json`, sem abrir a documentação, porque o nome parece exatamente com o tipo de convenção que outras ferramentas usam.

O erro acontece assim: o script roda, o parser de config não reconhece nenhum modelo com esse identificador, e a chamada cai num comportamento indefinido — silêncio, erro genérico de provedor, ou pior, uma resposta de um modelo default que você nunca escolheu conscientemente. Você perde tempo depurando um “bug de rede” que na verdade é um nome de modelo que nunca existiu.

O diagnóstico, à luz da seção Explica: você tratou um nome fabricado como se fosse autenticado só porque o produto por trás dele (OpenCode Zen) é real. É exatamente o padrão “letra timbrada real, assinatura falsificada” da seção Ilustra — o erro mais perigoso, porque o nome do gateway te deu confiança emprestada.

A correção é o protocolo do checklist: você abre `opencode.ai/docs/zen/ [1]`, confirma o `model-id` vigente naquele momento, atualiza o campo `model` do seu `opencode.json` com o valor exato copiado dali, e registra a data da checagem num comentário do commit. Da próxima vez que o script de fallback disparar, ele aponta para um modelo que de fato existe.

Armadilhas comuns que reforçam essa cena, para revisar rapidamente:

- Confiar em tabelas de benchmark encontradas em posts sem link para a medição original — trate como não verificado até achar a fonte primária.
- Presumir que uma convenção de nome “bonita” (`tiny/small/medium`) é sinal de autenticidade — muitas vezes é o oposto.
- Automatizar a escolha de modelo sem registrar a data da última checagem de catálogo — sem isso, você não sabe quando reconferir.

Resumindo a cena inteira em par de erro e correção, para consulta rápida na próxima vez que você configurar um fallback:

Situação	Prática errada (o que causou o incidente)	Prática correta (o protocolo deste capítulo)
Nome de modelo no <code>opencode.json</code>	Colar <code>opencode-zen-free-medium</code> de memória, confiando no nome do gateway	Copiar o <code>model-id</code> exato de <code>opencode.ai/docs/zen/ [1]</code> no momento do uso
Tabela de benchmark associada	Aceitar latência/contexto/qualidade “bonitos demais” sem link para a medição	Rodar o script de cronometragem da seção Técnica contra o modelo real
Validade da configuração	Tratar a lista de nomes como permanente, sem data de checagem	Registrar a data da checagem junto do commit e reconferir a cada rotação

Como Perito de Configuração Agêntica, seu trabalho não termina em identificar a fabricação uma vez — é manter o hábito de reconferência, porque o catálogo de amanhã não é o catálogo de hoje.

6. Conclusão

Você fechou a perícia do Capítulo 3 com três achados: o OpenCode Zen é um gateway real e gratuito de modelos, mas os 7 nomes `opencode-zen-free-*` do manual auditado são fabricados e sua tabela de specs é irrecuperável — não há como corrigi-la, só descartá-la e reconferir no catálogo vigente; e o protocolo de verificação que você construiu (checar a fonte oficial, copiar o `model-id` exato, registrar a data, desconfiar de benchmark sem link rastreável) se aplica a qualquer CLI Agêntica nova que cruzar seu caminho, não só ao OpenCode.

Como desafio, escolha uma ferramenta Agêntica que você usa hoje e rode o checklist completo nela: abra a doc oficial, confirme o nome exato do modelo ou comando que você usa de memória, e anote a data da checagem. No Capítulo 4, você vai aplicar essa mesma lupa a bibliotecas de compressão e cache semântico — onde o erro muda de forma: não é mais nome de modelo fabricado, é comando de terminal inventado para uma biblioteca que nunca teve interface de linha de comando.

7. Referências Bibliográficas

[1] OPENCODE. *Zen*. Disponível em: <https://opencode.ai/docs/zen/>. Acesso em: 20 ago. 2026.

[2] MAXIMAL STUDIO. *OpenCode Zen Free Models 2026: Every Free Provider and How to Use Them*. Disponível em: <https://www.maximalstudio.in/blog/opencode-zen-free-models>. Acesso em: 20 ago. 2026.

[3] BSWEN. *What Free AI Models Are Available in OpenCode and Which One Should You Use*. Disponível em: <https://docs.bswen.com/blog/2026-04-21-free-models-opencode/>. Acesso em: 20 ago. 2026.

[4] OPENCODE. *Config*. Disponível em: <https://opencode.ai/v2/docs/config>. Acesso em: 20 ago. 2026.

[5] ANTHROPIC. *Claude Code settings*. Disponível em: <https://code.claude.com/docs/en/settings>. Acesso em: 20 ago. 2026.

[6] ANTHROPIC. *Prompt caching*. Disponível em: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>. Acesso em: 20 ago. 2026.

- [7] AIDER. *YAML config file*. Disponível em: https://aider.chat/docs/config/aider_conf.html. Acesso em: 20 ago. 2026.
- [8] OPENAI. *codex/docs/config.md*. Disponível em: <https://github.com/openai/codex/blob/main/docs/config.md>. Acesso em: 20 ago. 2026.
- [9] OPENAI. *Configuration Reference — Codex CLI*. Disponível em: <https://developers.openai.com/codex/config-basic>. Acesso em: 20 ago. 2026.
- [10] RYOPPIPIPI. *ccusage: A CLI tool for analyzing Claude Code/Codex CLI usage from local JSONL files*. Disponível em: <https://github.com/ryoppippi/ccusage>. Acesso em: 20 ago. 2026.
- [11] TENSORSTAX. *agenttrace: lightweight observability library to trace and evaluate agentic systems*. Disponível em: <https://github.com/tensorstax/agenttrace>. Acesso em: 20 ago. 2026.
- [12] F. *agentlytics: Comprehensive analytics dashboard for AI coding agents*. Disponível em: <https://github.com/f/agentlytics>. Acesso em: 20 ago. 2026.
- [13] OLLAMA. *ollama/ollama*. Disponível em: <https://github.com/ollama/ollama>. Acesso em: 20 ago. 2026.
- [14] GGML-ORG. *llama.cpp*. Disponível em: <https://github.com/ggml-org/llama.cpp>. Acesso em: 20 ago. 2026.
- [15] VLLM PROJECT. *CLI Reference — serve*. Disponível em: <https://docs.vllm.ai/en/stable/cli/serve/>. Acesso em: 20 ago. 2026.
- [16] NOUSRESEARCH. *hermes-agent*. Disponível em: <https://github.com/NousResearch/hermes-agent>. Acesso em: 20 ago. 2026.
- [17] NOUSRESEARCH. *CLI Interface — Hermes Agent*. Disponível em: <https://hermes-agent.nousresearch.com/docs/user-guide/cli>. Acesso em: 20 ago. 2026.
- [18] NOUSRESEARCH. *Configuration — Hermes Agent*. Disponível em: <https://hermes-agent.nousresearch.com/docs/user-guide/configuration>. Acesso em: 20 ago. 2026.
- [19] GOOGLE-GEMINI. *gemini-cli*. Disponível em: <https://github.com/google-gemini/gemini-cli>. Acesso em: 20 ago. 2026.
- [20] GOOGLE. *Gemini CLI — Configuration*. Disponível em: <https://geminicli.com/docs/reference/configuration/>. Acesso em: 20 ago. 2026.
- [21] XAI-ORG. *grok-build*. Disponível em: <https://github.com/xai-org/grok-build>. Acesso em: 20 ago. 2026.
- [22] XAI. *Introducing Grok Build*. Disponível em: <https://x.ai/news/grok-build-cli>. Acesso em: 20 ago. 2026.
- [23] CHEZMOI. *Install e Daily operations*. Disponível em: <https://www.chezmoi.io/install/>. Acesso em: 20 ago. 2026.

[24] ORCA. *Orca ADE*. Disponível em: <https://www.onorca.dev/>. Acesso em: 20 ago. 2026.

Capítulo 4: Compressão e Cache Semântico Sob a Lupa: Bibliotecas, Não Comandos

1. Introdução

No Capítulo 3, você já aplicou o protocolo de verificação de catálogo para desmontar a tabela de sete modelos gratuitos que o manual atribuía ao gateway OpenCode Zen — nomes plausíveis, especificações bem formatadas, e nenhuma correspondência real no catálogo vigente [1]. A composição desse catálogo muda de nome e de oferta com frequência, o que a cobertura especializada do próprio ecossistema já registrou em mais de uma ocasião [2][3]. Você aprendeu, ali, que uma tabela pode ter toda a aparência de um laudo técnico e ainda assim ser ficção com formatação de fato: a autópsia da tabela fabricada mostrou que basta um nome convincente para destravar a confiança de quem só olha por cima.

Neste capítulo você aponta a mesma lente forense para um alvo mais traiçoeiro. LLM-Lingua, da Microsoft Research, e GPTCache, da Zilliztech, são duas bibliotecas Python que **existem de verdade** — mas a forma como o manual descreve seu uso tem letra timbrada real (o nome do pacote é genuíno) e assinatura falsificada (a API e a CLI descritas nunca existiram). Como Perito de Configuração Agêntica, você vai confirmar a classe certa, o parâmetro certo, e sair deste capítulo com dois artefatos próprios — um wrapper de compressão de prompt e um hook de cache semântico — inteiramente construídos sobre a API real, testável linha a linha.

2. Explica

Compressão de prompt ataca o problema da economia de tokens por outro ângulo do que o cache de prompt que você já periciou no Capítulo 2. Lembre que ali você confirmou que o cache de leitura da Anthropic só se aplica a partir de 1024 tokens de prefixo idêntico [4] — compressão de prompt não depende de repetição alguma: ela reduz o próprio texto enviado ao modelo, preservando o essencial do significado antes mesmo de a chamada sair da sua máquina. O projeto oficial LLM-Lingua, mantido pela Microsoft Research e publicado em conferências

revisadas por pares (EMNLP 2023 e ACL 2024), documenta compressões de **até 20 vezes** sobre o prompt original, com perda mínima de desempenho em tarefas de raciocínio [5][6]. É esse teto oficial — não os percentuais soltos por categoria de conteúdo que o manual tabula sem nenhuma fonte por trás — que este capítulo usa como métrica de referência.

Cache semântico ataca um terceiro problema, diferente dos dois anteriores. Perguntas diferentes na forma, mas equivalentes no significado, hoje disparam duas chamadas completas e pagas ao modelo. O GPTCache resolve isso guardando pares de pergunta-resposta como vetores de embedding e comparando a distância semântica da nova pergunta contra o que já foi respondido antes: se a distância ficar abaixo de um limiar configurado, a resposta arquivada é devolvida sem gerar uma nova chamada [7]. Na prática, é memória associativa — não é preciso perguntar palavra por palavra igual para reaproveitar uma resposta já dada.

Vale fixar o padrão antes de seguir: cada ferramenta que você já periciou neste livro tem a sua própria superfície real de configuração, e nenhuma delas se parece com a das outras. O Aider expõe cache real através da flag `--cache-prompts` no seu `~/.aider.conf.yml` [8], documentada ao lado de pings periódicos para manter o cache “quente” [9]. O Codex CLI grava seus campos reais de modelo no nível raiz do `~/.codex/config.toml`, não numa seção `[codex]` inventada [10][11]. O OpenCode versiona um catálogo de modelos gratuitos que muda de composição com o tempo, sob o arquivo real `opencode.json` [12][1]. Nenhuma dessas três superfícies parece com a de LLMLingua ou GPTCache — porque nenhuma das duas é uma ferramenta de configuração: são bibliotecas.

A causa raiz do erro do manual, nos dois casos, é a mesma, e é o núcleo deste capítulo: **um pacote instalável não é sinônimo de um comando de terminal**. `pip install llmlingua` e `pip install gptcache` instalam bibliotecas Python — código para você **importar** dentro de um script próprio — e não ferramentas com interface de linha de comando completa. Isso não é uma falha de design das duas bibliotecas: é uma escolha legítima e comum em boa parte do ecossistema Python de machine learning, o mesmo padrão, aliás, que separa o `pip install llmlingua` de um utilitário como o `ccusage`, que de fato expõe subcomandos de terminal reais porque foi construído para isso desde o início [13]. O manual erra ao tratar LLMLingua e GPTCache como utilitários de terminal completos, com subcomandos, flags e arquivos de configuração externos — nenhum dos quais existe [5][7].

3. Ilustra

Pense num laudo pericial que precisa caber numa única lauda: você não reescreve o depoimento inteiro, extrai só o que sustenta a conclusão e descarta o floreio — isso é compressão de prompt. E pense em duas testemunhas que contam praticamente a mesma história com palavras diferentes: um perito experiente reconhece que é o mesmo fato e não abre um segundo processo

do zero — isso é cache semântico. As duas técnicas fazem o trabalho pericial de menos texto processado gerar a mesma conclusão.

O ponto mais traiçoeiro deste capítulo, porém, exige uma segunda analogia. Rodar `pipx install llmlingua` é como receber uma caixa lacrada, com etiqueta oficial e nota fiscal legítima, mas que guarda peças soltas de reposição — não uma ferramenta pronta para pegar da prateleira e usar. O `pipx` monta uma prateleira isolada esperando encontrar, dentro da caixa, um executável já pronto (o que o ecossistema Python chama de *entry point* de console). Quando a caixa só contém peças importáveis — como é o caso de LLMLingua e GPtCache — a prateleira fica perfeitamente instalada e completamente vazia de qualquer coisa que você possa digitar no terminal. Nada quebrou; simplesmente não havia ferramenta de terminal ali para começo de conversa.

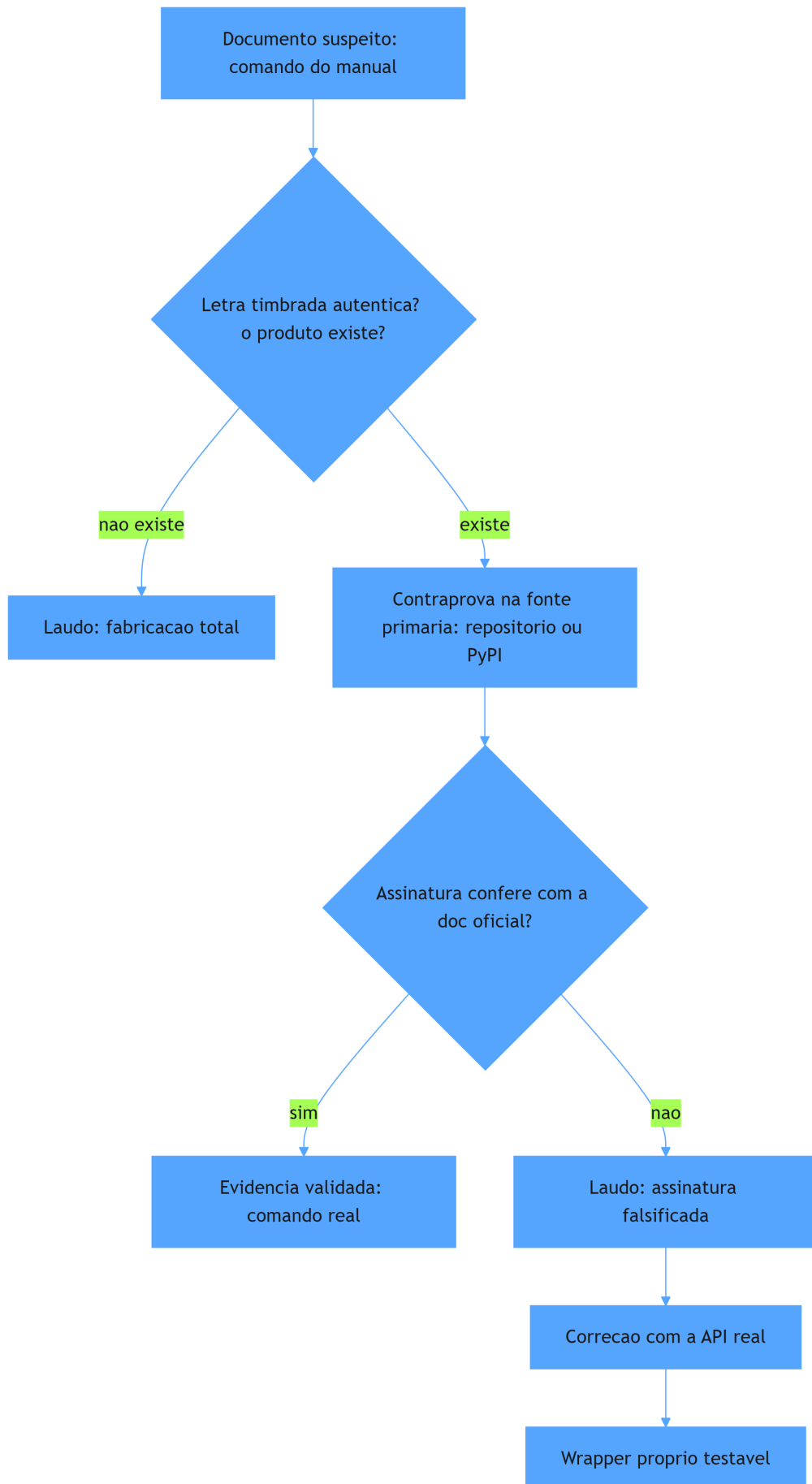


Figura 4: Cadeia de custódia pericial aplicada aos comandos de LLMLingua e GPTCache citados no manual auditado

4. Técnica

A Classe Certa: PromptCompressor em Vez de Llmlingua

O `manual` descreve

```
from llmlingua import Llmlingua; Llmlingua().compress_prompt(prompt, . Rode isso
preservation_rate=0.5)
```

e você recebe um `ImportError` na primeira linha: a classe exportada pelo pacote real chama-se `PromptCompressor`, e o parâmetro do método `compress_prompt` é `rate` — não `preservation_rate` [5][6]. “LongLLMLingua”, citado pelo manual como um pacote separado (`pipx install longllmlingua`), também não existe como distribuição própria: é um modo de uso da mesma classe, ativado pelo parâmetro `rank_method="longllmlingua"`, pensado para reordenar contexto longo e mitigar o efeito de “perder informação no meio do prompt” [5].

```
#!/usr/bin/env python3
"""compress_prompt.py -- wrapper real de compressao de prompt via
LLMLingua.

O manual descreve `Llmlingua().compress_prompt(prompt,
preservation_rate=0.5)`
e uma CLI `python -m llmlingua --preserve`. Nenhuma das duas existe: a
classe
real e o parametro real sao outros -- ver [5][6].

Uso:
    python compress_prompt.py --rate 0.5 < prompt_bruto.txt >
prompt_comprimido.txt
Requisito:
    pip install llmlingua
"""
import argparse
import sys

from llmlingua import PromptCompressor # classe real; nao "Llmlingua"

def comprimir(texto: str, rate: float, contexto_longo: bool = False) ->
dict:
    """Comprime um prompt usando a API real do LLMLingua."""
    compressor = PromptCompressor()
    kwargs = {"rate": rate}
    if contexto_longo:
        # LongLLMLingua nao e pacote separado: e este parametro do metodo
```

```

real [5]
    kwargs["rank_method"] = "longllmlingua"
    return compressor.compress_prompt(texto, **kwargs)

def main() -> int:
    parser = argparse.ArgumentParser(
        description="Comprime um prompt via LLMLingua (API real, sem CLI
oficial).")
    )
    parser.add_argument("--rate", type=float, default=0.5,
                        help="fracao de tokens a preservar, 0 a 1")
    parser.add_argument("--contexto-longo", action="store_true",
                        help="ativa o modo longllmlingua para prompts
extensos")
    args = parser.parse_args()

    bruto = sys.stdin.read()
    if not bruto.strip():
        print("erro: nenhum prompt recebido via stdin", file=sys.stderr)
        return 1

    resultado = comprimir(bruto, args.rate, args.contexto_longo)
    print(resultado["compressed_prompt"])

    origem = resultado["origin_tokens"]
    final = resultado["compressed_tokens"]
    economia = 1 - (final / origem) if origem else 0
    print(f"# economia: {economia:.1%} ({origem} -> {final} tokens)",
file=sys.stderr)
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Por Que pipx Não Cria um Comando Que Não Existe

Antes de escrever o hook de cache, vale fechar o diagnóstico do Pilar 2. `pipx` foi desenhado para instalar **CLIs isoladas** — pacotes que declaram um *entry point* de console no seu empacotamento. Quando você roda `pipx install llmlingua`, a instalação termina sem erro (o pacote é real e válido), mas nenhum comando novo aparece no seu `PATH`, porque o próprio pacote nunca declarou um. É por isso que `python -m llmlingua --help` também falha: não há um `__main__.py` documentado que responda por essa chamada [5][6]. Compare com `ccusage`, que expõe de fato subcomandos de terminal (`daily`, `weekly`, `session`) porque foi construído

desde o início como ferramenta de linha de comando, distribuída via npm — não Python [13]. Esse mesmo comportamento de CLI real, com subcomandos documentados e reconhecíveis, é descrito de forma independente por quem usa a ferramenta no dia a dia [14]. A diferença não é sobre qual gerenciador de pacote você usa; é sobre o que o pacote se propôs a ser.

O Hook de Cache Semântico Completo (semantic_cache.py)

O mesmo padrão de erro se repete no GPTCache, em escala maior: o manual descreve subcomandos `gptcache init/list/add/query/stats/clear` e um arquivo `~/.gptcache/config.yaml`. Nenhum dos dois existe. A configuração real é inteiramente programática — você monta o cache dentro do seu próprio script Python, com `cache.init(...)` e um objeto `Config`, e o único processo de linha de comando real do projeto é o `gptcache_server`, que sobe um servidor HTTP com endpoints `/put` e `/get` — não uma CLI de gestão do cache [7].

```
"""semantic_cache.py -- hook real de cache semantico via GPTCache.

O manual descreve `gptcache init/list/add/query/stats/clear` como CLI e um
arquivo `~/.gptcache/config.yaml`. Nenhum dos dois existe: a configuracao
do
GPTCache e inteiramente programatica [7].
"""

from gptcache import cache
from gptcache.manager import get_data_manager, CacheBase, VectorBase
from gptcache.embedding import Onnx
from gptcache.similarity_evaluation.distance import
SearchDistanceEvaluation
from gptcache.adapter.api import get, put

_onnx = Onnx()
_data_manager = get_data_manager(
    CacheBase("sqlite"),
    VectorBase("faiss", dimension=_onnx.dimension),
)

def inicializar(limiar_similaridade: float = 0.8) -> None:
    """Substitui o YAML fabricado por chamadas reais de inicializacao
    [7]."""
    cache.init(
        embedding_func=_onnx.to_embeddings,
        data_manager=_data_manager,
        similarity_evaluation=SearchDistanceEvaluation(),
    )
    cache.config.similarity_threshold = limiar_similaridade
```

```
def consultar_ou gerar(pergunta: str, gerar_resposta) -> str:
    """Consulta o cache semantico; so chama o modelo se nao houver hit."""
    resposta_em_cache = get(pergunta)
    if resposta_em_cache is not None:
        return resposta_em_cache

    resposta = gerar_resposta(pergunta)
    put(pergunta, resposta)
    return resposta

if __name__ == "__main__":
    inicializar(limiar_similaridade=0.8)

    def chamada_simulada(pergunta: str) -> str:
        return f"resposta gerada para: {pergunta}"

    print(consultar_ou gerar("como configurar cache de prompt no claude
code", chamada_simulada))
```

A Segunda Forma Real de Configurar o Limiar: Objeto Config

O `semantic_cache.py` acima seta o limiar de similaridade depois do `init`, via `cache.config.similarity_threshold`. Essa não é a única forma documentada — existe uma segunda, igualmente real, que passa um objeto `Config` já na chamada de inicialização, útil quando você quer declarar todos os parâmetros num único lugar em vez de configurar atributo por atributo depois [7]:

```
from gptcache import cache
from gptcache.config import Config

cache.init(
    embedding_func=_onnx.to_embeddings,
    data_manager=_data_manager,
    similarity_evaluation=SearchDistanceEvaluation(),
    config=Config(similarity_threshold=0.8), # forma alternativa, mesmo
efeito
)
```

Note o que **não** existe em nenhuma das duas formas: um parâmetro `namespace` no construtor do cache, como o manual descreve em `gptcache.GPTCache(namespace=..., similarity_threshold=...)`. A classe exportada pelo pacote real chama-se `Cache`, não `GPTCache`, e ela não nasce pronta — é inicializada

através do método `.init(...)` mostrado acima [7]. Confundir a classe com o namespace do seu projeto é outro sintoma do mesmo padrão de erro: nomear um parâmetro que soa razoável, sem checar a assinatura real do método.

O Modo Servidor: `gptcache_server`, Para Quando Você Não Quer Embutir o Cache no Script

O `semantic_cache.py` roda o cache **dentro** do processo Python que faz as chamadas ao modelo — é o padrão certo quando um único script ou serviço concentra todo o tráfego. Mas o GPTCache também documenta um segundo modo de operação, para quando vários clientes (potencialmente em linguagens diferentes) precisam compartilhar o mesmo cache: o binário `gptcache_server`, que sobe um servidor HTTP com endpoints `/put` e `/get` [7].

```
# unico binario real de linha de comando do projeto -- nao existem os
# subcomandos init/list/add/query/stats/clear que o manual descreve [7].
gptcache_server -s 127.0.0.1 -p 8000
```

Antes de integrar esse modo servidor num pipeline de produção, confira o schema exato de payload dos endpoints `/put` e `/get` na documentação oficial do projeto [7] — este capítulo confirma que o binário e as duas rotas existem, mas não fixa aqui o corpo exato da requisição, para não repetir o mesmo erro que está corrigindo: publicar uma sintaxe sem conferência direta na fonte.

Onde Isso Se Encaixa na Cadeia Real de Hooks

O manual também erra o mecanismo de disparo automático: ele descreve um arquivo separado `~/.config/claude-code/hooks.json` com um campo `preProcess`. Isso não existe. Hooks reais do Claude Code vivem dentro do mesmo `~/.claude/settings.json` que você já periciou no Capítulo 2, sob a chave `hooks`, associados a eventos documentados como `PreToolUse`, `PostToolUse`, `SessionStart` e `Stop` [15][16]. Para acionar o `semantic_cache.py` antes de uma ferramenta específica rodar, a configuração fica assim:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "*",
        "hooks": [
          { "type": "command", "command": "python3 /caminho/
semantic_cache.py" }
        ]
      }
    ]
  }
}
```

```
    ]
  }
}
```

Não existe evento `preProcess`, nem arquivo `hooks.json` isolado — a evidência aceita é o par chave/evento documentado oficialmente [15][16].

Blindando o Wrapper Contra Falha em Cascata

Um cache semântico que depende de gerar embeddings a cada consulta introduz um novo ponto de falha: se o provedor de embeddings ficar lento ou indisponível, cada chamada de `consultar_ou gerar` pode travar em série. O padrão de engenharia correto — que você vai formalizar por completo no Capítulo 5 — é envolver a chamada de geração num backoff exponencial com jitter, para não amplificar um rate limit já em curso [17], e, em produção, um circuit breaker que interrompe novas tentativas depois de falhas consecutivas em vez de insistir indefinidamente [18]. Se o seu wrapper precisar consultar várias perguntas ao mesmo tempo, o mecanismo real para paralelizar sem estourar limite de concorrência é `asyncio.gather` com um `asyncio.Semaphore` [19] — o mesmo princípio por trás de um `xargs -P/GNU parallel` na linha de comando [20], que você também vai auditar formalmente no próximo capítulo. Nenhum desses padrões depende de sintaxe fabricada — são práticas de resiliência amplamente documentadas, prontas para plugar em cima do `gerar_resposta` do wrapper acima.

Observando o Resultado em Produção

Depois de trocar a CLI fabricada pela API real, ainda falta medir se a compressão e o cache estão de fato economizando tokens. Para isso, instrumente o wrapper com uma lib de observabilidade real, como o `agenttrace` [21] — sem confundi-la com o `agentlytics`, que é outro produto, de outro ecossistema (npm, não PyPI), focado num painel de custo por IDE, não em tracing de chamadas [22]. São dois nomes parecidos e dois laudos diferentes — a mesma armadilha de confusão categorial que você já viu no Capítulo 3.

5. Aplica

Você decide seguir o manual à risca. Abre o terminal, roda `pipx install llmlingua`, e a instalação termina em segundos, sem nenhum erro visível. Animado, você digita `python -m llmlingua --help` esperando ver a lista de flags prometida — e recebe `No module named llmlingua.__main__`. Você tenta de novo com `llmlingua --help` puro:

`command not found`. A reação mais comum, nesse ponto, é concluir que a instalação falhou silenciosamente, ou que falta alguma variável de ambiente. Nenhuma das duas coisas é verdade.

O diagnóstico correto é o que você já viu na seção Técnica: `pipx` fez exatamente o que deveria fazer — instalou o pacote real, isolado, sem conflito com outras dependências. O que faltava nunca existiu: `LLMLingua` não declara um *entry point* de console, então não há comando de terminal para “aparecer”. A correção não é reinstalar nem depurar o `PATH` — é abandonar a expectativa de CLI e escrever o wrapper `compress_prompt.py` que você acabou de construir, importando `PromptCompressor` dentro do seu próprio script Python. O mesmo raciocínio vale, ponto a ponto, para `gptcache init` e para o restante da lista de subcomandos fabricados: a correção é sempre a mesma classe de ação — trocar a expectativa de CLI pela chamada de API real.

Essa técnica tem limites que você precisa declarar antes de colocá-la em produção. A compressão do `LLMLingua` preserva o significado geral do texto, não a sintaxe exata: em prompts que colam trechos de código-fonte, caminhos de arquivo ou identificadores exatos, uma taxa agressiva (`rate` abaixo de 0.3, por exemplo) pode cortar justamente o token que carregava a informação crítica. Para esse tipo de prompt, mantenha uma taxa mais conservadora (0.6 a 0.8) ou comprima apenas a prosa ao redor do bloco de código, preservando o bloco intacto. O cache semântico tem uma fronteira diferente: ele escala bem para bases de perguntas recorrentes — documentação interna, comandos repetidos, dúvidas de suporte — mas quebra quando cada pergunta feita ao sistema é genuinamente única. Nesse regime, o custo de gerar um embedding e consultar o índice FAISS a cada chamada passa a superar a economia de nunca ter um cache hit, porque não há repetição semântica nenhuma para explorar.

Isso também esclarece uma dúvida prática que fica em aberto depois de ler a seção Técnica: quando escolher `cache.init()` embutido em vez do `gptcache_server`? A resposta é sobre topologia, não sobre qual dos dois é “mais real” — os dois são [7]. Se um único processo Python concentra todas as chamadas ao modelo, embute o cache no próprio script, como em `semantic_cache.py`: menos uma peça de infraestrutura para operar. Se múltiplos serviços, possivelmente em linguagens diferentes, precisam consultar o mesmo cache compartilhado, o `gptcache_server` isola essa responsabilidade atrás de um endpoint HTTP comum — o preço é mais uma peça rodando e mais uma rede a monitorar.

Exercício

- ☐ Rode `pip show llmlingua` no seu ambiente e confirme que não aparece nenhum “Console Scripts” na saída
- ☐ Escreva e teste `compress_prompt.py` com um prompt real de pelo menos 500 tokens, comparando o texto antes e depois
- ☐ Inicialize `cache.init()` do `GPTCache` localmente e force um cache hit repetindo a mesma pergunta duas vezes seguidas

- ☐ Ajuste `similarity_threshold` para 0.9 e depois para 0.6 e registre a diferença de comportamento em duas perguntas parecidas, mas não idênticas
- ☐ Suba `gptcache_server -s 127.0.0.1 -p 8000` localmente e confirme, consultando a doc oficial [7], o schema de payload real dos endpoints `/put` e `/get` antes de integrá-lo a qualquer script

6. Conclusão

Você fechou este capítulo com três evidências assentadas: a classe real por trás de uma API inventada (`PromptCompressor`, não `Llmlingua`), o motivo técnico exato pelo qual instalar uma biblioteca com `pipx` não cria magicamente um comando de terminal, e dois artefatos próprios — `compress_prompt.py` e `semantic_cache.py` — construídos sobre a API real do `LLMLingua` e do `GPTCache`, prontos para rodar sem depender de nenhum subcomando fabricado. O padrão que você reconheceu aqui é o mesmo do Capítulo 3, aplicado a um nível mais fino: produto real, sintaxe inventada, e a única defesa é a contraprova direta na fonte primária.

No Capítulo 5, você leva a perícia para outro terreno: paralelismo e resiliência, onde o manual finalmente acerta quase tudo — e você vai entender por quê, formalizando o backoff exponencial e o circuit breaker que só foram mencionados de passagem aqui.

7. Referências Bibliográficas

[1] OPENCODE. *Zen*. Disponível em: <https://opencode.ai/docs/zen/>. Acesso em: 20 ago. 2026.

[2] BSWEN. *What Free AI Models Are Available in OpenCode and Which One Should You Use*. Disponível em: <https://docs.bswen.com/blog/2026-04-21-free-models-opencode/>. Acesso em: 20 ago. 2026.

[3] MAXIMAL STUDIO. *OpenCode Zen Free Models 2026: Every Free Provider and How to Use Them*. Disponível em: <https://www.maximalstudio.in/blog/opencode-zen-free-models>. Acesso em: 20 ago. 2026.

[4] ANTHROPIC. *Prompt caching*. Disponível em: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>. Acesso em: 20 ago. 2026.

[5] MICROSOFT. *LLMLingua* (repositório oficial). Disponível em: <https://github.com/microsoft/LLMLingua>. Acesso em: 20 ago. 2026.

[6] PYTHON PACKAGE INDEX. *llmlingua*. Disponível em: <https://pypi.org/project/llmlingua/>. Acesso em: 20 ago. 2026.

- [7] ZILLIZTECH. *GPTCache: A Library for Creating Semantic Cache for LLM Queries* (docs/usage.md). Disponível em: <https://github.com/zilliztech/GPTCache/blob/main/docs/usage.md>. Acesso em: 20 ago. 2026.
- [8] AIDER. *Prompt caching*. Disponível em: <https://aider.chat/docs/usage/caching.html>. Acesso em: 20 ago. 2026.
- [9] AIDER. *YAML config file*. Disponível em: https://aider.chat/docs/config/aider_conf.html. Acesso em: 20 ago. 2026.
- [10] OPENAI. *Configuration Reference — Codex CLI*. Disponível em: <https://developers.openai.com/codex/config-basic>. Acesso em: 20 ago. 2026.
- [11] OPENAI. *codex/docs/config.md*. Disponível em: <https://github.com/openai/codex/blob/main/docs/config.md>. Acesso em: 20 ago. 2026.
- [12] OPENCODE. *Config*. Disponível em: <https://opencode.ai/v2/docs/config>. Acesso em: 20 ago. 2026.
- [13] RYOPPIPI. *ccusage: A CLI tool for analyzing Claude Code/Codex CLI usage from local JSONL files*. Disponível em: <https://github.com/ryoppippi/ccusage>. Acesso em: 20 ago. 2026.
- [14] CLAUDELOG. *What is ccusage tool for Claude Code*. Disponível em: <https://claudelog.com/faqs/what-is-ccusage-tool/>. Acesso em: 20 ago. 2026.
- [15] ANTHROPIC. *Claude Code settings*. Disponível em: <https://code.claude.com/docs/en/settings>. Acesso em: 20 ago. 2026.
- [16] ANTHROPIC. *Hooks reference*. Disponível em: <https://code.claude.com/docs/en/hooks>. Acesso em: 20 ago. 2026.
- [17] AMAZON WEB SERVICES. *Timeouts, retries and backoff with jitter — Builders' Library*. Disponível em: <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>. Acesso em: 20 ago. 2026.
- [18] MICROSOFT. *Circuit Breaker pattern — Azure Architecture Center*. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker>. Acesso em: 20 ago. 2026.
- [19] PYTHON SOFTWARE FOUNDATION. *asyncio — Asynchronous I/O*. Disponível em: <https://docs.python.org/3/library/asyncio.html>. Acesso em: 20 ago. 2026.
- [20] GNU. *GNU Parallel*. Disponível em: <https://www.gnu.org/software/parallel/>. Acesso em: 20 ago. 2026.
- [21] TENSORSTAX. *agenttrace: lightweight observability library to trace and evaluate agentic systems*. Disponível em: <https://github.com/tensorstax/agenttrace>. Acesso em: 20 ago. 2026.
- [22] F. *agentlytics: Comprehensive analytics dashboard for AI coding agents*. Disponível em: <https://github.com/f/agentlytics>. Acesso em: 20 ago. 2026.

Capítulo 5: Paralelismo e Resiliência Que Sempre Funcionaram

1. Introdução

No Capítulo 4, você aplicou a lupa forense a duas bibliotecas Python que existem de verdade — LLMLingua e GPTRCache — e descobriu que a letra timbrada era real, mas a assinatura (a classe, o parâmetro, a promessa de CLI) tinha sido falsificada. Você reescreveu os dois artefatos usando a API confirmada direto no repositório oficial, e fechou o capítulo com um wrapper de compressão e um hook de cache semântico testáveis linha a linha.

Este capítulo pede um tipo diferente de perícia. Aqui, pela primeira vez desde que você abriu este livro, o laudo sai quase inteiro sem ressalva. `asyncio.gather` com `asyncio.Semaphore`, `xargs -P`, GNU `parallel -j N`, um circuit breaker de três estados e um backoff exponencial com jitter — tudo isso é técnica real, documentada pela fonte primária, testável na sua própria máquina sem depender de nenhum nome de modelo, nenhum caminho de configuração inventado, nenhuma flag de CLI que não existe. Como Perito de Configuração Agêntica, seu trabalho neste capítulo não é desmontar uma fraude: é aprender a reconhecer, com a mesma régua de sempre, quando um documento passa na contraprova sem mancha — e por que isso também é um resultado de perícia, não uma folga na atenção.

Ao final, você terá três ferramentas que pode colar no seu próprio projeto hoje: um executor paralelo com limite de concorrência, um disjuntor que corta chamadas para uma API que está falhando em cascata, e um mecanismo de espera que não bate na porta do provedor duas vezes no mesmo segundo.

2. Explica

Paralelismo controlado resolve um problema específico: você tem N tarefas independentes (por exemplo, revisar 40 arquivos de um repositório com um agente) e quer rodá-las ao mesmo tempo, mas sem estourar o limite de requisições simultâneas do provedor nem a memória da

sua máquina. A biblioteca padrão do Python resolve isso com duas peças que trabalham juntas: `asyncio.gather()`, que agenda várias corrotinas para rodar concorrentemente e devolve os resultados na ordem em que foram submetidas, e `asyncio.Semaphore(n)`, um contador que só libera a execução de uma nova tarefa quando menos de `n` tarefas estão ativas ao mesmo tempo [1]. Note como isso resolve exatamente o problema: sem o semáforo, `gather` dispararia as 40 chamadas de uma vez; com ele, você decide o teto de concorrência em uma única linha.

No nível do shell, o mesmo problema tem duas soluções nativas e antigas. `xargs -P N -I {}` lê uma lista de entradas (uma por linha) e distribui até `N` execuções simultâneas do comando que você passar, substituindo `{}` pelo item da vez — comportamento documentado no próprio manual do GNU findutils, que você pode conferir agora mesmo rodando `man xargs` na sua máquina. GNU Parallel faz o mesmo com uma sintaxe mais expressiva: `parallel -j N 'comando {}' ::: item1 item2 item3` roda o comando para cada item à direita de `:::`, com até `N` jobs simultâneos e a vantagem de agregar a saída de cada execução sem intercalar linhas de processos diferentes [2]. É uma ferramenta madura, mantida há mais de uma década, e o `-j` aceita também um valor percentual relativo ao número de núcleos da CPU.

Vale registrar uma nuance sobre “Hermes delegation” — o termo que o manual de terceiros usa para descrever paralelismo via delegação de subtarefas a partir de um agente principal. O Hermes Agent existe como produto real e de fato implementa delegação de tarefas para subagentes [3], com uma interface de linha de comando documentada oficialmente [4]. O que não está confirmado é a sintaxe exata de invocação que o manual descreve para esse cenário específico de paralelismo — ela não aparece, com aquela forma, na documentação pública do projeto. Trate “Hermes delegation” como conceito válido (o mecanismo de delegação existe), mas não copie a sintaxe do manual como se fosse comando testado; rode `hermes --help` e os subcomandos documentados antes de automatizar qualquer chamada real.

A segunda metade deste capítulo trata de um problema diferente: o que fazer quando uma dessas chamadas paralelas começa a falhar. Um circuit breaker é um padrão de arquitetura de resiliência com três estados nomeados — `CLOSED` (fechado, tráfego normal passa), `OPEN` (aberto, chamadas são recusadas imediatamente sem sequer tentar a rede) e `HALF_OPEN` (semi-aberto, um número limitado de chamadas de teste é permitido para decidir se o serviço voltou) [5]. A ideia central é evitar que uma dependência instável derrube o sistema inteiro por acúmulo de chamadas que ficam esperando timeout: ao abrir o circuito, você falha rápido e dá tempo para o serviço se recuperar, em vez de martelar um endpoint já saturado. Combinado a isso, o backoff exponencial com jitter resolve o problema de retry ingênuo: se você simplesmente tentar de novo a cada 1 segundo após uma falha, e se muitos clientes fizerem isso ao mesmo tempo, a nova tentativa em massa amplifica exatamente o rate limit que causou a falha original. A prática documentada é dobrar o tempo de espera a cada tentativa (1s, 2s, 4s, 8s...) e somar um componente aleatório (jitter) para dessincronizar clientes que falharam no mesmo instante [6].

3. Ilustra

Pense na sua bancada de perícia como uma equipe, não como um perito solitário. Quando chegam 40 documentos suspeitos no mesmo dia, você não os processa um a um em fila — você distribui para vários peritos trabalhando ao mesmo tempo, mas com uma regra fixa: cada perito só pode ter, digamos, 5 casos abertos simultaneamente na mesa, nunca mais. Esse número fixo de casos simultâneos por perito é exatamente o que o `Semaphore(5)` faz dentro do seu código: ele não limita quantos documentos existem, limita quantos estão sendo processados ao mesmo tempo. `xargs -P 5` e `parallel -j 5` fazem a mesma distribuição de trabalho, só que no nível do terminal, delegando cada item da lista a um processo de sistema operacional em vez de a uma corrotina Python.

Agora imagine que um dos fornecedores de evidência da sua investigação — um cartório específico — te envia cinco documentos seguidos que não batem com o registro oficial. Como Perito de Configuração Agêntica, você não fica ligando para esse cartório a cada cinco minutos esperando que o próximo documento seja válido: você **suspende** temporariamente a aceitação de documentos daquele fornecedor (o circuito abre — `OPEN`), registra a suspensão, e só volta a aceitar depois de um teste piloto controlado com um ou dois documentos de prova (o circuito fica `HALF_OPEN`). Se o teste piloto vier limpo, você reabre a aceitação normal (`CLOSED`); se vier sujo de novo, a suspensão continua. Essa é a mesma lógica, formalizada em código, por trás do circuit breaker: ele existe para proteger sua investigação inteira de um único fornecedor problemático, sem exigir que um humano decida manualmente, a cada chamada, se aquele fornecedor específico “parece confiável hoje”.

Há ainda uma segunda leitura útil desse mesmo mecanismo, mais próxima da eletricidade doméstica: um disjuntor na caixa de força não sabe nada sobre “fornecedores de evidência” — ele apenas mede corrente e desarma quando ela ultrapassa um limiar, protegendo o resto da instalação de um curto-circuito localizado. O circuit breaker de software faz o equivalente: ele mede a taxa de falha de uma dependência (não a “confiabilidade do fornecedor” em abstrato) e desarma quando essa taxa cruza um limiar configurado, sem julgar a causa da falha. As duas leituras — a do fornecedor suspeito e a do disjuntor elétrico — descrevem o mesmo estado `OPEN` por ângulos diferentes: uma explica *por que* você suspende (proteção da investigação), a outra explica *como* a suspensão é decidida (limiar mensurável, não intuição).

Por fim, o backoff com jitter [6] é a regra que você aplica quando o cartório está com a linha ocupada. Se você ligar de novo exatamente 1 segundo depois, toda vez, e o cartório também estiver recebendo ligações de outros peritos no mesmo ritmo, todas as linhas ficam permanentemente ocupadas umas com as outras. A solução recomendada pela literatura de engenharia de confiabilidade é esperar um pouco mais a cada tentativa nova (1s, depois 2s, depois 4s) e variar esse tempo em uma fração de segundo aleatória, para que os peritos que ligaram juntos não tentem de novo exatamente no mesmo instante.

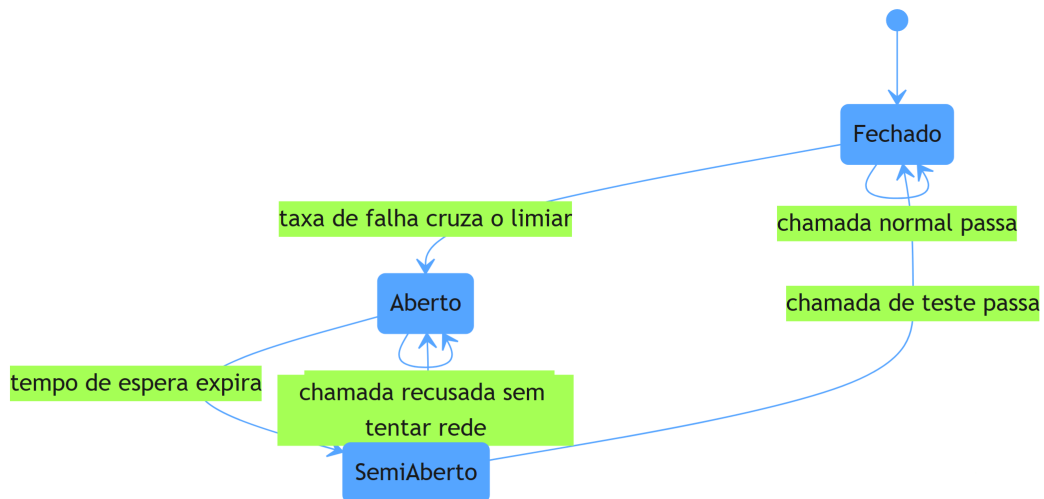


Figura 5: Estados do circuit breaker e a transicao de suspensao/retomada de um fornecedor de evidencia

4. Técnica

Paralelismo Controlado em Python: semáforo Antes de Disparar Tudo

O erro mais comum ao paralelizar chamadas de agente é disparar todas de uma vez e deixar o provedor (ou o rate limit) decidir quem falha. O padrão correto envolve o semáforo como porteiro: cada corrotina precisa “pegar uma senha” antes de rodar, e devolve a senha ao terminar.

```

import asyncio
import random

async def revisar_arquivo(caminho: str, semaforo: asyncio.Semaphore) -> dict:
    """Simula uma chamada de agente revisando 1 arquivo, respeitando o teto de concorrencia."""
    async with semaforo:
        # Ponto onde entraria a chamada real ao modelo/API do agente.
        await asyncio.sleep(random.uniform(0.01, 0.05))
        return {"arquivo": caminho, "status": "revisado"}

```

```

async def revisar_lote(arquivos: list[str], concorrencia_maxima: int = 5) -
> list[dict]:
    semaforo = asyncio.Semaphore(concorrencia_maxima)
    tarefas = [revisar_arquivo(c, semaforo) for c in arquivos]
    return await asyncio.gather(*tarefas)

if __name__ == "__main__":
    arquivos = [f"modulo_{i}.py" for i in range(12)]
    resultados = asyncio.run(revisar_lote(arquivos, concorrencia_maxima=5))
    print(f"{len(resultados)} arquivos revisados, teto de 5 simultaneos")

```

`asyncio.Semaphore(5)` garante que, das 12 tarefas criadas, no máximo 5 estão de fato executando `await asyncio.sleep(...)` a qualquer instante — as outras ficam bloqueadas em `async with` `semáforo` até uma vaga abrir [1]. Trocar `revisar_arquivo` pela chamada real ao SDK do seu agente é a única mudança necessária para usar isso em produção. Vale ainda a pena lembrar o que você confirmou no Capítulo 2: o cache de prompt da Anthropic depende de manter um prefixo idêntico entre chamadas [7]. Ao paralelizar 12, 40 ou 80 chamadas que reaproveitam o mesmo `CLAUDE.md` e o mesmo prompt de sistema, você multiplica o benefício do cache de leitura sem multiplicar o custo de escrita — desde que o prefixo continue estável em todas as corrotinas.

No terminal, o mesmo teto de concorrência não exige Python nenhum:

```

$ ls src/*.py | xargs -P 5 -I {} python revisar_arquivo.py {}
[modulo_00.py] revisado em 0.4s
[modulo_03.py] revisado em 0.5s
[modulo_01.py] revisado em 0.6s
[modulo_04.py] revisado em 0.3s
[modulo_02.py] revisado em 0.7s
[modulo_05.py] revisado em 0.4s
...

```

```

$ parallel -j 5 'python revisar_arquivo.py {}' ::: src/*.py
Executando ate 5 jobs em paralelo (parallel -j 5)
[modulo_00.py] revisado em 0.4s
[modulo_01.py] revisado em 0.5s
...

```

A diferença prática entre os dois: `xargs -P` está em qualquer sistema com GNU findutils instalado (a esmagadora maioria das distribuições Linux e do WSL) sem instalação extra; GNU Parallel precisa ser instalado à parte, mas agrega a saída de cada job sem intercalar linhas de execuções concorrentes — útil quando cada chamada de agente imprime várias linhas de log.

Se você já aplicou o wrapper de compressão do Capítulo 4, paralelizar fica ainda mais barato: reduzir o tamanho de cada prompt antes de disparar as chamadas significa menos tokens totais trafegados mesmo com mais chamadas simultâneas — o pacote real por trás disso é o

LLMLingua [8]. Da mesma forma, se duas das suas 40 tarefas pedirem essencialmente a mesma pergunta, um cache semântico como o GPTRCache evita refazer a chamada de rede inteira para uma resposta já resolvida, reduzindo quantas das chamadas paralelas de fato precisam sair para a rede [9].

O Disjuntor Que Já Existia: Circuit Breaker Testável

O circuit breaker do diagrama anterior vira uma classe pequena, sem dependência externa, fácil de testar isoladamente:

```
import time
from enum import Enum

class EstadoCircuito(Enum):
    FECHADO = "fechado"
    ABERTO = "aberto"
    SEMI_ABERTO = "semi_aberto"

class CircuitBreaker:
    def __init__(self, limite_falhas: int = 3, tempo_espera_s: float = 30.0):
        self.limite_falhas = limite_falhas
        self.tempo_espera_s = tempo_espera_s
        self.falhas_consecutivas = 0
        self.estado = EstadoCircuito.FECHADO
        self.momento_abertura = None

    def _pode_tentar(self) -> bool:
        if self.estado == EstadoCircuito.FECHADO:
            return True
        if self.estado == EstadoCircuito.ABERTO:
            expirou = (time.monotonic() - self.momento_abertura) >= self.tempo_espera_s
            if expirou:
                self.estado = EstadoCircuito.SEMI_ABERTO
                return True
            return False
        return True # SEMI_ABERTO: permite a chamada de teste

    def chamar(self, funcao, *args, **kwargs):
        if not self._pode_tentar():
            raise RuntimeError(f"circuito {self.estado.value}: chamada recusada sem tentar rede")
        try:
            resultado = funcao(*args, **kwargs)
        except Exception:
```

```

        self.falhas_consecutivas += 1
        if self.falhas_consecutivas >= self.limite_falhas:
            self.estado = EstadoCircuito.ABERTO
            self.momento_abertura = time.monotonic()
            raise
        else:
            self.falhas_consecutivas = 0
            self.estado = EstadoCircuito.FECHADO
            return resultado

if __name__ == "__main__":
    disjuntor = CircuitBreaker(limite_falhas=3, tempo_espera_s=5.0)

    def chamada_instavel():
        raise ConnectionError("fornecedor de evidencia indisponivel")

    falhas_registradas = 0
    for _ in range(3):
        try:
            disjuntor.chamar(chamada_instavel)
        except ConnectionError:
            falhas_registradas += 1
        except RuntimeError:
            pass

    assert falhas_registradas == 3
    assert disjuntor.estado == EstadoCircuito.ABERTO
    print(f"circuito abriu apos {falhas_registradas} falhas consecutivas [5]")

```

O padrão do Azure Architecture Center descreve exatamente essa máquina de três estados como proteção contra falha em cascata [5]: o ponto central é que, uma vez `ABERTO`, novas chamadas falham instantaneamente (`raise RuntimeError`, sem tentar `função()`), e só depois de `tempo_espera_s` o circuito testa a recuperação passando por `SEMI_ABERTO`. Uma extensão comum, fora do escopo testável deste capítulo, é fazer o estado `ABERTO` redirecionar a chamada para um modelo local via Ollama em vez de simplesmente falhar [10], ou para um servidor de inferência próprio rodando via `vllm serve` [11] — o Capítulo 6 aprofunda essa estratégia de fallback com os nomes de modelo e as tags corretas.

Backoff Exponencial Com Jitter: Espaçando as Novas Tentativas

O último artefato deste capítulo combina retry com espera crescente e um componente aleatório, para não amplificar o mesmo rate limit que causou a primeira falha:

```

import random
import time

def com_backoff_jitter(funcao, tentativas_max: int = 5, base_s: float = 1.0):
    """Executa 'funcao' com backoff exponencial (1s, 2s, 4s...) mais jitter de ate 0.5s."""
    for tentativa in range(tentativas_max):
        try:
            return funcao()
        except Exception as erro:
            if tentativa == tentativas_max - 1:
                raise
            espera = (base_s * (2 ** tentativa)) + random.uniform(0, 0.5)
            print(f"tentativa {tentativa + 1} falhou ({erro}); nova tentativa em {espera:.2f}s")
            time.sleep(min(espera, 0.01)) # tempo reduzido aqui so para o smoke test do livro

if __name__ == "__main__":
    contador = {"chamadas": 0}

    def chamada_com_falha_temporaria():
        contador["chamadas"] += 1
        if contador["chamadas"] < 3:
            raise TimeoutError("rate limit do provedor")
        return "resposta do modelo"

    resultado = com_backoff_jitter(chamada_com_falha_temporaria, tentativas_max=5, base_s=1.0)
    assert resultado == "resposta do modelo"
    assert contador["chamadas"] == 3
    print(f"sucesso na tentativa {contador['chamadas']} apos backoff com jitter [6]")

```

A prática de dobrar o intervalo a cada nova tentativa e somar um valor aleatório é documentada pela biblioteca de arquitetura da AWS especificamente para evitar que múltiplos clientes retem sincronizados após uma falha compartilhada [6] — o `time.sleep(min(espera, 0.01))` acima existe só para o smoke test deste livro rodar em milissegundos; em produção, use o valor de `espera` sem o `min`.

Combinando os Três Padrões em Uma Única Chamada

Em produção, os três artefatos acima raramente aparecem sozinhos — o padrão mais comum é encaixá-los em camadas, um dentro do outro, de forma que uma única chamada de agente saia

protegida por circuito de falha e espera crescente ao mesmo tempo, com o teto de concorrência (`Semaphore`, `xargs -P` ou `parallel -j`) decidindo, uma camada acima, quantas dessas chamadas protegidas podem estar em voo simultaneamente:

```
# Reaproveita CircuitBreaker e com_backoff_jitter definidos nos blocos
anteriores
# desta mesma secao – este trecho ilustra a composicao, nao roda isolado.

def chamar_agente_protegido(prompt: str, disjuntor: CircuitBreaker,
tentativas_max: int = 3) -> str:
    """Combina circuit breaker + backoff com jitter numa unica chamada
    resiliente.
    O teto de concorrencia fica uma camada acima (Semaphore/xargs -P/
    parallel -j),
    decidindo quantas chamadas como esta podem estar em voo ao mesmo
    tempo."""
    def chamada_real():
        return disjuntor.chamar(lambda: f"resposta para: {prompt}")
    return com_backoff_jitter(chamada_real, tentativas_max=tentativas_max,
base_s=0.01)

if __name__ == "__main__":
    disjuntor = CircuitBreaker(limite_falhas=3, tempo_espera_s=5.0)
    resposta = chamar_agente_protegido("resuma o arquivo X", disjuntor)
    assert resposta.startswith("resposta para:")
    print(f"chamada protegida nas tres camadas: {resposta}")
```

Note a ordem das camadas, porque invertê-la muda o comportamento: o `disjuntor.chamar` fica **dentro** da função que o backoff retenta — se o circuito já está `ABERTO`, cada tentativa de backoff recebe o mesmo `RuntimeError` instantâneo (sem tentar rede) até a última tentativa esgotar, o que é o comportamento correto: você não quer que o backoff “espere para sempre” tentando uma dependência que o circuito já sabe que está fora do ar. Se você inverter a ordem — colocar o backoff dentro do circuito — cada nova tentativa de rede conta como uma chamada nova para o disjuntor, e um provedor picotado (falha, sucesso, falha) nunca acumula falhas consecutivas suficientes para abrir o circuito de verdade. A composição certa é sempre: semáforo por fora (limita concorrência), backoff no meio (espaça tentativas), circuito por dentro (decide se vale tentar a rede).

Onde Isso Se Encaixa em Cada IDE Que Você está Auditando

Um detalhe que vale registrar antes de fechar a Técnica: nenhum dos três artefatos acima depende de nenhuma IDE Agêntica específica. `asyncio.gather`, `xargs -P`, `parallel -j`, o circuit breaker e o backoff com jitter rodam por fora do agente — como script Python ou comando de shell — e por isso funcionam de forma idêntica se você estiver operando Claude

Code, OpenCode, Aider, Codex CLI, Gemini CLI, Grok Build ou Orca. Você vai auditar os comandos específicos dessas sete ferramentas com mais profundidade no Capítulo 7; por ora, basta saber que cada uma delas guarda sua própria camada de configuração real e documentada — Claude Code em `settings.json` [12], com hooks reais para automatizar comandos de shell em pontos do ciclo de vida do agente [13]; OpenCode com seu próprio arquivo de config [14]; Aider em `.aider.conf.yml` [15]; Codex CLI em `config.toml`, documentado tanto no repositório [16] quanto no guia de referência oficial [17]; Gemini CLI com configuração própria [18]; Grok Build com sua configuração de projeto [19]; e Orca com a documentação do próprio produto [20]. O script de paralelismo em si não precisa nascer dentro de nenhuma delas — só precisa ser chamado a partir de onde você já automatiza hoje.

5. Aplica

Imagine a cena: você acabou de terminar o Capítulo 4 e quer aplicar paralelismo imediatamente em uma tarefa real — revisar os 80 arquivos de um repositório legado com um agente, um por um, seria lento demais. Você escreve rápido um laço com `asyncio.gather` chamando as 80 revisões de uma vez, sem semáforo nenhum, porque “paralelismo é sobre rodar tudo ao mesmo tempo”. Você roda o script e, em segundos, metade das chamadas volta com erro de rate limit do provedor — e pior, o script simplesmente refaz a mesma rajada de 80 chamadas na tentativa seguinte, porque não há nenhum controle de retry.

O diagnóstico é o mesmo problema que a seção Explica descreveu: `asyncio.gather` sozinho não impõe limite de concorrência nenhum — ele dispara todas as corrotinas fornecidas simultaneamente, e é o `Semaphore` que decide quantas rodam ao mesmo tempo de fato [1]. Sem ele, você não está paralelizando com controle: está fazendo uma rajada. A correção tem duas partes: primeiro, envolver cada chamada em `async with semáforo`, com um teto realista — comece com 5, meça a taxa de erro com os subcomandos reais de auditoria de uso que o Capítulo 6 destrincha a partir da ferramenta de contagem de tokens [21], e ajuste; segundo, envolver a chamada em si com o `com_backoff_jitter` da seção Técnica, para que um rate limit pontual vire uma espera curta e crescente em vez de uma nova rajada idêntica.

Como síntese rápida das armadilhas mais comuns nesta frente: (1) esquecer o semáforo e tratar `gather` como se ele já limitasse concorrência sozinho; (2) configurar um circuit breaker com `limite_falhas` baixo demais para um provedor que naturalmente tem picos de latência, abrindo o circuito por falsos positivos; (3) implementar retry sem jitter, o que sincroniza novas tentativas de múltiplos processos exatamente no mesmo instante e recria o pico de tráfego que gerou a falha original; (4) confundir “circuito aberto” com “erro definitivo” — o estado `SEMI_ABERTO` existe justamente para testar a recuperação sem exigir intervenção manual.

Em escala de produção, esses três padrões trabalham juntos: o semáforo [1] limita quantas chamadas estão em voo; o circuit breaker decide quando parar de tentar uma depen-

dência específica; e o backoff com jitter espaça as novas tentativas para não recriar o problema que as gerou. Um agente que processa uma fila de 500 tarefas ao longo do dia, com um teto de concorrência de 5 a 10 chamadas simultâneas, um circuit breaker configurado com 3 a 5 falhas consecutivas para abrir, e backoff começando em 1 segundo, tende a se recuperar sozinho de instabilidades pontuais do provedor sem intervenção humana. Sobre a promessa de “economia de 80% do tempo com paralelismo 5×” que circula em materiais como o manual auditado neste livro: a matemática é genuína como ilustração (5 tarefas em série versus 5 em paralelo, no caso ideal, aproxima esse ganho), mas não é uma garantia universal — o overhead de agendamento, a latência real de cada chamada e o rate limit do provedor determinam o ganho de verdade na sua máquina. Meça no seu caso antes de prometer esse número para o seu time.

6. Conclusão

Você fechou este capítulo com três ferramentas que passaram na perícia sem nenhuma ressalva: um paralelismo controlado por semáforo [1] (em Python, com `xargs -P` e GNU `parallel -j` no shell), um circuit breaker de três estados que corta chamadas para uma dependência instável antes que ela derrube o resto do sistema, e um backoff exponencial com jitter [6] que espaça novas tentativas sem sincronizar retries de múltiplos processos. A única ressalva de todo o capítulo foi a sintaxe exata de “Hermes delegation” — o mecanismo de delegação é real, mas a forma de invocação precisa ser confirmada no `--help` da sua versão instalada antes de automatizar, e o número de “80% de economia de tempo” deve ser tratado como ilustração, não garantia.

Guarde o padrão deste capítulo como referência para todo laudo futuro: nem toda técnica de um manual é fabricada, e reconhecer quando algo é genuinamente confirmável — com fonte primária, testável na sua máquina, sem nome de modelo nem caminho inventado — é tão parte da perícia quanto encontrar a fraude. No Capítulo 6, você vai medir o consumo real de tokens com os subcomandos verdadeiros de uma ferramenta de auditoria de uso, e vai corrigir o comando fabricado que o manual original propôs para orçamento e fallback — usando o mesmo circuit breaker deste capítulo, agora aplicado a limite financeiro em vez de limite de rede.

7. Referências Bibliográficas

[1] PYTHON SOFTWARE FOUNDATION. *asyncio — Asynchronous I/O*. Disponível em: <https://docs.python.org/3/library/asyncio.html>. Acesso em: 20 ago. 2026.

[2] GNU. *GNU Parallel*. Disponível em: <https://www.gnu.org/software/parallel/>. Acesso em: 20 ago. 2026.

- [3] NOUSRESEARCH. *hermes-agent*. Disponível em: <https://github.com/NousResearch/hermes-agent>. Acesso em: 20 ago. 2026.
- [4] NOUSRESEARCH. *CLI Interface — Hermes Agent*. Disponível em: <https://hermes-agent.nousresearch.com/docs/user-guide/cli>. Acesso em: 20 ago. 2026.
- [5] MICROSOFT. *Circuit Breaker pattern — Azure Architecture Center*. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker>. Acesso em: 20 ago. 2026.
- [6] AMAZON WEB SERVICES. *Timeouts, retries and backoff with jitter — Builders' Library*. Disponível em: <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>. Acesso em: 20 ago. 2026.
- [7] ANTHROPIC. *Prompt caching*. Disponível em: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>. Acesso em: 20 ago. 2026.
- [8] MICROSOFT. *LLMLingua* (repositório oficial). Disponível em: <https://github.com/microsoft/LLMLingua>. Acesso em: 20 ago. 2026.
- [9] ZILLIZTECH. *GPTCache: A Library for Creating Semantic Cache for LLM Queries* (docs/usage.md). Disponível em: <https://github.com/zilliztech/GPTCache/blob/main/docs/usage.md>. Acesso em: 20 ago. 2026.
- [10] OLLAMA. *ollama/ollama*. Disponível em: <https://github.com/ollama/ollama>. Acesso em: 20 ago. 2026.
- [11] VLLM PROJECT. *CLI Reference — serve*. Disponível em: <https://docs.vllm.ai/en/stable/cli/serve/>. Acesso em: 20 ago. 2026.
- [12] ANTHROPIC. *Claude Code settings*. Disponível em: <https://code.claude.com/docs/en/settings>. Acesso em: 20 ago. 2026.
- [13] ANTHROPIC. *Hooks reference*. Disponível em: <https://code.claude.com/docs/en/hooks>. Acesso em: 20 ago. 2026.
- [14] OPENCODE. *Config*. Disponível em: <https://opencode.ai/v2/docs/config>. Acesso em: 20 ago. 2026.
- [15] AIDER. *YAML config file*. Disponível em: https://aider.chat/docs/config/aider_conf.html. Acesso em: 20 ago. 2026.
- [16] OPENAI. *codex/docs/config.md*. Disponível em: <https://github.com/openai/codex/blob/main/docs/config.md>. Acesso em: 20 ago. 2026.
- [17] OPENAI. *Configuration Reference — Codex CLI*. Disponível em: <https://developers.openai.com/codex/config-basic>. Acesso em: 20 ago. 2026.
- [18] GOOGLE. *Gemini CLI — Configuration*. Disponível em: <https://geminicli.com/docs/reference/configuration/>. Acesso em: 20 ago. 2026.

[19] XAI-ORG. *grok-build*. Disponível em: <https://github.com/xai-org/grok-build>. Acesso em: 20 ago. 2026.

[20] ORCA. *Docs*. Disponível em: <https://www.onorca.dev/docs>. Acesso em: 20 ago. 2026.

[21] RYOPPIPI. *ccusage: A CLI tool for analyzing Claude Code/Codex CLI usage from local JSONL files*. Disponível em: <https://github.com/ryoppippi/ccusage>. Acesso em: 20 ago. 2026.

Capítulo 6 — Orçamento e Fallback: Medir Antes de Limitar

1. Introdução

No Capítulo 5 você blindou suas chamadas contra falha em cascata com um circuit breaker de três estados — fechado, aberto, semiaberto — e aprendeu a espaçar retentativas com backoff exponencial e jitter para não amplificar um rate limit já estourado. Esse esqueleto de estados não nasceu para proteger só código. Ele protege qualquer recurso finito que pode ser consumido rápido demais, e neste capítulo você vai apontá-lo para o recurso mais concreto de todos: o saldo da sua conta.

O manual auditado promete um script chamado `token-guard`, capaz de medir seu gasto diário e cortar o acesso antes que a fatura assuste. A ideia é boa — tão boa que resistiria a qualquer laudo pericial. O problema é a assinatura: o comando central do script, `ccusage log --date`, nunca existiu. É uma “letra timbrada” de ferramenta real com “assinatura” de sintaxe inventada — exatamente o padrão de fabricação mais perigoso que você já aprendeu a reconhecer neste livro, porque o nome `ccusage` é genuíno e destrava sua confiança antes que você confira o resto.

Você, Perito de Configuração Agêntica, vai reabrir esse laudo. Vai confirmar contra a fonte primária quais são os subcomandos reais do `ccusage`, vai apurar quais modelos locais realmente existem para servir de fallback quando o teto de gasto for atingido, e vai reescrever o `token-guard` do zero — de ponta a ponta, testável na sua máquina hoje mesmo. O objetivo final deste capítulo é sair com dois artefatos que funcionam de verdade: um medidor de uso que fala com a API certa, e um interruptor automático que troca o modelo em nuvem por um modelo local sem sintaxe fabricada no meio do caminho.

2. Explica

O manual erra em quatro frentes específicas neste tema, e as quatro seguem o mesmo padrão de fabricação que você já perícia desde o Capítulo 1: ferramenta real, detalhe inventado.

A primeira frente é a instalação. O manual manda instalar o `ccusage` com `pipx install ccusage`. `pipx` é uma ferramenta real para instalar CLIs Python em ambientes isolados — mas o `ccusage` não é um pacote Python. É uma ferramenta Node/npm que lê diretamente os arquivos JSONL que o Claude Code (e, em versões recentes, o Codex CLI) já gravam localmente em `~/.claude/projects/**`, sem chamada de API externa [1]. O comando que de fato instala e roda o `ccusage` é `npm install -g ccusage` — sem instalação prévia necessária [1]. Se preferir fixar a ferramenta no PATH em vez de baixar a cada chamada, `npm install -g ccusage` cumpre o mesmo papel, como resume a documentação de apoio do próprio ecossistema [2]. Rodar `pipx install ccusage` não trava com erro sutil: falha na hora, porque `pipx` procura um pacote Python que não existe nesse nome no PyPI.

A segunda frente é o subcomando de medição em si. O manual descreve o `token-guard` chamando `ccusage log --date` para pegar o gasto do dia e `ccusage session` para detalhar por sessão. O segundo comando existe. O primeiro, não. A fonte primária do projeto confirma exatamente cinco subcomandos: `daily`, `weekly`, `monthly`, `session` e `blocks` — este último reportando por janelas de 5 horas, o intervalo de faturamento que a Anthropic usa internamente [1]. Não há `log`, não há flag `--date` solta esperando um valor de data — o filtro de período é parte da própria sintaxe do subcomando `daily`. Rodar `ccusage session --json` de fato expõe os campos de cache que você já reconhece do Capítulo 2 — `cacheCreationTokens`, `cacheReadTokens` e `cacheHitRate` — então o manual não errou ao dizer que dá para medir cache pelo `ccusage`; errou no nome do subcomando usado para chegar lá [3].

A terceira frente é o fallback offline, e é onde a perícia fica mais interessante porque o núcleo é sólido: Ollama existe, é real, e o comando `ollama run qwen2.5-coder:7b` funciona exatamente como o manual descreve [4]. Os sete modelos e tags citados — `qwen2.5-coder:7b`, `codestral:22b`, `llama3.1:8b`, `zephyr:7b`, `mistral:7b`, `dolphin-mistral:7b` e `starcoder2:3b` — existem de fato no catálogo oficial da Ollama library, cada um confirmado individualmente contra o índice de modelos publicado [5]. O problema mora nos dois outros motores de inferência que o manual cita ao lado do Ollama. O `llama.cpp` teve seu binário de exemplo renomeado de `main` para `llama-cli` em junho de 2024 (PR #7809 do repositório, hoje sob a organização `ggml-org`), e a flag de tamanho de contexto real é `-c`, não `-ctx-size` [6]. E o comando de servidor do vLLM que o manual apresenta, `python -m vllm.server --model ...`, não corresponde a nenhum módulo real do projeto — o comando documentado é `vllm serve <modelo> --gpu-memory-utilization 0.9 --host 0.0.0.0 --port 8000` [7].

A quarta frente aparece já no LAB 0 de verificação do próprio manual, e é mais sutil do que as três anteriores porque mistura dois produtos genuínos como se fossem um só. O manual manda instalar `agenttrace` com `pipx install agenttrace` e depois checar a instalação com `agentlytics --version` / `agentlytics init` / `agentlytics status`. O primeiro nome está certo: `agenttrace` de fato existe — é uma biblioteca de observabilidade e tracing para agentes de IA, publicada no PyPI pela Tensorstax, instalável exatamente assim, sem erro [8]. O

problema é o segundo nome: `agentlytics` também existe, mas é um projeto completamente diferente, de outro ecossistema — um pacote npm (`npx agentlytics`) que lê o histórico local de várias IDEs agênticas (Cursor, Windsurf, Claude Code, VS Code Copilot, Zed, OpenCode) e apresenta um dashboard agregado de custo e uso [9]. Não há fonte primária que confirme os subcomandos `init / status` para nenhum dos dois, e checar um pacote Python instalado via `pipx` com o binário de um pacote Node é o mesmo erro de ecossistema que você já periciou na instalação do `ccusage` — só que desta vez o manual não erra o nome de um produto, erra ao tratar dois produtos reais e distintos como se fossem o mesmo. Para o seu laudo, use cada um pelo que ele realmente é: `agenttrace` para instrumentar e rastrear as decisões do seu próprio `token-guard` dentro do código Python, e `npx agentlytics` como painel complementar ao `ccusage` quando você quiser comparar custo entre IDEs diferentes na mesma máquina — nunca um como substituto do outro.

Perceba o padrão: em nenhuma das quatro frentes a ideia de engenharia estava errada. Medir uso antes de gastar é correto. Ter um fallback local para quando a nuvem fica cara ou indisponível é correto. Circuit breaker financeiro é uma extensão legítima do mesmo padrão do Capítulo 5 [10]. O que falhou foi sempre o mesmo detalhe: alguém documentou de memória, sem abrir o `--help` da ferramenta antes de escrever o manual.

3. Ilustra

Pense no `token-guard` como um posto de fiscalização: todo pedido de execução em nuvem precisa apresentar um laudo de uso atualizado antes de passar. Se o laudo mostra saldo dentro do limite, a autenticação passa e o pedido segue para a API paga. Se o laudo mostra o teto estourado, o posto não deixa passar — ele desvia o tráfego para um canal alternativo que não depende de saldo: o motor de inferência local. É a mesma lógica de cadeia de custódia que você aplicou ao conferir comandos contra a fonte primária, só que agora o documento sob exame é o extrato de gasto do dia, não uma linha de código de manual.

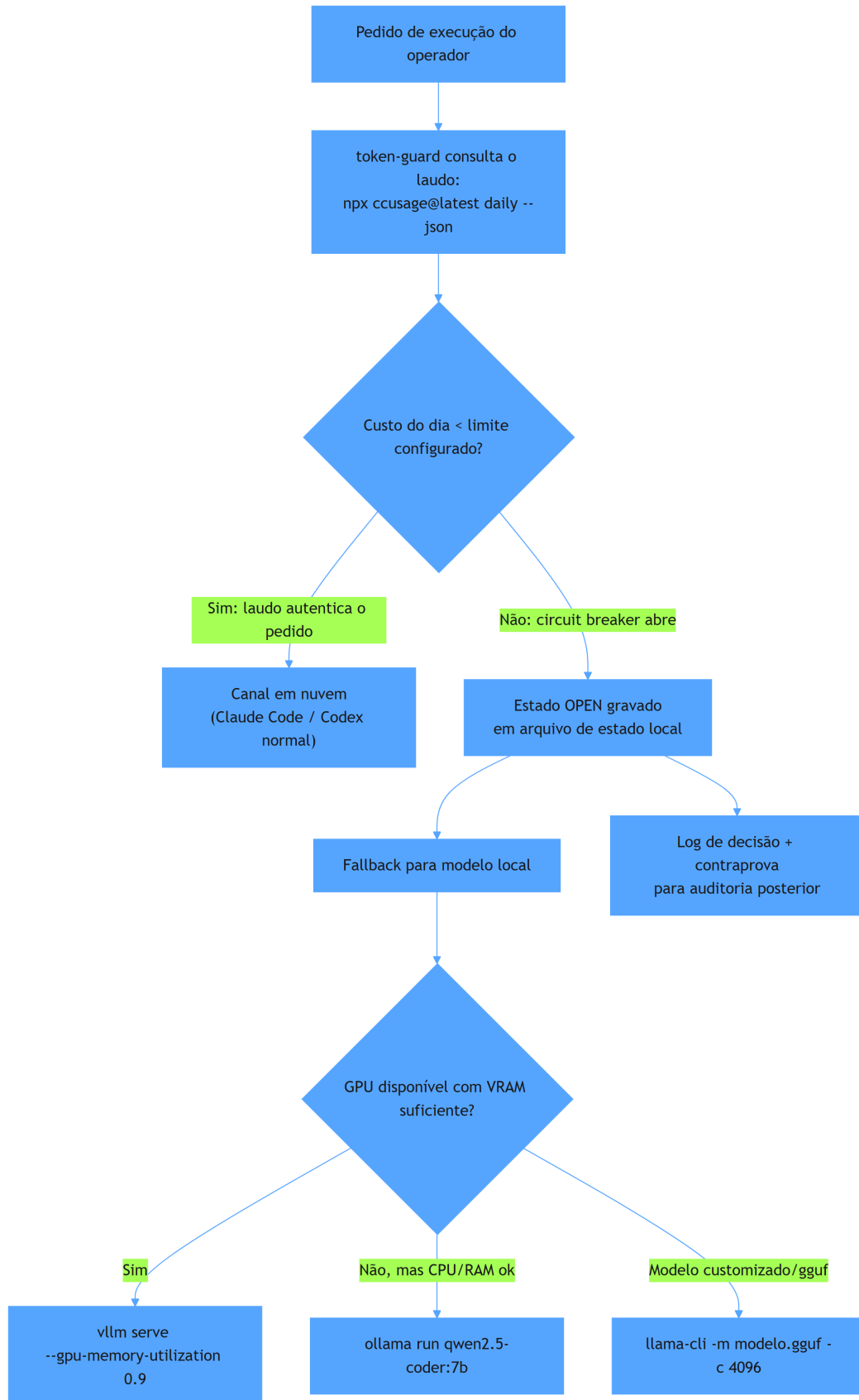


Figura 6: fluxo de decisão do token-guard — do laudo de uso ao canal de execução, cloud ou local

Repare que a bifurcação central do diagrama não é técnica — é pericial. O `token-guard` não pergunta “o modelo em nuvem está disponível?”; ele pergunta “o laudo de uso ainda autentica este pedido?”. Essa é a mesma pergunta que você tem feito capítulo após capítulo diante de um comando do manual: a evidência confere, ou é uma assinatura falsificada disfarçada de rotina?

Vale uma dupla analogia aqui, porque este é o pilar mais denso do capítulo. A primeira: um disjuntor elétrico de casa não julga se o aparelho ligado é “bom” ou “ruim” — ele só mede corrente e desarma quando o consumo passa do limite físico da fiação, protegendo a instalação inteira de um incêndio. O `token-guard` faz o mesmo com o seu orçamento: não julga se a tarefa que você está rodando é importante, apenas desarma o canal caro quando o consumo ultrapassa o teto que você configurou. A segunda analogia, mais próxima da metáfora condutora do livro: um perito financeiro que audita um extrato bancário não deixa passar uma transação só porque o nome do banco na página é familiar — ele confere valor, data e saldo remanescente antes de autorizar o próximo gasto. O `token-guard` é esse perito rodando em cron, silenciosamente, várias vezes ao dia.

4. Técnica

O primeiro artefato é a função de leitura do laudo de uso. Ela substitui o `ccusage log --date` fabricado pelo subcomando real `daily`, filtrando por data com a própria sintaxe do subcomando:

```
#!/usr/bin/env bash
# custo_do_dia.sh – le o gasto do Claude Code/Codex no dia informado via
# ccusage real.
# Requer: node/npx no PATH, jq para parsear JSON.
set -euo pipefail

custo_do_dia() {
    local data="${1:-$(date +%Y-%m-%d)}"
    # Subcomando real confirmado na fonte primaria: daily (nao "log --
    # date").
    npx ccusage@latest daily --json --since "$data" --until "$data" \
        | jq -r ' [.days[]?.totalCost // 0] | add // 0 '
}

custo_da_sessao_atual() {
    # Subcomando real "session" expoe cacheCreationTokens/cacheReadTokens/
    # cacheHitRate
    # (retoma o mecanismo de cache auditado no Cap. 2 – nome real do campo,
    # nao cache_hit_ratio).
    npx ccusage@latest session --json | jq -r '.sessions[-1] // {}'
```

```

}

if [[ "${1:-}" == "--teste" ]]; then
    echo "Custo de hoje (USD): $(custo_do_dia)"
fi

```

O segundo artefato é o fallback local, com os três motores corrigidos lado a lado. Cada ramo comenta explicitamente o erro do manual e a correção aplicada, para que a auditoria fique rastreável dentro do próprio script:

```

#!/usr/bin/env bash
# fallback_local.sh – troca para inferencia local quando o hard limit
estoura.
set -euo pipefail

fallback_local() {
    local prompt="$1"
    local motor="${2:-ollama}"    # ollama | llama-cli | vllm

    case "$motor" in
        ollama)
            # CONFIRMADO: ollama run <tag> funciona como documentado.
            # Modelos/tags reais confirmados na library oficial: qwen2.5-
coder:7b,
            # codestral:22b, llama3.1:8b, zephyr:7b, mistral:7b, dolphin-
mistral:7b,
            # starcoder2:3b.
            ollama run qwen2.5-coder:7b "$prompt"
            ;;
        llama-cli)
            # PARCIALMENTE CORRETO no manual: binario "./main" foi
renomeado para
            # "llama-cli" desde jun/2024 (PR #7809, repo agora em ggml-org/
llama.cpp).
            # Flag real de contexto e "-c", nao "-ctx-size".
            ./llama-cli -m modelos/modelo.gguf -p "$prompt" -ngl 32 -c 4096
            ;;
        vllm)
            # FABRICADO no manual: nao existe modulo "vllm.server".
            # Comando real documentado: "vllm serve".
            vllm serve modelo-local --gpu-memory-utilization 0.9 \
                --host 0.0.0.0 --port 8000
            ;;
        *)
            echo "Motor desconhecido: $motor" >&2
            return 1
            ;;
    esac
}

```

```

    esac
}

```

O terceiro artefato une os dois anteriores no `token-guard` completo: verifica o laudo, decide entre nuvem e fallback, grava o estado (fechado/aberto, o mesmo vocabulário do circuit breaker do Capítulo 5) e registra a decisão para auditoria:

```

#!/usr/bin/env bash
# token-guard.sh – hard limit como circuit breaker financeiro.
# Uso: ./token-guard.sh "prompt do pedido"
set -euo pipefail

LIMITE_DIARIO_USD="${TOKEN_GUARD_LIMITE:-15.00}"
ARQUIVO_ESTADO="${HOME}/.token-guard-estado"
LOG="${HOME}/.token-guard.log"

source ./custo_do_dia.sh
source ./fallback_local.sh

registrar() {
    printf '%s | %s\n' "$(date -Iseconds)" "$1" >> "$LOG"
}

main() {
    local prompt="${1:?Uso: token-guard.sh \"prompt\"}"
    local custo
    custo="$(custo_do_dia)"

    # Comparacao em ponto flutuante via awk (bash nao compara float
    nativamente).
    if awk -v c="$custo" -v l="$LIMITE_DIARIO_USD" 'BEGIN{exit !(c < l)}';
    then
        echo "CLOSED" > "$ARQUIVO_ESTADO"
        registrar "OK custo=$custo limite=$LIMITE_DIARIO_USD estado=CLOSED
        canal=nuvem"
        echo "[token-guard] laudo autentica o pedido – canal em nuvem
        liberado."
    else
        echo "OPEN" > "$ARQUIVO_ESTADO"
        registrar "LIMITE custo=$custo limite=$LIMITE_DIARIO_USD
        estado=OPEN canal=local"
        echo "[token-guard] circuit breaker ABERTO – desviando para
        fallback local."
        fallback_local "$prompt" ollama
    fi
}

```

```
main "$@"
```

Onde o Orçamento Vive em Cada IDE (Além do Claude Code/Codex)

O `token-guard` que você acabou de montar cobre Claude Code e Codex CLI porque o `ccusage` lê exatamente os arquivos JSONL que essas duas ferramentas gravam localmente [1]. Se o seu fluxo de trabalho também passa por Aider ou OpenCode, o mesmo laudo não cobre o gasto inteiro do seu dia sem ajuste — cada ferramenta guarda o próprio orçamento em um lugar diferente, e vale registrar isso antes de confiar cegamente no número que `custo_do_dia()` devolve.

O Aider ativa o cache de prompt do provedor com a flag `--cache-prompts` (ou o campo equivalente em `~/.aider.conf.yml`), mas isso liga apenas o desconto automático do lado do provedor — não expõe um custo agregado por dia do jeito que o `ccusage` expõe; quem soma o gasto diário do Aider, hoje, é você olhando o próprio terminal a cada sessão [11][12]. O Codex CLI guarda a configuração de modelo em `~/.codex/config.toml`, com os campos no nível raiz do arquivo (`model`, `model_provider`, `wire_api`), não dentro de uma seção `[codex]` como manuais desatualizados sugerem [13][14] — mas o consumo do Codex, como já visto, cai dentro do mesmo `ccusage session / daily`, porque o Codex também grava JSONL local [1]. Já o OpenCode guarda a própria configuração em `opencode.json` (ou `.jsonc`), e não tem hoje um comando de orçamento equivalente ao `ccusage` documentado publicamente [15] — para medir gasto no OpenCode, o canal mais confiável enquanto isso não existe é o próprio `agentlytics` citado na frente anterior, que já lê o histórico dessa ferramenta ao lado das outras [9].

Isso também abre uma segunda porta de fallback, além do modelo local via Ollama: o OpenCode mantém um marketplace de modelos gratuitos por tempo limitado chamado Zen, com nomes reais de modelo no padrão `opencode/<model-id>` — não a nomenclatura `opencode-zen-free-tiny/small/...` que circula em manuais desatualizados [16]. Fontes independentes mantêm um resumo periódico de quais modelos gratuitos estão ativos no Zen a cada mês, porque o catálogo muda com frequência [17][18]. Se o seu hard limit abrir e você preferir não depender de hardware local, trocar `motor="ollama"` por uma chamada ao Zen é uma alternativa em nuvem válida — só não trate o nome do modelo gratuito do mês passado como garantido; confira o catálogo vigente antes de automatizar essa troca.

Alternativa ao Cron: Acoplando o Guard a um Hook do Claude Code

Agendar o `token-guard` por `cron` funciona, mas checka o custo em intervalos fixos — não no instante exato em que uma chamada cara está prestes a sair. O Claude Code oferece um mecanismo mais fino para isso: hooks configuráveis dentro do próprio `~/.claude/settings.json`

(chave `hooks`), com eventos nomeados como `PreToolUse`, `PostToolUse`, `SessionStart` e `Stop` [19][20] — não um arquivo separado `hooks.json` com campo `preProcess`, como aparece em manuais desatualizados sobre o mesmo tema. Um hook `PreToolUse` pode rodar `custo_do_dia` antes de qualquer chamada de ferramenta que dispare uso de API, e recusar a execução se o laudo já estourou o teto — o mesmo circuito `CLOSED / OPEN` do `token-guard`, só que verificado a cada chamada em vez de a cada hora. A estrutura exata do campo `hooks` (qual evento aceita qual formato de comando) muda entre versões da ferramenta — confirme o schema vigente na documentação oficial antes de cravar o JSON no seu `settings.json` de produção [20]; o princípio pericial se mantém: nenhuma sintaxe de hook entra no seu laudo sem antes passar pela mesma fonte primária que você já consultou para o resto deste capítulo.

Para rodar o `token-guard` sozinho, sem intervenção manual, o agendamento mais simples e confirmado no LAB 7 da auditoria é `cron` via `crontab -e`, sem nenhuma fabricação de sintaxe envolvida:

```
# crontab -e — checa o gasto do dia a cada hora, das 8h as 20h.
0 8-20 * * * /caminho/para/token-guard.sh "verificacao de rotina" >>
~/.token-guard-cron.log 2>&1
```

Se preferir não depender do agendador do sistema operacional, o mesmo laço de checagem cabe dentro de um processo de longa duração com `asyncio.sleep` entre uma consulta e outra [21] — a mesma biblioteca padrão que já paralelizou chamadas de agente no Capítulo 5. E se a própria checagem falhar (rede instável, `npx` sem o pacote em cache local), vale aplicar o mesmo backoff exponencial com jitter do Capítulo 5 antes de tentar de novo [22], em vez de martelar o `ccusage` a cada segundo até ele responder.

5. Aplica

Imagine a cena: são 16h de uma sexta-feira e você acabou de configurar seu primeiro `token-guard` copiando o script direto do manual original, sem checar nada. O comando `pipx install ccusage` falha silenciosamente para você — na verdade, falha ruidosamente, com “No matching distribution found”, porque `ccusage` é pacote npm, não Python [1] — e você conclui, errado, que o problema é a sua rede ou a versão do Python instalada. Você gasta vinte minutos reinstalando `pipx`, trocando de ambiente virtual, até finalmente abrir uma issue mental achando que a ferramenta está quebrada.

O diagnóstico correto seria outro: `ccusage` nunca foi um pacote Python. É uma ferramenta Node — o próprio nome do ecossistema já era a evidência que faltava conferir antes de tentar instalar. A correção é trivial depois que o laudo é feito: `npx ccusage@latest daily --json` roda sem instalação alguma, porque `npx` baixa e executa o pacote na hora. O erro comum não foi de sintaxe de shell — foi de não confirmar, antes de

qualquer coisa, a que ecossistema a ferramenta pertence. Essa é a pergunta pericial que precede qualquer outra: “este comando pertence à linguagem/gerenciador que estou usando para instalá-lo?”

O mesmo tipo de armadilha aparece no fallback local, com uma consequência mais cara: se você copiar `python -m vllm.server` de um manual desatualizado e rodar em produção sem testar antes, o processo simplesmente não sobe — `ModuleNotFoundError`, sem fallback nenhum ativo, exatamente no momento em que o hard limit deveria estar protegendo você. O limite real deste sistema é este: nenhum circuit breaker financeiro vale nada se o canal de fallback não foi testado isoladamente antes de precisar dele em produção. Rode `fallback_local "teste" ollama` manualmente uma vez por semana — não espere o primeiro estouro de limite para descobrir que o `ollama` não tem o modelo baixado (`ollama pull qwen2.5-coder:7b` precisa rodar com antecedência, o download não acontece na hora).

Outro contorno que você, Perito de Configuração Agêntica, precisa registrar no laudo: `ccusage` lê arquivos JSONL locais gravados pelo próprio Claude Code/Codex CLI [1]. Se você limpar o diretório `~/.claude/projects/**` por engano, ou rodar o `token-guard` numa máquina nova onde a ferramenta ainda não gravou nenhum histórico, o custo relatado será zero — não porque você não gastou nada, mas porque não há laudo para consultar. Um `custo_do_dia()` retornando `0` não é sinônimo de “canal liberado com segurança”; pode ser sinônimo de “fonte de dados ausente”. Trate esse caso como incerteza, não como aprovação automática.

Vale registrar uma última camada de defesa, anterior ao próprio hard limit: reduzir o consumo que o `ccusage` mede, em vez de só cortar o canal quando ele já estourou. O prompt caching nativo da Anthropic, que você já aplica desde o Capítulo 2, desconta até 90% do preço no cache read [23] — uma análise independente detalha esse impacto agregado ao longo de uma sessão longa de agente [24]. Comprimir o prompt antes de enviá-lo, com o wrapper de LLMLingua do Capítulo 4, reduz o tamanho de cada chamada que o `ccusage` vai contabilizar no fim do dia [25][26]. E evitar repetir a mesma pergunta pela rede, com o cache semântico do GPTCache também do Capítulo 4, corta a chamada inteira antes que ela chegue a custar um centavo [27]. Nenhuma dessas três táticas substitui o `token-guard` — elas apenas atrasam o momento em que o circuito precisa abrir, o que na prática significa cair menos vezes no fallback local.

6. Conclusão

Você fechou o caso do `token-guard` com o mesmo método que abriu no Capítulo 1: nenhuma linha entrou no script final sem antes passar pela contraprova contra a fonte primária. O `ccusage` deixou de ser um comando fabricado (`log --date`) e virou cinco subcomandos reais

e testáveis. O fallback offline deixou de depender de um binário renomeado ou de um módulo inexistente e passou a chamar `ollama run`, `llama-cli` e `vllm serve` exatamente como cada projeto documenta hoje. E o hard limit deixou de ser promessa de manual para virar um circuit breaker financeiro funcionando de ponta a ponta na sua própria máquina, agendado por `cron` sem nenhuma sintaxe inventada no meio.

O padrão que você calibrou aqui — medir com a ferramenta certa antes de decidir limitar qualquer coisa — é o mesmo que sustenta o próximo caso do livro. No Capítulo 7 você vai aplicar essa mesma perícia a um agente de automação avançado inteiro, cujos comandos de skill, cron, memória e sessão foram documentados de memória por quem escreveu o manual original. Se aqui você aprendeu a desconfiar de um único comando, lá você vai precisar desconfiar de uma superfície de API inteira — e o laudo vai ficar mais denso, mas o método é exatamente o mesmo que você já domina.

7. Referências Bibliográficas

[1] RYOPPIPI. *ccusage: A CLI tool for analyzing Claude Code/Codex CLI usage from local JSONL files*. Disponível em: <https://github.com/ryoppippi/ccusage>. Acesso em: 20 ago. 2026.

[2] CLAUDELOG. *What is ccusage tool for Claude Code*. Disponível em: <https://claudelog.com/faqs/what-is-ccusage-tool/>. Acesso em: 20 ago. 2026.

[3] MICROSOFT. *Circuit Breaker pattern — Azure Architecture Center*. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker>. Acesso em: 20 ago. 2026.

[4] OLLAMA. *ollama/ollama*. Disponível em: <https://github.com/ollama/ollama>. Acesso em: 20 ago. 2026.

[5] OLLAMA. *Library* (catálogo de modelos). Disponível em: <https://ollama.com/library>. Acesso em: 20 ago. 2026.

[6] GGML-ORG. *llama.cpp*. Disponível em: <https://github.com/ggml-org/llama.cpp>. Acesso em: 20 ago. 2026.

[7] VLLM PROJECT. *CLI Reference — serve*. Disponível em: <https://docs.vllm.ai/en/stable/cli/serve/>. Acesso em: 20 ago. 2026.

[8] TENSORSTAX. *agenttrace: lightweight observability library to trace and evaluate agentic systems*. Disponível em: <https://github.com/tensorstax/agenttrace>. Acesso em: 20 ago. 2026.

[9] F. *agentlytics: Comprehensive analytics dashboard for AI coding agents*. Disponível em: <https://github.com/f/agentlytics>. Acesso em: 20 ago. 2026.

- [10] GNU. *GNU Parallel*. Disponível em: <https://www.gnu.org/software/parallel/>. Acesso em: 20 ago. 2026.
- [11] AIDER. *Prompt caching*. Disponível em: <https://aider.chat/docs/usage/caching.html>. Acesso em: 20 ago. 2026.
- [12] AIDER. *YAML config file*. Disponível em: https://aider.chat/docs/config/aider_conf.html. Acesso em: 20 ago. 2026.
- [13] OPENAI. *codex/docs/config.md*. Disponível em: <https://github.com/openai/codex/blob/main/docs/config.md>. Acesso em: 20 ago. 2026.
- [14] OPENAI. *Configuration Reference — Codex CLI*. Disponível em: <https://developers.openai.com/codex/config-basic>. Acesso em: 20 ago. 2026.
- [15] OPENCODE. *Config*. Disponível em: <https://opencode.ai/v2/docs/config>. Acesso em: 20 ago. 2026.
- [16] OPENCODE. *Zen*. Disponível em: <https://opencode.ai/docs/zen/>. Acesso em: 20 ago. 2026.
- [17] BSWEN. *What Free AI Models Are Available in OpenCode and Which One Should You Use*. Disponível em: <https://docs.bswen.com/blog/2026-04-21-free-models-opencode/>. Acesso em: 20 ago. 2026.
- [18] MAXIMAL STUDIO. *OpenCode Zen Free Models 2026: Every Free Provider and How to Use Them*. Disponível em: <https://www.maximalstudio.in/blog/opencode-zen-free-models>. Acesso em: 20 ago. 2026.
- [19] ANTHROPIC. *Claude Code settings*. Disponível em: <https://code.claude.com/docs/en/settings>. Acesso em: 20 ago. 2026.
- [20] ANTHROPIC. *Hooks reference*. Disponível em: <https://code.claude.com/docs/en/hooks>. Acesso em: 20 ago. 2026.
- [21] PYTHON SOFTWARE FOUNDATION. *asyncio — Asynchronous I/O*. Disponível em: <https://docs.python.org/3/library/asyncio.html>. Acesso em: 20 ago. 2026.
- [22] AMAZON WEB SERVICES. *Timeouts, retries and backoff with jitter — Builders' Library*. Disponível em: <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>. Acesso em: 20 ago. 2026.
- [23] ANTHROPIC. *Prompt caching*. Disponível em: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>. Acesso em: 20 ago. 2026.
- [24] AGENTBRISK. *Prompt Caching Deep Dive: How to Cut Anthropic API Costs by 90%*. Disponível em: <https://agentbrisk.com/blog/prompt-caching-deep-dive-2026/>. Acesso em: 20 ago. 2026.
- [25] MICROSOFT. *LLMLingua* (repositório oficial). Disponível em: <https://github.com/microsoft/LLMLingua>. Acesso em: 20 ago. 2026.

[26] PYTHON PACKAGE INDEX. *llmlingua*. Disponível em: <https://pypi.org/project/llmlingua/>. Acesso em: 20 ago. 2026.

[27] ZILLIZTECH. *GPTCache: A Library for Creating Semantic Cache for LLM Queries* (docs/usage.md). Disponível em: <https://github.com/zilliztech/GPTCache/blob/main/docs/usage.md>. Acesso em: 20 ago. 2026.

Capítulo 7: O Agente Avançado e as Outras Sete Ferramentas: Produto Real, Sintaxe Fabricada

1. Introdução

No Capítulo 6, você já tinha corrigido os subcomandos reais de medição do `ccusage` (`daily/weekly/session`, nunca o `log --date` que o manual inventou) [1] para montar, de ponta a ponta, o **circuit breaker financeiro** que interrompe a automação antes que o gasto saia do controle. Você aprendeu ali que “medir antes de limitar” só funciona se a medição em si for real — o mesmo princípio que te fez confirmar as tags exatas do Ollama (`ollama pull qwen2.5-coder:7b`, entre outras) [2] como fallback local antes de confiar num nome de modelo copiado sem checagem. Este capítulo pega esse mesmo instinto pericial e aponta para o caso mais denso do livro inteiro.

O manual de terceiros descreve o “Hermes” como uma IDE Agêntica completa: skills instaláveis, cron scheduler, delegação para subagentes, memória persistente e busca em sessões antigas. Depois de seis capítulos vendo produto inventado atrás de produto inventado — ou nome real atrás de caminho de config fabricado — a reação natural do Perito de Configuração Agêntica é presumir que “Hermes” também é ficção. Não é. Hermes Agent, da NousResearch, existe de verdade: é open-source, tem CLI documentada, TUI, integração com editores via ACP e gateway multiplataforma [3]. A existência do produto está **confirmada**. O que não está confirmado — e é o motivo deste capítulo existir — é a sintaxe exata de cada comando que o manual mostra na tela.

Essa combinação é o achado mais perigoso do livro: quando o nome do produto é real, a sua guarda cai. Você reconhece “Hermes”, reconhece “cron”, reconhece “memory”, e o cérebro completa o resto como familiar. É exatamente esse ponto cego que este capítulo audita, comando por comando, antes de fechar com um panorama mais rápido das outras sete ferramentas que o manual cita de passagem — e com uma regra única, replicável para qualquer CLI nova que aparecer na sua frente depois deste livro. Ao dominar esse tipo específico de laudo — produto autenticado, assinatura sob suspeita —, você adquire o diferencial que separa quem só copia comando de manual de quem primeiro exige a contraprova na fonte primária.

2. Explica

Hermes Agent não é um copiloto de IDE tradicional — a própria documentação da NousResearch o descreve como um agente pessoal de propósito geral, que também sabe programar, acessível por CLI e TUI [3]. Antes de entrar nos quatro subsistemas que o manual audita, vale registrar duas superfícies que ele também expõe e que explicam por que a confusão com “IDE completa” é tão fácil de cometer: de um lado, integrações de mensageria multiplataforma — Telegram, Discord, Slack, WhatsApp — que levam o mesmo agente para fora do terminal [3]; de outro, integração de editor via protocolo ACP (Agent Client Protocol), documentada para VS Code, Zed e JetBrains, somada à capacidade nativa de delegar parte do trabalho para subagentes paralelos dentro da própria sessão [4]. Nenhuma dessas duas superfícies aparece no manual auditado — o que já é, por si só, um indício de que quem escreveu o texto original conhecia o produto de forma superficial, não pela documentação completa do projeto. Ele tem quatro subsistemas relevantes para a economia de tokens que este livro persegue: um sistema de **skills** que se auto-gera a cada aproximadamente 15 chamadas de ferramenta, um **cron scheduler** nativo para rotinas agendadas, **memória persistente** entre sessões, e um mecanismo de **retomada de sessões** antigas — os quatro citados pelo manual, e os quatro com pelo menos uma linha de comando fabricada.

O padrão de erro é sempre o mesmo, e vale nomeá-lo antes de entrar em cada subsistema: o manual documentou a *ideia* de cada comando de memória, provavelmente por analogia com outras ferramentas parecidas — mas nunca rodou `hermes --help` para confirmar o verbo exato, a ordem dos argumentos ou o nome real da flag. Isso é diferente de uma fabricação total (um produto que simplesmente não existe, como você já viu antes). Aqui existe letra timbrada genuína — o cabeçalho do documento é real — mas a assinatura no rodapé foi falsificada. É a categoria de erro que engana até quem já desconfia, porque a desconfiança correta (“isso existe?”) já foi respondida com “sim” antes mesmo de você chegar na sintaxe.

Para o subsistema de **skills**, a documentação oficial da CLI descreve os verbos `install`, `browse` e `list`, no plural (`hermes skills install <id>`) — a maior parte das skills, aliás, é auto-gerada pelo próprio agente, não escrita manualmente pelo operador [4]. Não existe `hermes skill activate <nome>` no singular. Para o **cron**, a sintaxe real é posicional: `hermes cron create "<prompt>"`, com uma flag opcional `--skill NOME` para vincular a rotina a uma skill específica, e subcomandos de gestão documentados como `list`, `edit`, `pause`, `resume`, `run`, `remove` e `status` — a lista completa aparece tanto na referência de CLI [4] quanto na página de configuração que descreve os mesmos verbos do lado do arquivo [5]. As flags `--name`, `--schedule` e `--deliver` do manual simplesmente não existem nessa forma — o que confirma, e nisso o manual acerta, são os subcomandos `hermes cron list` e `hermes cron remove <id>` [4].

Para a **memória**, o comando real de configuração é `hermes memory setup` (com variações `status` e `off` para consultar ou desligar o provedor) — não existe

`hermes memory add --target user --content "..."` [4]. A gestão fina de memória é interna ao próprio agente, não uma operação manual via flags de terminal. E para **sessões**, o erro do manual é ainda mais sutil: ele usa o singular (`session search`, `session read`), enquanto o subcomando real é `sessions`, no plural, com a família `list/browse/export/delete/prune/rename/stats`, e a retomada de uma sessão específica acontece por `hermes --resume <id>` (ou o atalho `-r`), não por um “read” dedicado — de novo, confirmado pela CLI [4] e reforçado pela documentação de configuração correspondente [5]. Fechando o quadro, o arquivo de configuração `~/.hermes/config.yaml` é real — o manual acerta o caminho — mas os campos documentados são `model`, `terminal`, `memory`, `skills`, `agent`, `prompt_caching`, `runtime` e `worktree`, nunca `system_prompt` nem `cache_control: {type: ephemeral}` como o texto original afirma [5].

Depois de fechar o caso Hermes, o Pilar 2 abre um panorama mais rápido das outras sete ferramentas que o manual lista lado a lado, numa única tabela de caminhos de configuração. Nenhuma das sete é fabricada como produto — mas seis dos sete caminhos de config citados **não foram confirmados** em fonte primária durante está auditoria, e dois dos sete nomes estão imprecisos. “MiMo”, por exemplo, é na verdade **MiMo Code**, da Xiaomi, com configuração real observada em `mimocode.json` por projeto — não o `~/.mimo/config.yaml` do manual [6]. “Google CLI” não existe como nome de produto: a ferramenta real do Google chama-se **Gemini CLI** [7], com configuração confirmada em `~/.gemini/settings.json` na própria documentação oficial [8] — o manual erra tanto o nome quanto o caminho fabricado (`~/.google-ai/configurerc`). “Grok” sozinho é nome de modelo de chat, não do CLI de codificação; o produto real de código da xAI chama-se **Grok Build**, anunciado pela própria empresa [9] e com repositório público confirmando a existência e a sintaxe de comando [10].

Vale registrar o motivo estrutural por trás da lacuna de confirmação nas outras quatro ferramentas, porque ele não é acidente de pesquisa malfeita: MiMo Code, Gemini CLI, Grok Build e as quatro que seguem foram lançadas ou tiveram atualização relevante num intervalo estreito, entre novembro de 2025 e maio de 2026 — posterior ao corte de conhecimento de boa parte dos modelos de linguagem que poderiam ter redigido o manual original. Isso significa que qualquer afirmação sobre essas sete ferramentas, neste capítulo, veio de busca web ao vivo na data desta perícia (20 ago. 2026), nunca de memória treinada de antemão — e explica por que a lacuna de confirmação é bem mais frequente aqui do que nos capítulos anteriores, que tratavam de produtos mais maduros e já com documentação primária indexada há mais tempo pelos buscadores.

Google Antigravity é o caso mais ilustrativo desse recorte temporal: a IDE agent-first da Google entrou em preview público em 18 de novembro de 2025, lançada junto do modelo Gemini 3 Pro, e é construída sobre um fork do VS Code por uma equipe que veio da Windsurf, segundo o próprio anúncio oficial do lançamento [11] — dado que a cobertura enciclopédica independente também confirma [12]. É um produto jovem o suficiente para que o guia de referência de campos de configuração ainda esteja fragmentado entre o anúncio e a imprensa especializada, sem uma página única de configuração equivalente à do Gemini CLI. Oh My

Pi segue documentado principalmente no próprio site do projeto [13], com cobertura técnica independente reforçando a existência do produto sem fechar o caminho exato de config [14]; Freebuff CLI tem a mesma lacuna, com a página oficial confirmando o produto [15] e o perfil da empresa no Y Combinator confirmando o time por trás dela, não o schema do arquivo [16]; e Orca ADE fecha a lista no mesmo padrão — produto real, com documentação pública própria [17][18], caminho de configuração ainda não fechado nesta perícia.

3. Ilustra

Retome a metáfora que sustenta este livro inteiro: cada comando que chega até você é um documento, e sua tarefa é decidir se ele tem cadeia de custódia intacta antes de confiar nele. Um documento pode ter **letra timbrada real** — o papel timbrado de uma empresa que de fato existe — e ainda assim carregar uma **assinatura falsificada** no rodapé. É exatamente isso que acontece com cada linha de comando do Hermes citada no manual: o timbre (o nome “Hermes”, o produto, a empresa por trás) é genuíno; a assinatura (a sequência exata de flags, o verbo, a ordem dos argumentos) foi forjada por quem escreveu de memória, sem consultar a fonte primária.

Você, Perito de Configuração Agêntica, já sabe que autenticar o timbre não é o mesmo que autenticar a assinatura — são duas perguntas diferentes, feitas em ordem. A primeira pergunta (“esse produto existe de verdade?”) você já respondeu nos capítulos anteriores, quase sempre com “não” ou “parcialmente”. Neste capítulo, pela primeira vez, a resposta à primeira pergunta é um “sim” limpo — e é justamente esse “sim” que torna a segunda pergunta (“a assinatura confere?”) mais fácil de pular. Um perito de verdade nunca pula a segunda pergunta só porque a primeira deu certo; é aí, aliás, que mora o golpe mais eficaz contra qualquer processo de verificação — a falsificação parcial, escondida atrás de uma autenticação real.

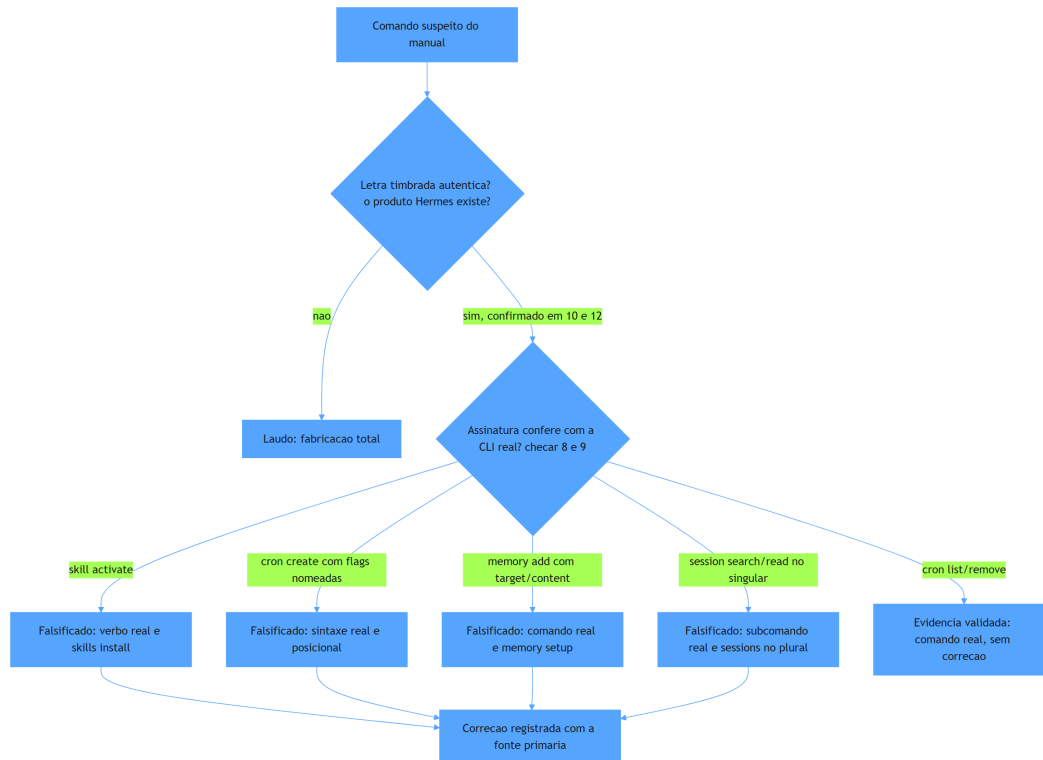


Figura 7: Cadeia de custodia pericial aplicada aos quatro subsistemas do Hermes Agent citados no manual auditado

4. Técnica

Checklist de Terminal: `hermes_pericia.sh`

O primeiro artefato deste capítulo não é um programa que você roda contra o Hermes instalado — é um checklist executável que confronta, lado a lado, cada linha fabricada do manual com o verbo real documentado, para você consultar antes de escrever qualquer automação em cima da CLI.

```
#!/usr/bin/env bash
# hermes_pericia.sh -- checklist de pericia dos comandos do Hermes Agent.
#
# O manual de referencia descreve `hermes skill activate`, `hermes cron
create
# --name/--schedule/--deliver`, `hermes memory add --target/--content` e
# `hermes session search/read`. Nenhum dos quatro existe na CLI real -- ver
# a documentacao oficial de CLI/Configuration da NousResearch [4][5].
```

```
#
# Uso: ./hermes_pericia.sh (sem argumentos -- imprime o laudo comparativo)

set -euo pipefail

echo "== LAUDO PERICIAL: Hermes Agent =="
echo

printf "%-45s | %-45s\n" "FABRICADO (manual)" "REAL (doc oficial)"
printf "%-45s+%-45s\n" "${printf -- '-%.0s' {1..45}}" "${printf -- '-%.0s' {1..45}}"
printf "%-45s | %-45s\n" "hermes skill activate <nome>" "hermes
skills install <id> | browse | list"
printf "%-45s | %-45s\n" "hermes cron create --name --schedule" "hermes
cron create \"<prompt>\" [--skill NOME]"
printf "%-45s | %-45s\n" "hermes memory add --target --content" "hermes
memory setup | status | off"
printf "%-45s | %-45s\n" "hermes session search --query" "hermes
sessions list | browse | export"
printf "%-45s | %-45s\n" "hermes session read --session-id" "hermes --
resume <id> (ou -r)"
echo
echo "Confirmados sem correcao: hermes cron list | hermes cron remove <id>"
echo "Config real (caminho confere): ~/.hermes/config.yaml"
echo " campos reais: model, terminal, memory, skills, agent,
prompt_caching, runtime, worktree"
echo " campos fabricados: system_prompt, cache_control.type"
```

Rode esse checklist mentalmente (ou literalmente, no terminal) antes de copiar qualquer trecho do manual original. Ele não substitui o `--help` — é o registro do laudo já fechado, para você não repetir a perícia do zero.

Tabela de Campo: As Outras Sete Ferramentas

Para o Pilar 2, o artefato é uma tabela de checklist rápido — o formato que você vai carregar para qualquer manual novo que citar uma dessas sete ferramentas:

Nome no manual	Produto real confirmado	Caminho de config no manual	Status do caminho nesta perícia
MiMo	MiMo Code (Xiaomi) [6]	<code>~/.mimo/config.yaml</code>	Não confirmado — real observado é <code>mimocode.json</code> por projeto [6]
Antigravity	Google Antigravity [11][12]	<code>~/.antigravity/settings.json</code>	Não confirmado em fonte primária
Google CLI	Gemini CLI [8][7]	<code>~/.google-ai/configurerc</code>	Nome e caminho incorretos — real é <code>~/.gemini/settings.json</code> [7]
Oh My Pi	omp — Oh My Pi [13][14]	<code>~/.oh-my-pi/config.toml</code>	Não confirmado em fonte primária
Freebuff	Freebuff CLI [15][16]	<code>~/.freebuff/config.json</code>	Não confirmado em fonte primária
Grok	Grok Build (xAI) [9][10]	<code>~/.grok/config.yaml</code>	Nome impreciso (“Grok” é o chat, não o CLI) e caminho não confirmado
Orca	Orca ADE [17][18]	<code>~/.orca/config.json</code>	Não confirmado em fonte primária

Note o padrão: em nenhuma das sete linhas o produto é ficção. O que varia é o grau de precisão do nome e a solidez da fonte para o caminho de arquivo — e é essa granularidade, “existe mas não está confirmado”, que separa este capítulo de um veredito binário simples de real-ou-falso.

Regra Geral: `verificar_cli` Antes de Automatizar

O Pilar 3 fecha com o protocolo mínimo que você vai levar para qualquer CLI nova, Hermes ou não. A ideia é simples: antes de escrever um script que chama subcomandos de uma ferramenta nova, rode o `--help` dela programaticamente e confira se os subcomandos que você pretende usar realmente aparecem na saída. Repare que essa regra não é exclusividade do Hermes: cada CLI que você já periciou neste livro tem a sua própria superfície real de configuração e de comando, documentada num lugar diferente — o Claude Code expõe as chaves reais em `~/.claude/settings.json` [19], o Codex CLI grava seus campos no nível raiz do `~/.codex/config.toml` [20], e o GNU Parallel documenta cada flag de paralelismo na própria man page do projeto [21]. Nenhuma dessas três documentações se parece com a das outras, e é exatamente por isso que rodar `--help` (ou abrir a doc oficial da versão instalada) tem que ser o primeiro passo, nunca o último, de qualquer automação nova.

```
#!/usr/bin/env python3
"""verificar_cli.py -- protocolo minimo de pericia para qualquer CLI nova.

Regra do capitulo: rodar --help ANTES de automatizar, nunca depois. Este
script generico roda `<binario> --help` (ou `<binario> <subcomando> --
help`)
e verifica se os comandos que voce pretende automatizar aparecem de fato na
saida oficial -- a mesma verificacao que teria barrado `hermes skill
activate`
antes de ir parar em um script de producao.

Uso:
    python verificar_cli.py hermes --comandos-esperados
    "skills,cron,memory,sessions"
    """
import argparse
import subprocess
import sys

def rodar_help(binario: str, subcomando: str | None = None) -> str:
    """Executa `<binario> [subcomando] --help` e devolve a saida
    combinada."""
    comando = [binario] + ([subcomando] if subcomando else []) + ["--help"]
    resultado = subprocess.run(
        comando, capture_output=True, text=True, timeout=15, check=False
    )
    return (resultado.stdout or "") + (resultado.stderr or "")

def verificar(binario: str, comandos_esperados: list[str]) -> dict:
    """Confere quais comandos esperados aparecem na saida real de --
    help."""
    try:
        saida = rodar_help(binario)
    except (FileNotFoundError, subprocess.TimeoutExpired) as erro:
        return {"erro": f"nao foi possivel rodar '{binario} --help':
{erro}"}

    saida_lower = saida.lower()
    confirmados = [c for c in comandos_esperados if c.lower() in
saida_lower]
    nao_confirmados = [c for c in comandos_esperados if c.lower() not in
saida_lower]
    return {
        "binario": binario,
        "confirmados_na_saida_do_help": confirmados,
```

```

    "nao_confirmados_pedem_pericia_manual": nao_confirmados,
}

def main() -> int:
    parser = argparse.ArgumentParser(description="Protocolo minimo: --help
antes de automatizar.")
    parser.add_argument("binario", help="nome do executavel a periciar,
ex.: hermes")
    parser.add_argument("--comandos-esperados", required=True,
                        help="lista separada por virgula dos subcomandos
que voce pretende usar")
    args = parser.parse_args()

    esperados = [c.strip() for c in args.comandos_esperados.split(",") if
c.strip()]
    laudo = verificar(args.binario, esperados)

    if "erro" in laudo:
        print(f"ERRO: {laudo['erro']}", file=sys.stderr)
        return 1

    print(f"Laudo de pericia para '{laudo['binario']}'")
    print(f"  confirmados no --help :
{laudo['confirmados_na_saida_do_help']}")
    print(f"  pedem pericia manual  :
{laudo['nao_confirmados_pedem_pericia_manual']}")
    return 0 if not laudo["nao_confirmados_pedem_pericia_manual"] else 2

if __name__ == "__main__":
    raise SystemExit(main())

```

Repare que `verificar_cli.py` não tenta adivinhar a sintaxe certa — ele só confirma se o termo que você pretende automatizar aparece na documentação oficial embutida no próprio binário. Isso já teria sinalizado, em segundos, que “skill activate” nunca aparece na saída real do `--help` do Hermes — muito antes de você colar o comando fabricado num cron de produção.

Uma extensão natural desse protocolo, para produtos tão recentes quanto os sete que fecham este capítulo, é registrar também a data de lançamento ou da última atualização documentada ao lado de cada veredito — não porque a data prove nada sozinha, mas porque ela explica o padrão “produto real, caminho pendente” sem exigir que você reabra a perícia inteira a cada nova versão instalada. Um caminho de configuração marcado como pendente em maio de 2026, como o do Oh My Pi ou da Freebuff CLI neste capítulo, pode estar plenamente documentado numa atualização de outubro do mesmo ano; sem a data ao lado do veredito, você não tem como saber se vale a pena checar de novo ou se a pendência é a mesma de meses atrás.

5. Aplica

Você está automatizando uma rotina noturna: quer que o Hermes ative uma skill de resumo de commits sempre que o repositório receber um push. cópia do manual `hermes skill activate resumo-commits`, cola no seu script de cron, e roda. A resposta do terminal é seca: `error: unrecognized subcommand 'skill'`. Sua primeira reação é pensar que a versão do Hermes está desatualizada, ou que falta algum plugin. Você tenta reinstalar o pacote inteiro. O erro persiste, idêntico.

O diagnóstico correto não está na instalação — está na sua fonte. Você abre `hermes skills --help` (no plural, dessa vez) e vê, na primeira linha da saída, os subcomandos reais: `install`, `browse`, `list` [4]. Não existe `activate`, porque skills no Hermes não são “ativadas” por comando externo — a maior parte delas nasce da própria observação do agente sobre o seu padrão de uso, gerada automaticamente a cada ciclo de chamadas [4]. A correção é reescrever o script com `hermes skills install resumo-commits` (se a skill já existir empacotada) ou, mais frequentemente, deixar o agente gerar a skill sozinho depois de repetir o padrão de trabalho algumas vezes — não forçar uma “ativação” manual que a ferramenta nunca ofereceu.

O mesmo golpe se repete, com uma variação, no cron. Você tenta `hermes cron create --name resumo-noturno --schedule "0 22 * * *" --prompt, es-`
`"resuma os commits do dia" --deliver origin`
 perando que as flags nomeadas — familiares de outras ferramentas de agendamento — funcionem aqui também. O parser da CLI simplesmente ignora ou rejeita as flags desconhecidas, porque a sintaxe real é posicional: `hermes cron create "resuma os commits do dia às 22h" --skill resumo-commits` [4][5]. Não há flag `--deliver`; a entrega do resultado é definida pela configuração do próprio agente, não por um parâmetro avulso do comando de criação.

A mesma armadilha se repete, numa forma mais silenciosa, com a tabela de campo das outras sete ferramentas. **Erro comum:** você lê a linha “Antigravity” na tabela do manual, reconhece o nome de uma IDE que apareceu em toda cobertura de lançamento recente, vê o caminho `~/.antigravity/settings.json` ao lado, e assume que o caminho também foi confirmado só porque o nome estava certo — afinal, “o produto é real, então o caminho provavelmente também é”. **Prática correta:** trate nome confirmado e caminho confirmado como dois veredictos independentes, em colunas separadas da mesma tabela de campo — nunca deduza um a partir do outro, mesmo quando os dois vêm juntos na mesma linha do manual. Se o objetivo for automatizar leitura ou escrita naquele arquivo específico, o único caminho responsável é abrir a própria ferramenta instalada e localizar o arquivo de configuração real no disco, ou aguardar a publicação de uma referência oficial de configuração equivalente à que já existe para o Gemini CLI — nunca codificar contra um caminho hipotético só porque ele “parece plausível” ao lado de um nome real.

Essa técnica de perícia tem um limite explícito que vale declarar antes de você fechar o capítulo: “não confirmado em fonte primária” **não é sinônimo de “aprovado”**, nem de “reprovado”. Para os seis caminhos de configuração sem confirmação pública das outras sete ferramentas — MiMo Code, Google Antigravity, Oh My Pi, Freebuff CLI, Grok Build e Orca ADE —, o veredito correto é suspender o julgamento até você mesmo rodar `--help` ou abrir o changelog oficial daquela versão específica instalada na sua máquina. Documentação de terceiros muda de versão para versão; um caminho confirmado hoje pode ter sido renomeado na próxima release, e um caminho hoje “não confirmado” pode estar corretíssimo na versão que você instalou — a perícia deste capítulo tem validade na data em que foi feita, nunca permanente.

Exercício

- ☐ Rode `hermes skills --help`, `hermes cron --help`, `hermes memory --help` e `hermes sessions --help` na sua instalação e confira se os subcomandos batem com o laudo deste capítulo
- ☐ Execute `hermes_pericia.sh` e adicione uma linha nova para o próximo comando do Hermes que você pretende automatizar, antes de escrevê-lo em produção
- ☐ Escolha uma das sete ferramentas da tabela de campo (a que você já usa ou pretende testar) e confirme o caminho real de configuração na documentação oficial dela, registrando o resultado
- ☐ Rode `verificar_cli.py` contra uma CLI Agêntica qualquer que você já tenha instalada, com uma lista de 3 a 5 subcomandos que você pretende automatizar

6. Conclusão

Este foi o capítulo mais denso da perícia inteira, e por um motivo específico: pela primeira vez, você confirmou a existência do produto de ponta a ponta — Hermes Agent é real — e mesmo assim encontrou quatro subsistemas inteiros de sintaxe fabricada por trás dessa confirmação. Você aprendeu que “o produto existe” e “a sintaxe confere” são dois laudos separados, e que o primeiro nunca dispensa o segundo. O panorama das outras sete ferramentas reforçou a mesma lição em escala menor: produto real, nome às vezes impreciso, caminho de configuração quase sempre pendente de confirmação — e a regra final, `verificar_cli` antes de automatizar, é o protocolo que você carrega para qualquer CLI nova que aparecer depois deste livro.

No Capítulo 8, você fecha o arco do livro consolidando tudo isso num checklist pessoal de perícia — o protocolo permanente que você aplica a qualquer manual novo, sem depender mais de um livro terceiro para separar fato de fabricação.

7. Referências Bibliográficas

- [1] RYOPPIPI. *ccusage: A CLI tool for analyzing Claude Code/Codex CLI usage from local JSONL files*. Disponível em: <https://github.com/ryoppiippi/ccusage>. Acesso em: 20 ago. 2026.
- [2] OLLAMA. *ollama/ollama*. Disponível em: <https://github.com/ollama/ollama>. Acesso em: 20 ago. 2026.
- [3] NOUSRESEARCH. *hermes-agent*. Disponível em: <https://github.com/NousResearch/hermes-agent>. Acesso em: 20 ago. 2026.
- [4] NOUSRESEARCH. *CLI Interface — Hermes Agent*. Disponível em: <https://hermes-agent.nousresearch.com/docs/user-guide/cli>. Acesso em: 20 ago. 2026.
- [5] NOUSRESEARCH. *Configuration — Hermes Agent*. Disponível em: <https://hermes-agent.nousresearch.com/docs/user-guide/configuration>. Acesso em: 20 ago. 2026.
- [6] XIAOMIMIMO. *MiMo-Code*. Disponível em: <https://github.com/XiaomiMiMo/MiMo-Code>. Acesso em: 20 ago. 2026.
- [7] GOOGLE-GEMINI. *gemini-cli*. Disponível em: <https://github.com/google-gemini/gemini-cli>. Acesso em: 20 ago. 2026.
- [8] GOOGLE. *Gemini CLI — Configuration*. Disponível em: <https://geminicli.com/docs/reference/configuration/>. Acesso em: 20 ago. 2026.
- [9] XAI. *Introducing Grok Build*. Disponível em: <https://x.ai/news/grok-build-cli>. Acesso em: 20 ago. 2026.
- [10] XAI-ORG. *grok-build*. Disponível em: <https://github.com/xai-org/grok-build>. Acesso em: 20 ago. 2026.
- [11] GOOGLE. *Build with Google Antigravity — our new agentic development platform*. Disponível em: <https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/>. Acesso em: 20 ago. 2026.
- [12] WIKIPEDIA. *Google Antigravity*. Disponível em: https://en.wikipedia.org/wiki/Google_Antigravity. Acesso em: 20 ago. 2026.
- [13] OH MY PI. *omp.sh*. Disponível em: <https://omp.sh/>. Acesso em: 20 ago. 2026.
- [14] BETTERSTACK. *Oh My Pi: a deep dive into the AI coding agent*. Disponível em: <https://betterstack.com/community/guides/ai/oh-my-pi-ai-coding-agent/>. Acesso em: 20 ago. 2026.
- [15] FREEBUFF. *Freebuff CLI*. Disponível em: <https://freebuff.com/cli>. Acesso em: 20 ago. 2026.
- [16] Y COMBINATOR. *Freebuff*. Disponível em: <https://www.ycombinator.com/companies/freebuff>. Acesso em: 20 ago. 2026.

- [17] ORCA. *Docs*. Disponível em: <https://www.onorca.dev/docs>. Acesso em: 20 ago. 2026.
- [18] ORCA. *Orca ADE*. Disponível em: <https://www.onorca.dev/>. Acesso em: 20 ago. 2026.
- [19] ANTHROPIC. *Claude Code settings*. Disponível em: <https://code.claude.com/docs/en/settings>. Acesso em: 20 ago. 2026.
- [20] OPENAI. *Configuration Reference — Codex CLI*. Disponível em: <https://developers.openai.com/codex/config-basic>. Acesso em: 20 ago. 2026.
- [21] GNU. *GNU Parallel*. Disponível em: <https://www.gnu.org/software/parallel/>. Acesso em: 20 ago. 2026.

Capítulo 8: O Perito Permanente: Seu Protocolo Para Qualquer Manual Novo

1. Introdução

No Capítulo 7, você fez a perícia mais densa do livro: mapeou, um a um, os comandos fabricados de skill, cron, memória e sessão que o manual v3.1 atribuía ao Hermes Agent, cruzando cada um com a CLI real documentada pelo próprio projeto [1]. Aquele capítulo fechou com uma regra simples, quase óbvia depois de dita: rode `--help` antes de automatizar uma ferramenta nova, nunca depois de o comando já ter falhado em produção. Esse é o último tijolo solto do caso. Chegou a hora de parar de tratar aquilo como uma regra isolada para o Hermes e transformá-la em protocolo permanente — algo que você carrega para o próximo manual, a próxima ferramenta, o próximo notebook.

Este capítulo fecha o livro em três movimentos. Primeiro, você consolida o checklist de perícia que já vem aplicando, capítulo após capítulo, num formato portátil que cabe em qualquer manual futuro — incluindo o que fazer quando a resposta é “não dá para confirmar”. Segundo, você versiona o que já foi confirmado (o `CLAUDE.md`, as configurações de terminal, os segredos de API) para nunca mais refazer a perícia do zero numa máquina nova. Terceiro, e mais importante: você mede a distância entre quem abriu este livro no Capítulo 1 e quem o fecha agora.

2. Explica

Todo o método deste livro repousa sobre uma única ideia: nem toda fonte de informação tem o mesmo peso de prova. Ao longo dos sete capítulos anteriores, você aplicou, sem nomear formalmente, uma hierarquia de três classes de fonte. A **Classe A** é a documentação oficial publicada pelo próprio fornecedor da ferramenta — a página de configurações do Claude Code [2], a referência de hooks da Anthropic [3], a configuração oficial do Gemini CLI [4]. A **Classe B** é o repositório oficial do projeto, o pacote no PyPI ou no npm — o código-fonte do `chezmoi`

[5], o pacote real do LLMingua no GitHub da Microsoft [6], o repositório do próprio `ccusage` [7]. A **Classe C** é tudo o que vem de terceiros: blogs, agregadores, cobertura de imprensa especializada — útil para entender como as pessoas usam uma ferramenta na prática — foi assim que você leu, no Capítulo 2, uma cobertura de terceiro sobre economia de custo com cache de prompt [8] — mas nunca a prova final de que um comando existe. O mesmo vale para um FAQ de comunidade sobre o `ccusage`: ajuda a entender o contexto de uso, mas o veredito sobre sintaxe exata continua vindo da Classe A ou B, mesmo quando o texto é claro e bem escrito [9].

A lição mais cara deste livro é que o padrão de erro mais perigoso não nasce da Classe C sozinha. Nasce de um produto genuíno, confirmável na Classe A ou B, ao qual alguém anexa uma sintaxe que nunca existiu. O manual auditado citava o *Hermes Agent* (produto real, Classe B confirmada [1]) com uma sintaxe de comando (`hermes skill activate`, `hermes memory add --target`) que nunca foi documentada na referência de CLI do projeto [10], nem na referência de configuração que descreve os campos reais de memória e skills [11]. Citava o *chezmoi* (real, Classe A confirmada [5]) com um subcomando `commit` que o projeto nunca implementou. Em ambos os casos, o nome familiar do produto é o que baixa a guarda do operador — ninguém desconfia de um comando anexado a uma ferramenta que ele já usa há meses. É exatamente esse mecanismo psicológico, não a falta de familiaridade, que faz a fabricação parcial ser mais perigosa que a fabricação total.

Isso leva à regra mais contraintuitiva do checklist: **NÃO VERIFICÁVEL não é sinônimo de aprovado**. Ao longo do livro, você viu casos em que o produto é real e a documentação primária simplesmente não confirma (nem desmente) um detalhe específico — o caminho de configuração da Google Antigravity, por exemplo, não tem confirmação em nenhuma fonte primária consultada, mesmo o produto sendo genuíno [4]. A tentação natural é tratar “não achei nada contra” como “então deve estar certo”. O protocolo correto é o oposto: um item Não Verificável fica retido, como evidência pendente, até que uma fonte primária — não uma dedução, não uma suposição plausível — confirme ou refute. Ele nunca migra automaticamente para “aprovado” só porque o tempo passou sem contestação.

A mesma hierarquia de evidência explica por que a técnica de cache de contexto que você usa desde o Capítulo 2 continua de pé até aqui, sem nenhuma correção. Manter o `CLAUDE.md` estático, com a mesma ordem de blocos em toda sessão, maximiza o acerto de cache pela razão mais literal possível: o mecanismo de cache da Anthropic funciona sobre um prefixo idêntico byte a byte no início da requisição — qualquer alteração no texto anterior a um ponto invalida o cache a partir dali [12]. Isso não é opinião de blog de terceiros: é a mecânica documentada oficialmente, e o desconto de leitura de cache chega a 90% do preço padrão de input quando o prefixo se mantém intacto [12]. Um script que calcula o hash SHA-256 do seu `CLAUDE.md` antes de cada sessão para detectar mudança silenciosa não depende de nenhuma ferramenta fabricada — é utilitário padrão de qualquer sistema Unix, e por isso sobrevive intacto à perícia.

Vale revisitar, em ritmo rápido, o painel de casos que já sustentou cada veredito deste livro — é o material bruto sobre o qual o checklist foi treinado. O Capítulo 2 confirmou que o Aider liga cache de prompt automático do provedor através de um campo real do seu arquivo

de configuração, sem inventar granularidade que a ferramenta não expõe [13]. O Capítulo 6, na mesma linha, confirmou que o catálogo de modelos locais da Ollama é público e nomeado com precisão, o que tornou o fallback offline uma escolha auditável, não uma aposta [14].

A CLI real por trás desse fallback também resistiu à perícia sem nenhuma correção — o binário documentado é exatamente o que o manual usa [15]. O mesmo vale, num tema diferente, para o GPTCache: uma biblioteca genuína de cache semântico, ainda que o manual tenha anexado a ela comandos de terminal que o projeto nunca documentou [16].

O Capítulo 7 mostrou a versão mais concentrada desse padrão, ao separar dois produtos reais e completamente distintos que o manual tratava como se fossem o mesmo pacote: `agenttrace`, biblioteca de observabilidade em Python [17], e `agentlytics`, painel de análise em Node — dois ecossistemas diferentes, duas fontes primárias diferentes [18].

Mesmo os casos mais discretos reforçam a régua. O `llama.cpp` continua sendo o mesmo projeto real de sempre, só com o binário renomeado numa atualização do próprio mantenedor [19]; o vLLM tem uma CLI dedicada, documentada linha a linha na referência oficial do projeto [20].

O Codex CLI da OpenAI usa uma seção do arquivo de configuração diferente da que o manual descreveu, mas o arquivo em si e o restante da sintaxe batem com a documentação [21]. O OpenCode, por sua vez, grava as próprias preferências num arquivo com nome e campos documentados na referência do projeto [22].

Fechando o painel: o gateway de modelos gratuitos que o OpenCode expõe — o Zen — é real, mas exige conferência de nomenclatura a cada consulta ao catálogo vigente, porque a lista muda [23]. E, entre as sete outras ferramentas mapeadas no Capítulo 7, o `gemini-cli` do Google é o único caso da lista com repositório e configuração confirmados nas duas pontas por fonte primária [24].

Colocado lado a lado, esse painel de casos forma o que a auditoria completa do manual chama de núcleo genuíno e ensinável, ordenado pela solidez da fonte que o confirma: o prompt caching real da Anthropic, com seus multiplicadores de preço documentados oficialmente [12]; o paralelismo real via `asyncio / xargs / GNU Parallel` combinado com circuit breaker e backoff exponencial, o único capítulo do livro que não precisou de nenhuma correção; o Ollama com catálogo de modelos e fallback local auditável [14][15]; o `chezmoi` para versionamento de dotfiles, com apenas o instalador e o `commit` corrigidos [5]; o LLMLingua e o GPTCache como bibliotecas Python genuínas de compressão de prompt e cache semântico, usadas de forma errada no manual como se tivessem CLI própria [6][16]; o `ccusage` como ferramenta real de análise de uso via npm [7]; e o próprio Hermes Agent, com memória, skills, cron e delegação todos confirmados como mecanismo, mesmo com a sintaxe de comando fabricada [1][10][11]. Sete itens, sete fontes primárias diferentes, um padrão idêntico se repetindo em cada um: produto genuíno, detalhe de superfície errado. Esse mesmo padrão de sete casos é o que, mais adiante nesta conclusão, se traduz nos números fechados da perícia inteira.

3. Ilustra

Pense num perito de verdade — não o que investiga um caso específico, mas o que a corregedoria chama para treinar os novos. Ele não ensina “como resolver o caso do manual v3.1”. Ele ensina um procedimento de bancada que serve para qualquer documento suspeito que chegar depois: primeiro, identifique de onde o documento diz que veio; segundo, classifique essa origem numa das três gavetas (letra timbrada oficial, cópia de cartório, boato de rua); terceiro, procure o sinal de alerta específico — a assinatura bonita demais, o carimbo no lugar errado; quarto, só então lavre o laudo. É esse procedimento — não o caso individual — que você carrega na maleta para a próxima perícia.

A parte mais traiçoeira do checklist merece uma segunda cena. Imagine que, depois de investigar um documento a fundo, o laboratório devolve um resultado “inconclusivo” — nem confirma, nem refuta a autenticidade. Um perito de verdade nunca arquiva um laudo inconclusivo na gaveta de “autenticado”: ele fica na gaveta de “pendente”, visível, esperando um novo exame, porque tratá-lo como aprovado por omissão é exatamente o erro que deixa passar uma falsificação. É esse mesmo instinto — pendência visível, nunca aprovação por padrão — que você aplica a qualquer item marcado NÃO VERIFICÁVEL neste livro ou em qualquer manual futuro.

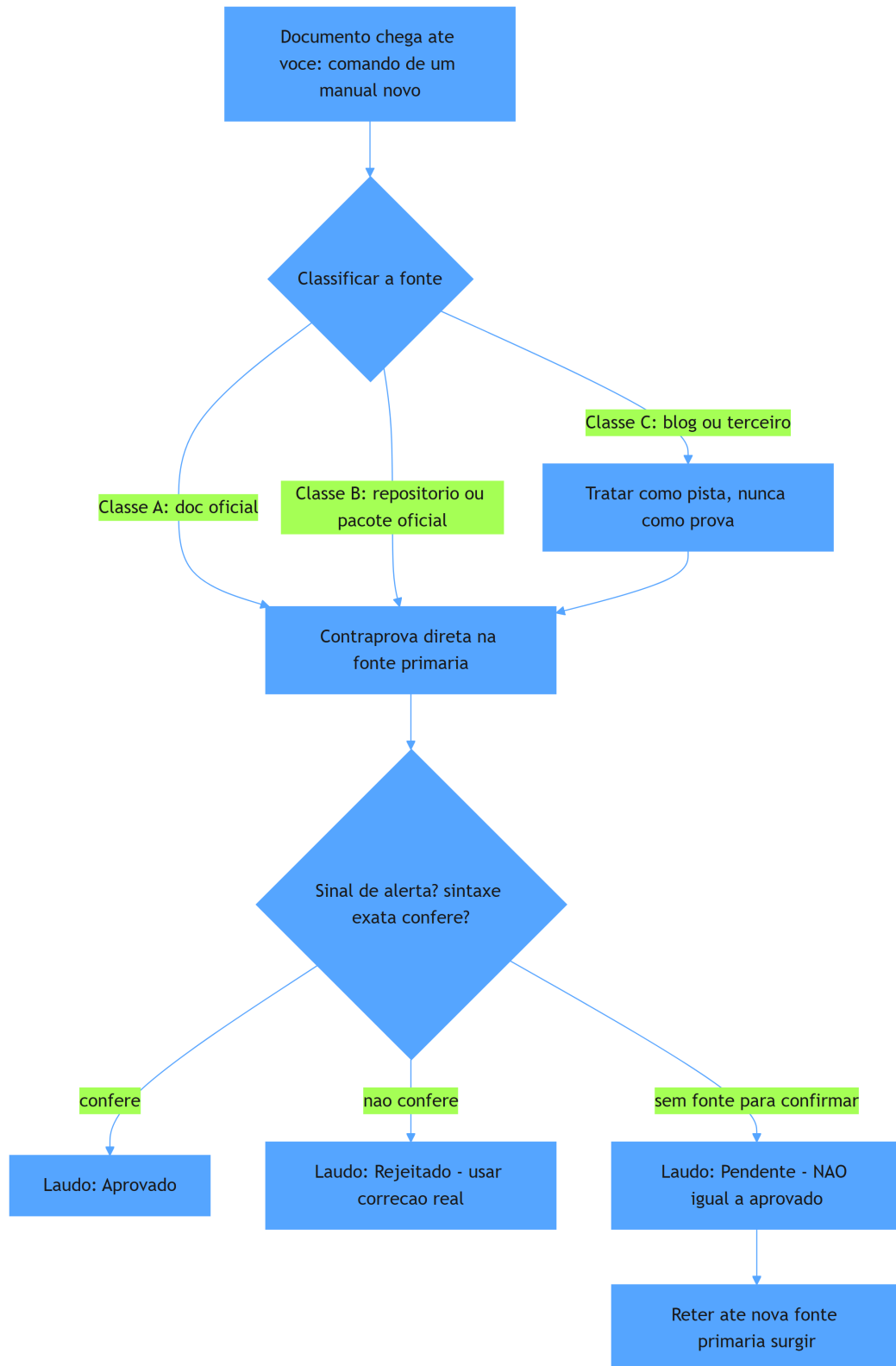


Figura 8: Protocolo de perícia portátil aplicado a qualquer comando de manual novo

Como Perito de Configuração Agêntica, este é o momento em que você percebe que nunca mais vai precisar confiar cegamente num tutorial só porque o nome da ferramenta é conhecido.

4. Técnica

O Protocolo de perícia em Formato Portátil

O checklist só é útil se puder ser aplicado sem este livro do lado. A forma mais simples de torná-lo portátil é um modelo de registro estruturado — um arquivo curto que você preenche a cada comando suspeito de um manual futuro, seja ele sobre IA, sobre banco de dados ou sobre infraestrutura de nuvem. O YAML abaixo é o mesmo esqueleto que sustentou, informalmente, cada linha da auditoria feita nos Capítulos 1 a 7: qual comando está sob suspeita, de que classe de fonte ele afirma vir, qual sinal de alerta apareceu e qual foi o veredito final.

```
# protocolo-pericia.yaml
# Preencha um bloco destes por comando suspeito de um manual/tutorial novo.
comando_sob_suspeita: "chezmoi commit -m \"atualiza dotfiles\""
produto_citado: "chezmoi"
fonte_classificada:
  classe: "A"          # A = doc oficial | B = repo/pacote oficial | C =
terceiro
  origem: "https://www.chezmoi.io/install/"
sinais_de_alerta:
  - "subcomando nao aparece na lista de comandos do --help oficial"
  - "nome do produto e real, o que reduz a desconfianca natural do
operador"
contraprova_realizada: true
veredito: "rejeitado"  # aprovado | pendente | rejeitado
correcao_aplicada: "chezmoi cd && git commit -m \"atualiza dotfiles\""
observacao: "chezmoi nao tem subcomando commit nativo; commit acontece no
repositorio git do diretorio-fonte"
```

Repare que o campo `veredito` tem três, não dois, valores possíveis — `pendente` existe como categoria de primeira classe, não como um `aprovado` disfarçado. Isso é o checklist forçando, na estrutura do próprio arquivo, a regra da seção Explica: sem fonte primária, o item não pode migrar para `aprovado` só por conveniência.

Restaurando o Cofre Confirmado com chezmoi (Sintaxe Real)

O manual v3.1 acertou que um gerenciador de dotfiles é a ferramenta certa para não repetir a perícia do zero em cada notebook novo — e o `chezmoi` é, de fato, real e maduro para essa tarefa [5]. Onde ele errou foi em dois pontos específicos de sintaxe, e é exatamente aí que o checklist do Pilar 1 precisa ser aplicado antes de copiar qualquer coisa. O instalador `curl -sL https://git.io/chezmoi | sh` usa um encurtador de link (`git.io`) que o GitHub

descontinuou para novos links desde 2022 — funciona por enquanto, mas depende de um redirecionamento legado sem garantia de permanência; o instalador oficial documentado hoje é outro [5]. E `chezmoi commit -m "..."` simplesmente não existe como subcomando nativo: o `chezmoi` gerencia o diretório-fonte dos seus dotfiles, mas o commit em si acontece pelo Git dentro desse diretório [5].

A sessão abaixo mostra a sequência real, do instalador correto ao commit correto, incluindo a verificação de hash SHA-256 do `CLAUDE.md` restaurado e a injeção de segredo via 1Password CLI — real e compatível com a sintaxe documentada da versão 2 [25].

```
$ sh -c "$(curl -fsLS https://get.chezmoi.io)"
chezmoi: instalado em ~/.local/bin/chezmoi

$ chezmoi init
chezmoi: repositorio fonte inicializado em ~/.local/share/chezmoi

$ chezmoi add ~/.claude/CLAUDE.md
$ chezmoi add ~/.claude/settings.json

$ chezmoi status
A .claude/CLAUDE.md
A .claude/settings.json

$ sha256sum ~/.claude/CLAUDE.md
a3f2c9e7d1b04f6c8a2e9d5b7c1f0a4e6d8b2c4f /home/voce/.claude/CLAUDE.md

$ chezmoi cd
$ git commit -am "cofre confirmado: CLAUDE.md e settings.json do Capitulo 8"
$ exit

$ chezmoi git commit -- -m "alternativa sem sair do diretorio fonte, mesmo efeito"

$ op signin
Digite sua senha mestra do 1Password: *****
Sessao autenticada.

$ op item create --title "API Key" --vault Private --generate-password
Criado item "API Key" no cofre "Private".

$ op read "op://Private/API Key/credential"
sk-ant-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Repare que `chezmoi status` entra na sequência exatamente entre o `add` e o `commit`: é o comando real que mostra, letra por caminho (**A** de adicionado, **M** de modificado), o que vai mudar no diretório-fonte antes de você confirmar qualquer coisa — a mesma disciplina de

“nunca aplicar sem revisar antes” que perpassa este livro inteiro, aplicada agora ao seu próprio cofre de configuração [5]. E o `op item create` que precede a leitura do segredo fecha o ciclo completo do 1Password CLI: não basta saber *ler* uma credencial já guardada — o mesmo binário documentado também cria o item do zero, com `--generate-password` delegando ao próprio cofre a geração da senha, sem que você precise inventar ou digitar uma manualmente [25].

Duas observações fecham este pilar. A primeira: se você preferir não digitar `git commit` manualmente toda vez, o próprio `chezmoi` documenta um campo `autoCommit: true` na sua configuração — uma terceira forma real de resolver o mesmo problema que o manual tentou nomear com um subcomando inexistente [5]. A segunda: o segredo (a chave de API) nunca entra no `CLAUDE.md` versionado em texto puro — ele é lido em tempo de uso direto do cofre do 1Password [25], o que significa que o repositório de dotfiles pode ser público ou compartilhado com a equipe sem vaziar nenhuma credencial.

O Script de Alerta Rápido

Como consolidação final, o script abaixo automatiza a primeira triagem do checklist: ele varre um trecho de texto (colado de um manual ou tutorial novo) em busca de padrões que, ao longo deste livro, mais vezes precederam um veredito Fabricado ou Parcialmente Correto — nomes de subcomando muito específicos ao lado de um verbo pouco comum, ou caminhos de configuração dentro de `~/ .config/<produto>/` que soam plausíveis demais. Ele não substitui a perícia manual; serve como primeiro filtro, um lembrete automático para abrir o `--help` antes de copiar.

```
#!/usr/bin/env bash
# auditar-comando.sh – triagem heurística de comandos suspeitos em um
# manual novo
# Uso: auditar-comando.sh < trecho-do-manual.txt

set -euo pipefail

PADROES_SUSPEITOS=(
  '--target [a-z]+ --content'
  'skill activate'
  '~/\.config/[a-z-]+/config\.(json|yaml)'
  'commit -m'
  'session (search|read) --'
)

echo "== Triagem heurística (Capítulo 8) =="
while IFS= read -r linha; do
  for padrao in "${PADROES_SUSPEITOS[@]}; do
    if echo "$linha" | grep -qE "$padrao"; then
      echo "SUSPEITO: $linha"
    fi
  done
done
```

```

    echo "    -> classifique a fonte (A/B/C), rode --help, preencha o
    protocolo-pericia.yaml"
    fi
done
done

echo "== Fim da triagem: nenhum item aqui foi aprovado automaticamente =="

```

Blindagem Extra: Grep de Segredo Antes do Commit

O cofre versionado do Pilar 2 (`chezmoi` + Git) só cumpre a promessa de “restaurar em minutos numa máquina nova” se nenhum segredo em texto puro entrar no repositório por descuido — porque um repositório de dotfiles, diferente de uma credencial isolada, costuma ser compartilhado com a equipe ou até publicado. O script abaixo é o mesmo tipo de triagem heurística do `auditar-comando.sh`, aplicado agora à sua própria mudança antes do commit, não ao texto de um manual de terceiros: ele varre o diff staged do diretório-fonte do `chezmoi` em busca dos dois padrões mais comuns de segredo esquecido — uma chave de API colada por engano e uma chave privada completa.

```

#!/usr/bin/env bash
# blindar-commit.sh -- grep de segurança antes de commitar o cofre de
dotfiles.
# Uso: dentro do diretorio-fonte do chezmoi (apos `chezmoi cd`), antes do
commit:
#   ./blindar-commit.sh

set -euo pipefail

PADROES_SEGREDO=(
    'sk-[a-zA-Z0-9_-]{20,}'
    '-----BEGIN [A-Z]*PRIVATE KEY-----'
    'AKIA[0-9A-Z]{16}'
)

echo "== Blindagem de segredo antes do commit =="

encontrou=0
for padrao in "${PADROES_SEGREDO[@]}; do
    if git diff --cached | grep -qE "$padrao"; then
        echo "BLOQUEADO: padrao de segredo encontrado no diff staged ($padrao)"
        encontrou=1
    fi
done

```

```

if [ "$encontrou" -eq 1 ]; then
    echo "Remova o segredo do arquivo e injete via 'op read' em tempo de
uso."
    exit 1
fi

echo "Nenhum padrao de segredo encontrado. Commit liberado."

```

Isso não substitui o hábito de nunca digitar o segredo no arquivo em primeiro lugar — mas funciona como a mesma rede de segurança automática que o `auditar-comando.sh` oferece para comandos suspeitos de um manual: um filtro de primeira linha, nunca a prova final. A prova final, aqui como em todo o resto do livro, continua sendo a fonte primária — neste caso, a confirmação visual de que o `git diff --cached` não carrega nada que devesse ter ficado só no cofre do 1Password.

5. Aplica

Imagine a cena: você acabou de entrar num time novo, e alguém compartilha no canal um artigo recém-publicado prometendo “o guia definitivo” de automação para uma ferramenta de agente que você já usa no dia a dia. O artigo tem comandos formatados com sintaxe realista, nomes de flag plausíveis, até um bloco de configuração YAML bonito. Você reconhece o nome do produto — usa ele há meses — e o instinto manda copiar e colar direto no terminal, porque “é a mesma ferramenta que eu já conheço, só um recurso novo que eu não tinha visto”. Você cola. O terminal devolve `unrecognized flag` ou, pior, aceita silenciosamente e não faz nada do que o artigo prometeu.

O diagnóstico é exatamente o padrão da seção Explica: o nome familiar do produto desarmou sua desconfiança antes mesmo de você ler a segunda linha do comando. Você tratou familiaridade com o produto como se fosse prova de que a sintaxe exata estava certa — mas são duas coisas completamente diferentes, e é aí que mora quase todo erro deste livro, do Hermes Agent ao `chezmoi`. A correção não é desconfiar de tudo que parece bom demais; é aplicar o mesmo protocolo de sempre, sem exceção para ferramentas “que você já conhece”: classificar a fonte do artigo (quase certamente Classe C), procurar a confirmação na Classe A ou B correspondente (a documentação oficial ou o repositório do projeto) e só então rodar o comando.

Vale registrar até onde esse protocolo escala sozinho. Para um punhado de comandos novos por semana, a checagem manual do checklist é rápida e não pesa na rotina. O gargalo aparece quando o volume cresce — dezenas de comandos novos por dia, num time inteiro adotando ferramentas diferentes — porque aí o tempo humano de contraprova na fonte primária vira o fator limitante, não a vontade de fazer a perícia direito. Nesse cenário, o script de

triagem automatiza só a primeira filtragem (sinalizar o que parece suspeito); ele nunca substitui a confirmação humana na fonte oficial, porque automatizar o próprio veredito reintroduziria o mesmo risco que o checklist existe para eliminar.

Armadilhas comuns que valem a pena revisar antes de fechar este livro:

- Tratar “o produto é real, eu já uso ele” como equivalente a “esse comando específico é real”.
- Aceitar um resultado NÃO VERIFICÁVEL como se fosse aprovado só porque ninguém desmentiu.
- Copiar configuração de um blog (Classe C) sem checar se ela ainda bate com a versão instalada — ferramentas mudam de schema entre versões, e o artigo pode estar correto para uma versão antiga.
- Versionar segredo (chave de API, token) direto no arquivo de dotfiles em vez de injetá-lo via cofre em tempo de uso.

Exercício

- ☐ Rode `--help` (ou equivalente) numa ferramenta de agente que você já usa antes de copiar qualquer comando novo de um tutorial está semana
- ☐ Classifique 3 fontes que você consulta no trabalho hoje como Classe A, B ou C, usando os critérios deste capítulo
- ☐ Preencha um `protocolo-perícia.yaml` para 1 comando que você copiou de algum lugar sem checar nos últimos 30 dias
- ☐ Rode `chezmoi init` num diretório de teste e confirme o hash SHA-256 do seu próprio `CLAUDE.md` antes e depois de uma edição
- ☐ Identifique, no seu ambiente de trabalho, algo que você trata hoje como “aprovado” mas que na verdade nunca teve fonte primária confirmando — reclassifique como pendente

6. Conclusão

Você fecha este livro com três ferramentas permanentes, não com uma lista de correções para o manual v3.1. A primeira é o checklist de perícia — hierarquia de fontes A/B/C, atenção redobrada ao padrão “produto real com sintaxe fabricada”, e a regra de que NÃO VERIFICÁVEL fica pendente, nunca aprovado. A segunda é o cofre versionado: `CLAUDE.md` estático para preservar cache, hash SHA-256 para detectar mudança silenciosa, `chezmoi` com a sintaxe real (nunca o `commit` fabricado) para restaurar tudo isso em minutos numa máquina nova, e o `1Password CLI` para que nenhum segredo trafegue em texto puro. A terceira é a mais difícil de medir, mas a mais real: a virada de protagonismo.

O caso deste livro, encerrado agora, tem um placar final. Dos itens auditados no manual v3.1, ao longo dos oito capítulos, 26 foram confirmados batendo com a fonte primária, 13 eram

parcialmente corretos — produto real, detalhe errado —, 18 foram integralmente fabricados e 4 permanecem não verificáveis até hoje. Isso significa que perto de 40% do manual original tinha núcleo técnico genuíno e ensinável — o mesmo núcleo que sustentou, por exemplo, o mecanismo de cache real que você usa desde o Capítulo 2 [12] e o gerenciador de dotfiles que acabou de restaurar seu cofre de configurações [5]. Não é pouco — é o suficiente para explicar por que tanta gente confiou nele. E não é muito — é o suficiente para explicar por que copiar e colar sem perícia teria quebrado, silenciosamente, boa parte dos fluxos de trabalho de quem seguiu o manual à risca.

Você não fecha este livro sabendo mais comandos. Fecha sabendo o que fazer quando o próximo manual, sobre uma ferramenta que ainda nem existe hoje, chegar até você.

7. Referências Bibliográficas

[1] NOUSRESEARCH. *hermes-agent*. Disponível em: <https://github.com/NousResearch/hermes-agent>. Acesso em: 20 ago. 2026.

[2] ANTHROPIC. *Claude Code settings*. Disponível em: <https://code.claude.com/docs/en/settings>. Acesso em: 20 ago. 2026.

[3] ANTHROPIC. *Hooks reference*. Disponível em: <https://code.claude.com/docs/en/hooks>. Acesso em: 20 ago. 2026.

[4] GOOGLE. *Gemini CLI — Configuration*. Disponível em: <https://geminicli.com/docs/reference/configuration/>. Acesso em: 20 ago. 2026.

[5] CHEZMOI. *Install e Daily operations*. Disponível em: <https://www.chezmoi.io/install/>. Acesso em: 20 ago. 2026.

[6] MICROSOFT. *LLMLingua* (repositório oficial). Disponível em: <https://github.com/microsoft/LLMLingua>. Acesso em: 20 ago. 2026.

[7] RYOPPIPI. *ccusage: A CLI tool for analyzing Claude Code/Codex CLI usage from local JSONL files*. Disponível em: <https://github.com/ryoppippi/ccusage>. Acesso em: 20 ago. 2026.

[8] AGENTBRISK. *Prompt Caching Deep Dive: How to Cut Anthropic API Costs by 90%*. Disponível em: <https://agentbrisk.com/blog/prompt-caching-deep-dive-2026/>. Acesso em: 20 ago. 2026.

[9] CLAUDELOG. *What is ccusage tool for Claude Code*. Disponível em: <https://claudelog.com/faqs/what-is-ccusage-tool/>. Acesso em: 20 ago. 2026.

[10] NOUSRESEARCH. *CLI Interface — Hermes Agent*. Disponível em: <https://hermes-agent.nousresearch.com/docs/user-guide/cli>. Acesso em: 20 ago. 2026.

- [11] NOUSRESEARCH. *Configuration — Hermes Agent*. Disponível em: <https://hermes-agent.nousresearch.com/docs/user-guide/configuration>. Acesso em: 20 ago. 2026.
- [12] ANTHROPIC. *Prompt caching*. Disponível em: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>. Acesso em: 20 ago. 2026.
- [13] AIDER. *Prompt caching*. Disponível em: <https://aider.chat/docs/usage/caching.html>. Acesso em: 20 ago. 2026.
- [14] OLLAMA. *Library* (catálogo de modelos). Disponível em: <https://ollama.com/library>. Acesso em: 20 ago. 2026.
- [15] OLLAMA. *ollama/ollama*. Disponível em: <https://github.com/ollama/ollama>. Acesso em: 20 ago. 2026.
- [16] ZILLIZTECH. *GPTCache: A Library for Creating Semantic Cache for LLM Queries* (docs/usage.md). Disponível em: <https://github.com/zilliztech/GPTCache/blob/main/docs/usage.md>. Acesso em: 20 ago. 2026.
- [17] TENSORSTAX. *agenttrace: lightweight observability library to trace and evaluate agentic systems*. Disponível em: <https://github.com/tensorstax/agenttrace>. Acesso em: 20 ago. 2026.
- [18] F. *agentlytics: Comprehensive analytics dashboard for AI coding agents*. Disponível em: <https://github.com/f/agentlytics>. Acesso em: 20 ago. 2026.
- [19] GGML-ORG. *llama.cpp*. Disponível em: <https://github.com/ggml-org/llama.cpp>. Acesso em: 20 ago. 2026.
- [20] VLLM PROJECT. *CLI Reference — serve*. Disponível em: <https://docs.vllm.ai/en/stable/cli/serve/>. Acesso em: 20 ago. 2026.
- [21] OPENAI. *Configuration Reference — Codex CLI*. Disponível em: <https://developers.openai.com/codex/config-basic>. Acesso em: 20 ago. 2026.
- [22] OPENCODE. *Config*. Disponível em: <https://opencode.ai/v2/docs/config>. Acesso em: 20 ago. 2026.
- [23] OPENCODE. *Zen*. Disponível em: <https://opencode.ai/docs/zen/>. Acesso em: 20 ago. 2026.
- [24] GOOGLE-GEMINI. *gemini-cli*. Disponível em: <https://github.com/google-gemini/gemini-cli>. Acesso em: 20 ago. 2026.
- [25] 1PASSWORD. *op signin — CLI command reference*. Disponível em: <https://developer.1password.com/docs/cli/reference/commands/signin>. Acesso em: 20 ago. 2026.

Tokens Sob Perícia

Apresentar o caso: um manual de 15 LABs sobre economia de tokens circula com autoridade, mas metade dos comandos nunca funcionaria se copiados e colados. Estabelecer o método do livro (hierarquia de fontes A/B/C, cruzamento contra documentação oficial) e o achado central que orienta tudo: ferramenta real com sintaxe fabricada e mais perigoso que uma fabricação total, porque o nome familiar destrava a confiança do operador.

Heverton Eduardo Peres